Adelaide University

ENGI 4900

Engineering Research Project

Final Report

Detecting high-speed airborne vehicles from space using machine learning

Authors

Mithusha Kulatunga	Leah Menzies	Brian Park
a1879572	a1851923	a1832453

Supervisors

Adelaide University: Dr. Yee Wei Law, Dr. Jordan Kildare, Dr. Michael Evans

Northrop Grumman Australia: Dr. Dushy Tissainayagam

Project ID: 2026s1-EME_EE-YWL-UG-14552

School of Electrical and Mechanical Engineering

College of Engineering and Information Technology

Adelaide University

November 2026

Summary of AI Tool Use

Categories of AI tool use in this assignment are as indicated in the following table. The detailed Statement of AI Tool Use follows the References.

Table 1: Summary of AI tool use.

AI tool use	**Used**	**Extent of use**
Spelling	✓	Moderate
Grammar	✓	Moderate
Other copy editing	✓	Moderate
Literature review support		
Visuals	✓	Moderate
Substantive editing	✓	Low
Idea generation		
Translation		
Research technical support	✓	Low
No AI tool was used		

Executive Summary

A central challenge in hypersonic defence is reliable detection and tracking during the early stages and midcourse of flight. Ground-based radar loses sight of these vehicles behind the curvature of the Earth, and existing overhead infrared systems are constrained by field of view and background clutter (Dahlgren 2023). Satellite-based machine learning detection offers an alternative, but it must find small, dim, fast-moving targets that may cover only a few pixels of a thermally complex scene. Working with Northrop Grumman Australia, this project set out to determine whether a modern neural network building block could improve that detection.

The Progress Report established the first half of this work. A literature review of four architectural families selected the Mamba selective state space model, because it mixes information according to image content at a cost that grows only linearly with input size. A formal technical description then set out how it operates. This report completes the story by implementing Mamba-based designs, placing them in a working detector, and measuring what they achieve.

The chosen host was MOCID, a published infrared detector that follows moving small targets across five video frames using a Mamba-based displacement module. MOCID was reported to reach 95.93% average precision on DAUB, a public infrared video dataset of small aircraft. The team therefore began by rebuilding a dependable version of it, and this became the project's first major achievement. The inherited implementation collapsed unpredictably during training and could not reproduce the published results. Systematic experiments traced the failures to three separate causes in the training settings, the evaluation procedure and the transition between training stages. Correcting them produced a stable plain single-frame baseline scoring 88.83% and a complete MOCID scoring 90.12%. This repaired detector gave every later comparison a trustworthy reference, along with verification tools that any future team can reuse.

Three new displacement modules were then designed, implemented and trained from the same starting point, changing only that module. The first, inspired by MambaIR, added local image processing and channel attention around the scan, and reached 89.53%. The second, inspired by CFMW, added a branch giving each comparison between frames a view of the whole scene, and reached 90.27%. The third, inspired by MambaTree, replaced the fixed row-by-row scan with a tree that follows image content, and reached 90.19%. An additional control run showed that the tree scan's small gain came mainly from a smaller internal memory, because the original scan matched it at the same memory size.

The central finding is that none of the three designs improved detection beyond the variation of about 0.3 percentage points seen between consecutive evaluations of a single run. All three kept the same broad accuracy as the repaired detector, while the extra processing added cost. The displacement module adds about 10 milliseconds to the 14.0 milliseconds per clip of the detector without it, and the MambaIR version adds a further 4 milliseconds. The tree scan needed about 50% more time per training step. The most revealing result came from scoring the CFMW model with its displacement module switched off. It then scored higher at 48 of 50 evaluations, reaching 91.99%. This points away from the design of the scan and towards the way displacement information reaches the rest of the detector, which is where future work is most likely to succeed.

The project therefore delivers a working MOCID framework, three fully evaluated Mamba adaptations, a clear explanation of why they did not raise accuracy, and a verified procedure for fair comparison. It does not yet demonstrate performance on space-based imagery, because all measurements use the public dataset. The recommended next steps are to redesign how displacement information is combined, repeat the comparison across several training seeds, and evaluate the best design on representative high-speed imagery in the industry partner's benchmark.

Contents

Summary of AI Tool Use ii

Executive Summary iii

Nomenclature xii

Acknowledgments xv

Authorship and Contributions Statement xvi

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Motivation . . . 2
- 1.3 Aims and Scope . . . 2
- 1.4 Objectives . . . 3
- 1.5 Document Overview . . . 3

2 Identifying and Justifying the Building Blocks 4
- 2.1 Introduction and Scope . . . 4
- 2.2 Challenges in Small Object Detection . . . 4
- 2.3 Recurrent Neural Networks . . . 5
- 2.4 Convolution Neural Networks . . . 5
- 2.5 Transformers and Attention-Based Models . . . 5
- 2.6 State Space Models . . . 5
- 2.7 Domain-Specific Evidence . . . 8
- 2.8 Knowledge Gaps . . . 10
- 2.9 Justified Selection for Mamba . . . 11
- 2.10 MOCID: A Domain-Specific Precedent . . . 11
- 2.11 Justification of the Candidate Adaptation . . . 12
- 2.12 Conclusion . . . 12

3 Theory and Technical Description of Mamba 13
- 3.1 Overview . . . 13
- 3.2 State Space Models . . . 13
- 3.3 Recurrent and Convolutional Representations . . . 16
- 3.4 The Selection Mechanism . . . 18
- 3.5 Hardware-Aware Implementation . . . 19
- 3.6 The Mamba Block . . . 21
- 3.7 From a sequence to an image block . . . 21
- 3.8 Suitability for Small-Object Detection . . . 22
- 3.9 Interface requirements . . . 22
- 3.10 Summary . . . 22

4 MOCID and the Implemented Detection Architecture 23
- 4.1 Overview . . . 23
- 4.2 Clip-level feature extraction . . . 24
- 4.3 Frame-pair displacement processing . . . 24
- 4.4 Fusion, detection, and model variants . . . 25
- 4.5 How the FISTA and DAM operations work . . . 25
- 4.6 Two-stage training . . . 27
- 4.7 Summary . . . 27

5 Rebuilding MOCID and Developing the Displacement Modules **28**
5.1 Starting point and investigation strategy . . . 28
5.2 Structural checks and the first complete run . . . 28
5.3 Separating training faults from evaluation faults . . . 29
5.4 Why these faults arise . . . 29
5.5 Establishing a stronger baseline . . . 31
5.6 Correcting the transition into stage 2 . . . 31
5.7 Developing the MambaIR-inspired DAM . . . 32
5.8 MambaIR experiments and results . . . 34
5.9 Developing the CFMW-inspired DAM . . . 34
5.10 CFMW experiments and results . . . 35
5.11 Developing the MambaTree-inspired DAM . . . 36
5.12 MambaTree experiments and results . . . 36
5.13 Summary . . . 38

6 Integration into the MOCID Software Pipeline **38**
6.1 Input and output contract . . . 38
6.2 Module replacement . . . 39
6.3 Configuration . . . 39
6.4 Summary of integration . . . 40

7 Training and Evaluation **40**
7.1 Dataset and preprocessing . . . 40
7.2 Training configurations . . . 40
7.3 Evaluation methodology . . . 41
7.4 Worked metric examples . . . 42
7.5 Baseline results . . . 43
7.6 Comparison of the displacement modules . . . 43
7.7 Per-video behaviour and demonstrations . . . 45
7.8 Computational efficiency . . . 45
7.9 Limitations and interpretation . . . 46
7.10 Summary . . . 47

8 Recommendations for Future Work **47**
8.1 Priority recommendations . . . 47
8.2 Further experiments . . . 48
8.3 Decision criteria . . . 48

9 Conclusion **49**

References **50**

Statement of AI Tool Use **53**

A Understanding State and Memory **54**
A.1 A physical example . . . 54
A.2 A scalar memory . . . 54
A.3 The State Space of a Race Car . . . 55
A.4 Application in Language . . . 56

B Recurrence and Convolution Worked Example **58**
B.1 Calculating one step at a time . . . 58
B.2 Calculating the same result as a convolution . . . 58

B.3 Step-by-step Unrolling of the Recurrence . . . 58
B.4 Recognising the Convolution . . . 59
B.5 Numerical Verification . . . 59
B.6 What Selectivity Changes . . . 60

C Selection and Discretisation Explained **61**
C.1 From continuous parameters to a discrete update . . . 61
C.2 What the elementary operations do . . . 61
C.3 A deliberately selective example . . . 62
C.4 Why a Selection Mechanism is Needed . . . 62
C.5 What "Linear$_N(x)$" Means . . . 62
C.6 What "Broadcast$_D$" Means . . . 63
C.7 What "Softplus" Does and Why . . . 63
C.8 What "Gating" Means . . . 63
C.9 A Worked Numerical Example . . . 64
C.10 Summary of the Mechanism . . . 65

D Tensor Shapes and GPU Computation **66**
D.1 Following one clip through the detector . . . 66
D.2 Why hidden-state storage can be large . . . 66
D.3 What a Tensor Is . . . 66
D.4 The Three Indices of a Sequence Input . . . 67
D.5 The Project's Setting . . . 67
D.6 Where the Hidden State Lives . . . 67
D.7 Why the Size Matters . . . 67

E Why the Hardware-Aware Algorithm is Necessary **69**
E.1 Two Kinds of GPU Memory . . . 69
E.2 Why a Naive Implementation is Slow . . . 69
E.3 Trick One: Parallel Scan . . . 69
E.4 Trick Two: Kernel Fusion . . . 69
E.5 Trick Three: Recomputation . . . 70
E.6 Summary . . . 70

F Reproduction Protocol and Evidence Package **71**
F.1 Dataset audit . . . 71
F.2 Reproducing an existing checkpoint . . . 71
F.3 Comparing new variants . . . 71
F.4 Recommended inference timing protocol . . . 72

G Project Record and Provenance **73**
G.1 Inherited work and undergraduate development . . . 73
G.2 Levels of evidence . . . 73
G.3 Diagnostics retained in project notes . . . 73
G.4 Evidence records . . . 74

H Result Tables from the Literature **75**
H.1 Detection Performance of Leading Object Detectors . . . 75
H.2 Performance Comparison of 4 Pillars . . . 75
H.3 Performance Comparison of IRMamba Against Existing IRSTD Approaches . . . 76
H.4 ImageNet VID Detection Performances . . . 76

I Future Work Plan and Outstanding Measurements **78**

I.1 Priority order 78
I.2 Outstanding measurements 78

J Project Technical Objectives **79**

K Project Gantt Chart **81**

L Work Breakdown Structure **82**

List of Figures

1 Line-of-sight limitations of ground-based radar for hypersonic missile detection. A vehicle flying below the radar horizon stays hidden until it is close, which motivates observing it from space. . . . 1

2 Computational complexity comparison across input resolutions, demonstrating linear scaling versus quadratic scaling, which exhausts available GPU memory at larger input scales. Reproduced from Guo et al. 2025, Figure 5. . . . 6

3 Illustration of fixed scanning strategies used in previous visual SSMs, including raster, continuous, and local scan patterns. Reproduced from Xiao et al. 2024, Figure 1. . . . 7

4 Block diagram of a continuous-time linear state space model. The latent state $h(t)$ is updated through $\mathbf{A}$ from its previous value and through $\mathbf{B}$ from the input $x(t)$. The output $y(t)$ is produced from the latent state through $\mathbf{C}$. . . . 14

5 The zero-order hold rule samples a continuous signal at a fixed step size Δ. The learnable parameter Δ controls how finely the underlying continuous dynamics are sampled and therefore how aggressively the state is updated at each step (Grootendorst 2024). . . . 15

6 Conceptual view of the zero-order hold. A discrete token sequence (top) is treated as a piecewise-constant continuous signal (bottom) by holding each value over an interval of width Δ. The continuous-time SSM dynamics are integrated over this signal, and the result is sampled to give the discrete recurrence used in practice. 15

7 The two equivalent representations of a linear time-invariant SSM. . . . 17

8 Comparison of the S4 and S6 (selective) algorithms. In S4, $\mathbf{B}$, $\mathbf{C}$, and Δ are static parameters, allowing the recurrence to be expressed as a global convolution. In S6, $\mathbf{B}$, $\mathbf{C}$, and Δ are produced by linear projections of the input and therefore vary across the sequence length, making the model content-aware at the cost of breaking the convolutional form. . . . 19

9 Memory traffic in the fused selective scan. The SSM parameters are loaded from HBM into SRAM once. Discretisation, scan, and output projection are all performed in SRAM. Only the output tensor is written back to HBM. The intermediate hidden states are not stored and are recomputed during backpropagation. . . . 20

10 The Mamba block. The input projection expands the model dimension by a factor E and produces two branches. The first branch applies a one-dimensional convolution, a SiLU activation, and the selective SSM (S6). The second branch produces a gating signal through a SiLU activation. The branches are combined by element-wise multiplication and projected back to the model dimension. The block is wrapped in a residual connection and normalisation layer (not shown). . 22

11 The MOCID detector and the three designs tested in this project. The top row is the complete inference path, in which only the highlighted DAM changes between experiments. The dashed box expands the DAM block, and the lower row marks where each design acts. In stage 1, raw backbone features bypass the DAM and enter the same temporal pool. . . . 23

12 The R0 stage-1 validation curve on 4,767 clips, as logged before BatchNorm recalibration was introduced. The collapse around epoch 20 prompted the investigation in this section, which later showed that part of it came from the evaluation and not from the learned weights. . . . 29

13 The mathematical relationship between a bias and sigmoid output when other logit terms are zero. Marked values illustrate the initial bias, zero, and the inspected bias. The curve is not a measured confidence distribution. . . . 30

14 The MambaIR-inspired DAM . TIDS retains the temporal scan. The scan projection and final CAB convolution start at zero, residual scales start at one, and the final projection starts as the identity. The complete block therefore initially returns the reference feature. 33
15 The CFMW-inspired DAM . TIDS is retained from the original DAM, and the CFM branch (shaded) runs beside it on the same projected features. The scan-output projection P starts at zero and the final projection Q starts as the identity. The complete block therefore initially returns the reference feature. 35
16 The same CFMW-inspired model scored with its DAM on and off at every evaluation on 4,767 clips. Bypassing the DAM scored higher at 48 of 50 evaluations, so the module reduced AP50 for most of training. The DAM-off curve falls after epoch 60, which suggests that the feature pyramid and head begin to overfit. . . 36
17 The MambaTree-based DAM . SDS and TIS are retained from the original DAM, and the tree scan (shaded) replaces the bidirectional raster scan. The scan-output projection starts at zero and the final projection starts as the identity. The complete block therefore initially returns the reference feature. 37
18 Illustrative sensitivity of equal square boxes to horizontal localisation error. The smaller box reaches the AP50 matching boundary after a one-pixel shift. 42
19 Base baseline validation curves on 4,795 frames, using COCO AP50 without smoothing. The corrected batch-4 run stays stable late in training, whereas the pretrained run declines. The pretrained run also changes input scaling, so its curve does not isolate the effect of pretraining. 43
20 Every evaluation of the four stage-2 runs on 4,767 clips, under the R0 recipe. The vertical axis is restricted to 86–91% to make the small differences visible. The curves overlap throughout, which shows that no module separates clearly from the corrected original DAM once normal run-to-run variation is considered. These are single runs, not seed averages. 44
21 Per-video scores at epoch 51 of the corrected batch-4 Base run. A few scenes account for most of the lost accuracy, especially data15 and data21. Video lengths differ, so the overall AP50 is not the simple average of these seven values. 46
22 Illustrative retention of a unit state under $h_t = a^t h_0$, with $h_0 = 1$ and zero subsequent inputs. Different transition values produce different memory lengths. 54
23 Evolution of the race car's four state components over twelve days. Between maintenance events, each component decays according to the transition matrix $\overline{\mathbf{A}}$, with the rate determined by the diagonal entries (gas at 0.90, oil at 0.95, tyres at 0.80, motor at 0.85). Maintenance inputs x_t produce upward spikes through the matrix $\overline{\mathbf{B}}$: fluid top-ups raise gas and oil simultaneously, while repairs raise tyres and motor. 57
24 Effect of selectivity on a sequence with a noise token. The LTI model applies the same fixed kernel at every step, so the noise input $x_1 = 0$ still leaves a strong contribution at $y_2 = 1.81$. The selective model detects the noise input and effectively resets the state, suppressing both y_1 and the noise's contribution to y_2. 60
25 A hand-calculated illustration of selective retention. Small gates retain state through the two intervening zero inputs. No experimental detection performance is represented. 62
26 Step-by-step trace of the selective SSM on the input sequence $x = (1.0, 0.0, 1.0)$ with weights $\mathbf{W}_B = \mathbf{W}_C = 1$, $\mathbf{W}_\Delta = 2$, $b_\Delta = -1$, and $\mathbf{A} = -1$. At $t = 0$ the large step size $\Delta_0 \approx 1.31$ opens the gate and the input is absorbed. At $t = 1$ the noise input produces a small step size $\Delta_1 \approx 0.31$, so the state is preserved and the output suppressed. At $t = 2$ the gate reopens, the new input is absorbed, and the output reflects both the new input and the retained memory of h_0. 64

27 Tensor shapes encountered when feeding satellite imagery into an SSM. Step 1: a feature map of shape (H, W, D) produced by the convolutional backbone is flattened by reading its spatial positions in a fixed order, producing a one-dimensional sequence of length $L = H \times W$ in which each position carries D feature channels. Step 2: a mini-batch is formed by stacking B such sequences along an additional batch dimension, giving the input tensor of shape (B, L, D) that the SSM operates on. The same SSM is applied independently to each of the D channels within each sequence, producing a hidden state tensor of the larger shape (B, L, D, N). . . . 68
28 Sequential recurrence versus parallel scan for a sequence of length $L = 8$. Left: the naive evaluation produces the hidden states $h_0, h_1, \ldots, h_7$ in a single chain, with seven sequential steps required before h_7 becomes available. Each step must wait for the previous one to finish, so the GPU's parallel arithmetic units sit idle most of the time. Right: the parallel scan exploits the associativity of the recurrence to combine partial results in a binary tree. Each level of the tree can be evaluated entirely in parallel, and the full prefix $h_{0:7}$ is produced in $\log_2 8 = 3$ parallel steps. The right-hand structure recovers GPU parallelism along the sequence length without changing the final result. 70
29 Detection performance of leading generic object detectors measured by Average Precision (AP) for small, medium and large objects, with corresponding Degree of Reduction (DOR) scores, showing the performance gap between small and large object detection. Reproduced from (G. Chen et al. 2020), Table 1. 75
30 Average precision comparison of object detection models across four architectural groups (multi-scale representation, contextual information, super-resolution, and region-proposal) on MS COCO. Reproduced from (G. Chen et al. 2020), Table 1. 75
31 Performance comparison of traditional, CNN-based, hybrid, and Mamba-based IRSTD methods across three benchmark datasets. Reproduced from (M. Zhang et al. 2025a), Table 1. 76
32 Comparison of video object detection methods on ImageNet VID across AP metrics and inference speed. Reproduced from (Yao et al. 2026), Table 1. 76
33 Detection performance comparison across slow, medium, and fast motion speeds on ImageNet VID. Reproduced from (Yao et al. 2026), Table 2. 77
34 Project Gantt Chart Outlining the timeline 81

List of Tables

1 Summary of AI tool use. ii
2 Contributor Roles Taxonomy (CRediT) statement, with roles grouped into the four contribution categories used in the Contributions Declaration template. Each cell contains a short description of the specific contribution. xvii
3 Comparison of architectural building blocks evaluated in this review. Properties derived from (G. Chen et al. 2020; Guo et al. 2025; M. Zhang et al. 2025a; Xiao et al. 2024; Gu and Dao 2024). 8
4 Mamba adaptations considered for the DAM, the weakness of a plain scan each addresses, and its relevance to MOCID. IRMamba and OVSMMFA-Net informed the review but were not implemented. 12
5 The three faults found in the inherited implementation, how each was identified, and its correction. Together they turned an unstable prototype into a dependable reference for comparing displacement modules. 28
6 Stage-2 validation results of the MambaIR-inspired DAM on DAUB (%). Both runs start from the R0 stage-1 checkpoint and are evaluated every second epoch on 4,767 clips. The better value in each row is in bold. 33

7 Parameter count of the DAM blocks with and without the CFM branch (millions). 34
8 Stage-2 validation results of the CFMW-inspired DAM on DAUB (%). All columns start from the R0 stage-1 checkpoint and use 4,767 clips. The DAM-off column scores the CFMW model at each evaluation with its DAM bypassed. The best value in each row is in bold. 35
9 Stage-2 validation AP50 (%) of the MambaTree-inspired DAM on DAUB. All runs start from the R0 stage-1 checkpoint (88.65%) and are evaluated every second epoch. *Best* is the highest score of the run, with its epoch in parentheses. *Final* is the score at epoch 100. Training speed is in steps per second on one NVIDIA L40S. The best value in each row is in bold. 37
10 Stage-2 validation F1 (%) for the MambaTree runs in Table 9. F1 is the highest value over all confidence thresholds at an IoU of at least 0.5. *Best*, *Final* and the *Means* are defined as in Table 9. The best value in each row is in bold. 38
11 Tensor dimensions for the default five-frame, 512×512 configuration. 39
12 Configuration profiles used to select each experiment. 39
13 Validation frame and clip counts from the local DAUB split. 40
14 Training settings for the Base baseline and for the stage-2 displacement module runs. 41
15 Base baseline validation results on all 4,795 frames, using COCO AP50 and SSTNet-style F1 (%). Batch is the number of images in each training step. . . . 43
16 Comparison of the four displacement modules on 4,767 validation clips under the R0 recipe. AP50 and F1 are in %. F1 is the best value over the confidence sweep at the final epoch. The best value in each row is in bold. 44
17 Measured computational cost on one NVIDIA L40S. Inference is per five-frame clip. Training cost is relative to the corrected original DAM, from the steps per second in Table 9. Dashes mark values that were not measured. 46
18 A complete scalar recurrence calculation. 58
19 Milestones supported by the local history and experiment records. 73
20 Diagnostics retained in project notes, with their evidence limits. AP50 values are percentages. 74
21 Repository evidence records for each part of the report. 74
22 Recommended future work and the evidence each task should produce. 78
23 Status of measurements for each model. Done marks a recorded measurement, and Outstanding marks one left to complete using Appendix F. 78
24 Technical objectives for the investigation of a Mamba-based building block, specifying evaluation criteria and deliverables for each objective. 79

Nomenclature

Abbreviations

AI Artificial intelligence.

AP Average precision, the area under a precision–recall curve.

AP50 Average precision when a detection must overlap its labelled box with an IoU of at least 0.50. Reported as a percentage.

BN Batch normalisation, called BatchNorm in the code.

CAB Channel attention block, the local processing unit of MambaIR.

CFM Cross-modality Fusion Mamba, the fusion module of CFMW.

CFMW Cross-modality Fusion Mamba with Weather-removal, a visible and infrared object detector (H. Li, Q. Hu, et al. 2025).

CNN Convolutional neural network.

COCO Common Objects in Context, a large public object detection dataset that also defines widely used evaluation conventions, including a size definition for small objects (T.-Y. Lin et al. 2014).

CRediT Contributor Roles Taxonomy.

DAM Displacement-aware Mamba Module, the part of MOCID that compares the target frame with each reference frame.

DAUB A public infrared video dataset of dim, small aircraft targets against ground and air backgrounds, containing 17 sequences and about 13,800 frames (Hui et al. 2019).

DOR Degree of reduction, the AP difference between small and large objects.

EMA Exponential moving average of model parameters and buffers.

FISTA Fourier-inspired spatio-temporal attention in MOCID. It is unrelated to the optimisation algorithm that shares this abbreviation.

FLOP Floating-point operation.

FPN / PAFPN Feature pyramid network / path aggregation feature pyramid network.

GAN Generative adversarial network.

GPU Graphics processing unit.

HBM High-bandwidth memory.

HiPPO High-order Polynomial Projection Operator.

IoU Intersection over union of predicted and labelled boxes.

LSTM Long short-term memory.

LTI Linear time-invariant.

mAP Mean average precision.

MIRSTD Moving infrared small target detection.

MLP Multilayer perceptron.

MOCID Motion Context and Displacement Information learning, the multi-frame detector used as the host in this project (M. Zhang et al. 2025b).

MST Minimum spanning tree.

NMS Non-maximum suppression, which keeps the most confident of several overlapping detections and removes the others.

OPIR Overhead persistent infrared.

PADM Perturbation-Adaptive Diffusion Model, the weather-removal component of CFMW.

RNN Recurrent neural network.

ROI Region of interest.

RSSB Residual state space block, the core unit of MambaIR.

S4 Structured state space sequence model (Gu et al. 2021a).

S6 Selective state space algorithm, the core layer of Mamba.

SGD Stochastic gradient descent, the optimiser that updates network weights from small groups of training examples.

SiLU Sigmoid linear unit.

SPIE Society of Photo-Optical Instrumentation Engineers, an international society for optics and photonics.

SRAM Static random-access memory.

SSD Single Shot MultiBox Detector.

SSM State space model.

STB Spatio-temporal backbone, the part of MOCID that extracts features from the clip using FISTA.

TIDS / SDS / TIS Temporal Interpolation and Difference-aware Selective Scan, the DAM's scan. It consists of Spatio-temporal Difference Selection and the Temporal Interpolation Scan.

ViT Vision Transformer.

VOC Pascal Visual Object Classes, the challenge whose AP calculation is used for the main results in this report (Everingham et al. 2010).

YOLOX A single-frame object detector. Its small variant, YOLOX-S, provides the Base model and MOCID's detection head.

ZOH Zero-order hold.

Symbols

B, T, C, H, W Batch size, number of frames, channels, height, and width.

D Channel count in the sequence-model notation of the Mamba paper. It is the same quantity as C.

L Sequence length.

N State size, the number of state variables stored per feature channel.

F_T, F_R Target and reference feature maps.

Terms

Ablation An experiment designed to isolate the effect of one component or setting.

Backbone / neck / head The components that extract features, combine feature scales, and predict boxes and scores.

Base The plain single-frame YOLOX-S detector used as a baseline.

Batch size The number of training examples processed together in one optimisation step.

BatchNorm recalibration Recomputing the stored BatchNorm statistics of the evaluated model from training batches before scoring it. No weights change.

Checkpoint Saved model weights and, where recorded, the training state.

Epoch One complete pass through the training data.

Handover The transition from stage 1 to stage 2, when the untrained DAM is first switched on.

Percentage point The absolute difference between two percentages. All AP50 differences in this report use this convention.

R0 The team's first complete training recipe, kept unchanged as the common reference for all displacement module comparisons.

Stage 1 The first training stage, in which the backbone, feature pyramid and head learn with the DAM bypassed.

Stage 2 The second training stage, in which the backbone is frozen and the DAM is switched on and trained.

Acknowledgments

This report records the outcomes of an Honours research project at Adelaide University. The authors would like to acknowledge the people whose support made the work possible.

The authors first express their gratitude to their lead supervisor, Dr Yee Wei Law. His expertise in machine learning and his continuous guidance shaped the direction of this project, and he was always generous with his time and feedback. They also thank Dr Jordan Kildare and Dr Michael Evans for their technical advice and for the context they provided on the wider research programme.

The authors thank Northrop Grumman Australia for the opportunity to undertake this technical project, and particularly Dr Dushy Tissainayagam, whose industry guidance kept the work relevant to the broader application. They also acknowledge Gaurav Singh for the earlier MOCID implementation from which this work began.

This work was carried out within the School of Electrical and Mechanical Engineering at Adelaide University, whose research environment and computing resources were integral to the project. The contribution of each student author is recorded in the Authorship and Contributions Statement.

Authorship and Contributions Statement

This report was completed by three students. Each student's contribution is recorded below using the Contributor Roles Taxonomy (CRediT) framework, in alignment with the Adelaide University Authorship Procedure. Following the Engineering Research Project Contributions Declaration template, the individual CRediT roles are grouped into four contribution categories (Funding acquisition and Supervision omitted).

1. Approach (Conceptualization, Data curation, Methodology)
2. Technical output (Investigation, Formal Analysis, Software, Validation)
3. Writing (Visualisation, Writing – original draft, Writing – review & editing)
4. Project management (Project administration, Resources)

Each member's specific contributions are recorded against these categories in Table 2.

Table 2: Contributor Roles Taxonomy (CRediT) statement, with roles grouped into the four contribution categories used in the Contributions Declaration template. Each cell contains a short description of the specific contribution.

Role	**Mithusha Kulatunga**	**Leah Menzies**	**Brian Park**
Approach			
Conceptualization	Proposed the MambaIR adaptation and the demonstration tool	Proposed the tree scan adaptation and its matched control	Proposed the CFMW adaptation and the review of the inherited code
Data curation	Curated recipe and per-video records	Organised literature sources and tree scan records	Curated the DAUB baseline and R0 records
Methodology	Designed the weight-decay and MambaIR experiments	Designed the state size and scan experiments	Designed the baseline, handover and DAM-off experiments
Technical output			
Investigation	Ran the recipe diagnosis and MambaIR training	Ran the literature investigation and MambaTree training	Ran the Base, R0, repaired DAM and CFMW training
Formal Analysis	Analysed the training collapse and MambaIR results	Analysed gaps in the literature and scan results	Analysed baseline, handover and CFMW results
Software	MambaIR DAM, configuration profiles, video demonstration tool	Tree scan GPU operations, tree DAM, speed benchmark	Base models, shared pipeline, handover repair, BatchNorm recalibration, CFMW DAM
Validation	Checked the MambaIR block against its source	Checked the tree scan with a matched control	Built the DAM verification tool
Writing			
Visualisation	Figures for MOCID, the MambaIR design and results	Figures for the tree scan design and results	Figures for the CFMW design and results
Writing – original draft	Introduction, theory, MOCID, reproduction and MambaIR sections	Literature review, tree scan sections	CFMW sections, integration sections
Writing – review & editing	Reviewed the complete report	Reviewed the complete report	Reviewed the complete report
Project management			
Project administration	Maintained the shared code repository	Managed literature records	Coordinated the lab server work
Resources	Managed team access to the code and results	Literature database	Lab server environment and dependencies

Co-Author Agreement

By signing this Statement of Authorship, each author certifies that:

- The candidate's stated contribution to the publication is accurate (as detailed above),
- The candidate accepts responsibility for the integrity of their individual contributions,
- The contributions of all co-authors combined make up the remainder after the candidate's contribution is subtracted from 100%.

Name of Co-Author	Mithusha Kulatunga
Signature	**Date:** _____

Name of Co-Author	Leah Menzies
Signature	**Date:** _____

Name of Co-Author	Brian Park
Signature	**Date:** _____

1 Introduction

The detection of small, fast-moving aerial objects against complex backgrounds is among the most challenging open problems in modern remote sensing. This project investigated whether a recent class of neural network building blocks, the selective state space models, could improve the detection of such targets in multi-frame infrared imagery, with the long-term aim of monitoring high-speed airborne vehicles from space. Three Mamba-based designs were built into a working detector and measured against a carefully repaired reference.

1.1 Background

Hypersonic vehicles are aerial platforms capable of sustained flight at or above Mach 5. They include both manoeuvring re-entry bodies and air-breathing cruise platforms, and they have been the subject of significant aerospace research and development over the past decade (Van Wie 2021). Three properties make them particularly difficult to track with conventional sensors. Their high speeds compress the time available for detection and response. Their flight regimes place them outside the typical envelope of ground-based air-defence radars, reducing the range at which they can be observed. Their capacity for in-flight manoeuvre also makes ballistic trajectory extrapolation unreliable beyond short time horizons (Dahlgren 2023). Figure 1 shows the resulting line-of-sight limitation.

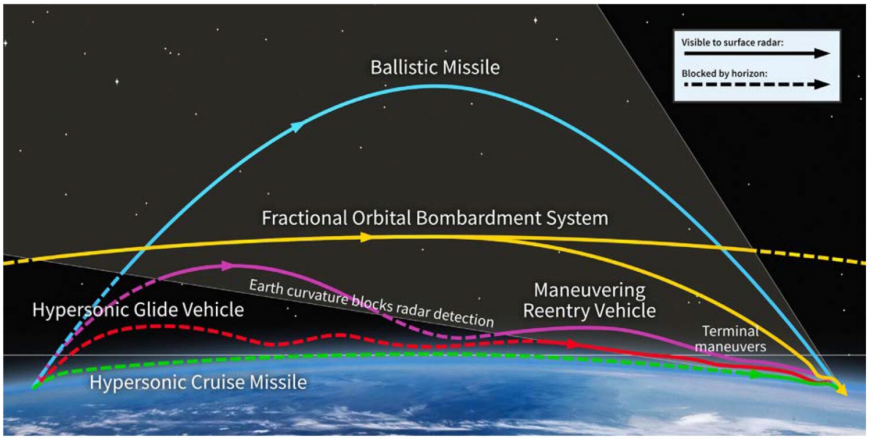


Figure 1: Line-of-sight limitations of ground-based radar for hypersonic missile detection. A vehicle flying below the radar horizon stays hidden until it is close, which motivates observing it from space.

Space-based optical and infrared imaging has emerged as a complementary sensing option for this class of target. A satellite-mounted sensor observes the upper atmosphere from above and is not subject to the horizon limits that constrain ground-based systems (Dahlgren 2023). The technical problem then shifts from radar signal processing to computer vision. Targets of interest may occupy only a handful of pixels, and they appear against cloud, atmospheric features, terrain, and the curvature of the Earth's limb. The signal-to-clutter ratio is typically low and varies between frames.

Locating such targets is known as small-object detection, and standard deep learning detectors underperform at it (G. Chen et al. 2020). The repeated downsampling in convolutional backbones removes the fine spatial detail in which small targets are encoded. Attention-based Transformer mechanisms recover global context, but their quadratic scaling makes them expensive on the high-resolution feature maps that matter most here (Fournier et al. 2023). Motion offers further evidence, because a target's position changes between frames while much of the background does not. A multi-frame detector can exploit this cue, provided it does not mix target information with moving clutter.

Selective state space models address the cost of global context. Mamba processes an image as a sequence and decides from the content of each position what to remember, at a cost that grows linearly with sequence length (Gu and Dao 2024). Several recent designs adapt it to vision, and three of them shaped this project. MambaIR surrounds the scan with local convolution and channel attention to recover the neighbourhood detail that flattening an image into a sequence destroys (Guo et al. 2025). CFMW places two images of one scene in a single sequence, so that each can draw on the whole of the other (H. Li, Q. Hu, et al. 2025). MambaTree replaces the fixed row-by-row reading order with a tree that links similar neighbouring pixels (Xiao et al. 2024). Each targets a different weakness of a plain Mamba scan, which made them natural candidates for improving a Mamba-based detector.

1.2 Motivation

The project was undertaken in collaboration with Northrop Grumman Australia and Adelaide University. The industry partner's interest is space-based monitoring of high-speed airborne platforms, and its brief sought a modern neural network building block that could be implemented in PyTorch and evaluated in an existing detection framework. The choice matters because the building block determines both the accuracy achievable on small, low-contrast targets and the computational cost of inference, which governs whether a detector could run on board or in near real time.

The literature review selected Mamba for its linear scaling and input-dependent information mixing, but those properties motivate an experiment rather than guarantee an improvement. MOCID, a published multi-frame infrared detector, provided the ideal setting because its displacement-aware Mamba module already applies a selective scan to pairs of frames (M. Zhang et al. 2025b). That module compares the target frame with each reference frame one position at a time, in a fixed order, so it presented three clear opportunities. MambaIR offered a way to add local processing and channel attention around the scan. CFMW offered scene-wide context, to help separate genuine target motion from camera motion. MambaTree offered a reading order that keeps similar neighbouring pixels together, in place of a raster scan that can place them far apart in the sequence. To the team's knowledge, no published evaluation had compared these designs inside a multi-frame infrared detector.

Answering that question fairly proved to be a substantial engineering challenge in its own right. A training defect can conceal the ability of a model, and inconsistent evaluation can change the apparent benefit of a new component. A dependable MOCID had to exist before any building block could be judged, so the project became both a reconstruction of the detector and a comparison within it.

1.3 Aims and Scope

The aim was to provide an evidence-based assessment of Mamba-based building blocks for small moving-target detection, with accuracy and computational cost considered together. The assessment rests on a controlled comparison in which only the displacement module changes while the backbone, starting weights, training recipe and evaluation stay fixed. This isolates the contribution of each design from other architectural and training choices.

The scope is bounded in three respects. First, the literature review surveys the main architectural families, but only the selected Mamba family was taken forward into implementation. Second, experiments use DAUB, a public infrared video dataset of small aircraft that provides a realistic test of dim moving targets and allows direct comparison with the published MOCID results. Third, the project delivers a research-grade detector and evaluation. Onboard deployment and operational sensor data lie beyond its scope, and they form part of the recommended future work in Section 8.

1.4 Objectives

The project was structured around five technical objectives, defined in the Project Plan and addressed in sequence over the year. Their full specifications and deliverables are given in Appendix J, their breakdown into tasks in Appendix L, and the project timeline in Appendix K.

O1. Identify and justify the building block. Conduct a systematic literature review of recent neural network architectures for small-object detection, and select a candidate on the basis of detection performance, computational efficiency, and availability of open-source implementations.

O2. Develop a formal technical description. Produce a technical description of the chosen building block, including its mathematical formulation, architectural structure, and the mechanism by which it propagates information, sufficient to support implementation and integration.

O3. Implement in PyTorch and verify. Implement the building block as reusable PyTorch modules, and verify their correctness and computational behaviour through controlled tests.

O4. Integrate into the object detection framework. Substitute the new building block into a detector while keeping the other components fixed, so that its contribution can be measured.

O5. Evaluate quantitatively. Compare the resulting detectors against a baseline in terms of detection accuracy, inference time, and memory footprint, and identify both the improvements and the limitations introduced.

Objectives O1 and O2 were achieved through Sections 2 and 3. O3 was achieved with three implemented and verified modules, described in Section 5. O4 was achieved within the complete MOCID training and evaluation pipeline in Section 6, with evaluation inside the industry partner's benchmark left as a recommendation for the reasons given there. O5 was achieved for accuracy and parameter count on all variants in Section 7, and inference timing was measured for all variants except CFMW and MambaTree, because the project's GPU server became unavailable late in the year.

1.5 Document Overview

This report follows the path of the investigation. Section 2 begins with the problem, reviewing why conventional building blocks struggle with small high-speed targets and why Mamba, and MOCID as its host, were selected. Section 3 then explains how Mamba works, from classical state space models to the selective scan. Section 4 places that theory in the complete MOCID detector and identifies the displacement module as the point of change.

Section 5 is the heart of the report. It follows the diagnosis that turned an unstable implementation into a working detector, then presents the design and results of the MambaIR, CFMW and MambaTree modules in turn. Section 6 shows how these modules fit into the shared pipeline. Section 7 brings the results together under one evaluation procedure and discusses what they mean and where their limits lie. Section 8 sets out recommendations for future work, and Section 9 concludes.

The appendices support future students who continue this work, providing worked explanations of the mathematics, the reproduction procedure, and the project records.

2 Identifying and Justifying the Building Blocks

2.1 Introduction and Scope

The detection of small, high-speed airborne vehicles is a complex challenge because conventional feature extraction architectures lose the little information these targets provide. Targets in this regime occupy as few as 3×3 pixels in a 128×128 frame, appearing as faint point sources against thermally complex and dynamic backgrounds (M. Zhang et al. 2025a). Detecting them requires simultaneous global context awareness, pixel-level spatial retention and robustness to low signal-to-clutter ratio conditions (M. Zhang et al. 2025a). Multi-frame detection introduces an additional temporal dimension, requiring the building block to recognise motion correlation and spatial consistency across sequential frames that are degraded by motion blur and out-of-focus conditions (Yao et al. 2026). The choice of building block is therefore the critical architectural decision, one that fundamentally determines detection performance in this regime.

This literature review systematically evaluates four principal architectural families, namely recurrent neural networks, convolutional approaches, transformers and state space models, against these domain-specific demands. It draws upon evidence across infrared small target detection, video object detection, and image restoration to establish convergent support for a candidate building block. The review additionally examines domain-specific adaptations of the building block across detection and restoration tasks. This provides empirical support for its architectural properties and exposes the persistent gaps that motivate adapting it to this detection regime.

2.2 Challenges in Small Object Detection

The MS COCO definition of a small object is a target with a bounding box less than 32×32 pixels, covering less than 1% of the original image (G. Chen et al. 2020). Alternatively, SPIE defines a small object as a target covering an area less than 1% of the original image (Zang et al. 2025). The hypersonic vehicle setting in this work involves targets occupying approximately 3×3 pixels in a 128×128 pixel image, representing an exceptionally low spatial scale, covering approximately 0.055% of the total frame.

Limited pixel representation reduces the system's ability to characterise features. Distinguishing small targets from overlapping objects and background noise becomes difficult. Average Precision (AP) is a standard detection performance metric that uses Intersection over Union (IoU) thresholds to classify detections as true or false positives. However, at this scale, a single pixel shift in a predicted bounding box causes disproportionately large IoU changes for targets. The Degree of Reduction (DOR) score, defined as the AP difference between small and large objects, quantifies this deficit. Across leading conventional detectors, this gap ranges from 28 to 40 percentage points, confirming that small object detection represents a persistent and unresolved performance problem (G. Chen et al. 2020).

These challenges are further compounded in infrared sensing. Object detection is compromised when cameras cannot see through rain, snow and fog. Visible images are affected by poor lighting and occlusion from adverse weather conditions, reducing contrast, colour degradation and capturing excessive noise. Infrared sensors remain useful in reduced visibility and detect the contours of a target precisely, but they lack detail in colour and texture (H. Li, Q. Hu, et al. 2025).

2.3 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) are a class of neural networks trained on sequential data and time-series information. They therefore make predictions based on the current input and its previous state. RNN-based contextual modules are slow due to taking inputs one at a time. By scanning a feature's surrounding regions sequentially, these modules process redundant context information, introducing information noise (G. Chen et al. 2020). The network processes data by maintaining a hidden state, with a finite capacity. When sequences increase in length, early information is overwritten, and backpropagation suffers from vanishing gradients (Mienye et al. 2024). For small object detection, where targets are followed across extended frame sequences, the fixed bottleneck memory and sequential processing are fundamental limitations.

2.4 Convolution Neural Networks

Unlike RNNs, which process inputs sequentially, Convolution Neural Networks (CNNs) have favourable efficiency through parallel convolution operations (Guo et al. 2025). CNNs are a dominant class of architecture for extracting and classifying features from visual data. They process images using learned spatial filters to progressively build complex feature maps. Each convolution attends to a local neighbourhood of pixels, expanding the semantic receptive field through stacking convolution layers. In increasing convolution layers, repeated downsampling operations progressively reduce spatial resolution (M. Zhang et al. 2025a). Therefore, there exists a fundamental limitation in CNN-based backbones to capture global context and low-level features, critical to correctly identifying a small target.

2.5 Transformers and Attention-Based Models

Transformers have the ability to capture global context. These models address the constraints of sequential processing and local receptive fields through self-attention. This mechanism allows the model to dynamically weigh the significance of input elements independently of their position in the sequence (Guo et al. 2025). Gu and Dao (2024) demonstrates that global field modelling experimentally outperforms CNN-based local feature extraction in image restoration. This can be extended to detection tasks, where small targets are distinguished from background noise. However, self-attention mechanisms scale computational costs quadratically with sequence length. This creates a risk of exhausting GPU memory, especially with a full-attention baseline at larger input scales (Guo et al. 2025).

2.6 State Space Models

State space models (SSMs) are a class of machine learning algorithms that draw inspiration from control systems. Unlike the quadratic scaling of transformers, SSMs scale linearly, offering a computational advantage for large image inputs (Guo et al. 2025; Xiao et al. 2024). Figure 2 illustrates this, where full attention exhausts available GPU memory at larger input scales while linear alternatives continue to operate efficiently (Guo et al. 2025). Similar to RNNs and CNNs, standard SSMs process sequences by maintaining a hidden state. They are linear time invariant (LTI), meaning their parameters are fixed regardless of the input. The S4 model is a structured SSM built on four parameters (Δ, A, B, C), which can be computed as either linear recurrence or global convolution, efficient for training the model and autoregressive inference (Gu and Dao 2024). However, the adherence to linear time invariance is a fundamental limitation of S4 and earlier structured SSMs as it prevents the model from performing dynamic and contextually aware reasoning (Gu and Dao 2024; Xiao et al. 2024).

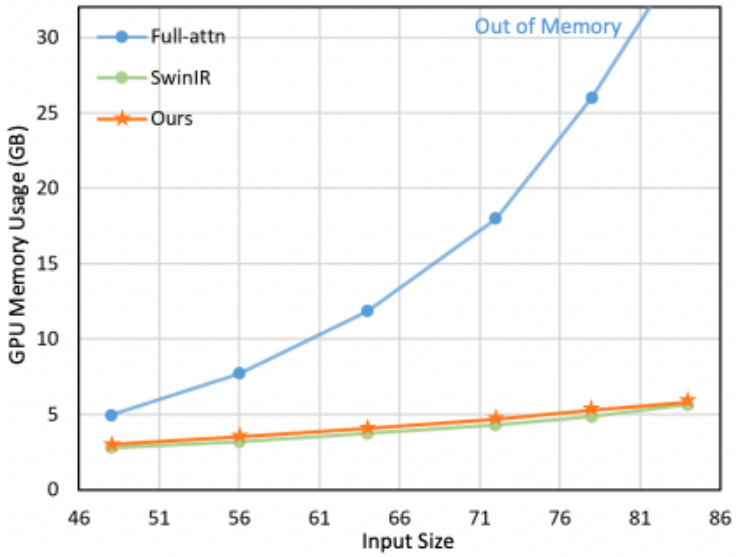


Figure 2: Computational complexity comparison across input resolutions, demonstrating linear scaling versus quadratic scaling, which exhausts available GPU memory at larger input scales. Reproduced from Guo et al. 2025, Figure 5.

Mamba, built upon S6, is a selective state space model, in which the parameters are input-dependent functions, updating at each timestep (Gu and Dao 2024). This model directly addresses the dilemma between global context and computational efficiency by retaining relevant information and discarding what is irrelevant, instead of treating input elements uniformly (M. Zhang et al. 2025a; Gu and Dao 2024).

This mechanism is validated through two synthetic tasks that LTI models fail to perform. Selective Copying requires memorising relevant tokens and filtering irrelevant ones with randomised spacing, and Induction Heads requires retrieving contextually relevant information from memory. For the Selective Copying task, the standard S4 model achieved only 18.3% accuracy. In contrast, Mamba solved the task with 99.8% accuracy. Additionally, Mamba solved the Induction Heads task with perfect accuracy across all tested sequence lengths up to 1,048,576 tokens. In comparison to competing methods that failed to generalise to sequences beyond twice its training length, Mamba was impressively able to extrapolate 4000 times longer than its training length (Gu and Dao 2024). Processing long sequences of sensor data is instrumental in meeting demands of high-velocity multi-frame detection scenarios.

However, using Mamba for visual detection exposes a critical limitation. In the application of SSMs, 2D images are flattened into 1D sequences, disrupting local spatial relationships that cue detection for tiny infrared targets (M. Zhang et al. 2025a). Standard Mamba implementations attempt to solve this by applying fixed scanning patterns such as Raster Scan, Continuous Scan and Local Scan, illustrated in Figure 3. These methods introduce spatial discontinuities between semantically related pixels, disrupting effective information flow in a sequence and local pixel forgetting, where the model loses awareness of local spatial relationships. The consequence is limited ability to model long-range spatial dependencies, a fundamental limitation to spatial understanding tasks (M. Zhang et al. 2025a).

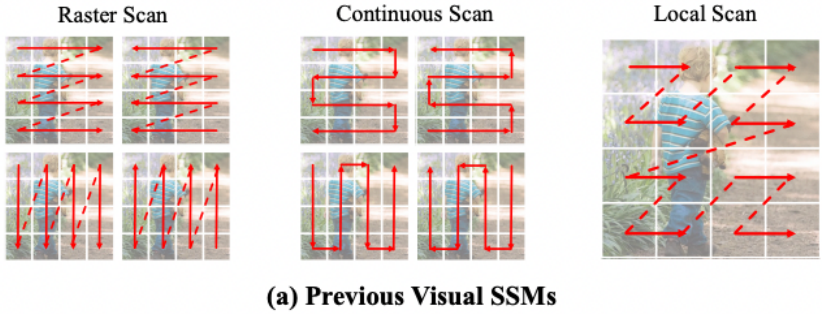


Figure 3: Illustration of fixed scanning strategies used in previous visual SSMs, including raster, continuous, and local scan patterns. Reproduced from Xiao et al. 2024, Figure 1.

To address this architectural problem, Xiao et al. (2024) propose MambaTree, which replaces fixed scanning with adaptive tree topology built from input features. As the propagation path is not pre-determined, the model respects true semantic and spatial structure. Its Tree Scanning Algorithm constructs a Minimum Spanning Tree (MST) from the input feature map by connecting pixels of similar characteristics. Mamba's standard recurrent state transition is then applied along this adaptive tree structure (Xiao et al. 2024). With this approach the algorithm initially treats each pixel as a root, aggregating from all other pixels. Therefore, if a frame contains L pixels, there exists L×L operations. Decomposing the computation into two linear passes, leaf to root and then root to leaf, first aggregates features towards the root and then distributes context back out to every pixel. This reduces the computational complexity from $\mathcal{O}(L^2)$ to $\mathcal{O}(L)$, matching standard Mamba (Xiao et al. 2024). In comparison to alternate methods, MambaTree outperformed strong SSM-based competitors in image classification accuracy, object detection precision and pixel-level segmentation accuracy. A direct ablation of scanning strategies further identifies tree topology contribution to high performance, demonstrating an accuracy improvement of approximately 1% for Raster Scan and 0.4% for Cross Scan (Xiao et al. 2024).

The local pixel forgetting problem is further addressed by MambaIR, through the introduction of Residual State Space Blocks (RSSBs). The RSSB combines the SSM with a local convolution layer to restore neighbourhood relationships lost by 1D flattening, and then adds channel attention to suppress redundant channel information. Additionally, RSSB integrates learnable scale factors on skip connections, allowing the network to dynamically control the balance between passing through original input and processed information (Guo et al. 2025). These contributions enable the model to maintain both global and local spatial awareness simultaneously.

Ablation studies confirm that standard Mamba without domain-specific modifications produces inferior results (Guo et al. 2025). Additionally, SSMs and attention mechanisms are not interchangeable across architectural designs, and cannot simply be substituted for one another within the same block configuration without compromising results (Guo et al. 2025). As the spatial limitations are shown to be solvable architectural challenges rather than fundamental flaws of SSMs, the combination of linear complexity and adaptive spatial modelling identifies Mamba-based architectures as a promising direction for visual detection tasks. The remaining limitations are addressed by domain-specific adaptations in the next section.

Table 3 summarises the key architectural properties of each building block evaluated in this review, highlighting the fundamental limitations that motivate the domain-specific investigation in Section 2.7.

Table 3: Comparison of architectural building blocks evaluated in this review. Properties derived from (G. Chen et al. 2020; Guo et al. 2025; M. Zhang et al. 2025a; Xiao et al. 2024; Gu and Dao 2024).

Property	RNNs	CNNs	Transformers	SSMs (Mamba)
Receptive field	Local - sequential scan of surrounding regions	Local - fixed convolutional neighbourhood	Global - full self-attention across entire input	Global - selective state space scan across entire sequence
Computational complexity	Linear but sequential - cannot parallelise	Linear - parallel convolution operations	Quadratic - exhausts GPU memory at scale	Linear - matching standard Mamba complexity
Spatial detail retention	Compressed into fixed hidden state	Lost through progressive downsampling	Sacrificed by efficient attention variants	Preserved through selective content-aware mechanism
Long-range dependency	Limited by vanishing gradients and hidden state capacity	Not supported - fixed local receptive field	Strong but at quadratic cost	Strong - selective retention across entire sequence
2D visual adaptation	Architecture-specific contextual modules - information noise	Native 2D processing	Shifted window attention sacrifices global receptive field	Requires 1D flattening - local pixel forgetting
Training stability	Vanishing gradients during backpropagation	Stable	Stable	Stable - validated through synthetic tasks

2.7 Domain-Specific Evidence

Existing Small Object Detection Approaches

To compensate for the deficiencies in conventional building blocks, recent studies have pursued four key augmentation methods: multiscale representation, contextual information, super-resolution and region-proposal (G. Chen et al. 2020). While each strategy targets primary challenges in small object detection, the performance gap between small and large object only improves slightly. This suggests that the limitations are likely architectural in origin, motivating the investigation of a more capable building block (G. Chen et al. 2020).

Multiscale representation combines feature maps from different neural network layers. Shallow layers preserve fine spatial resolution, whereas deep layers sacrifice spatial detail for strong semantics. Methods such as FPN, SSD's pyramidal hierarchy, and various deconvolution-based fusion strategies attempt to recover detail loss by fusing information across multiple scales, while HRNet maintains high-resolution representations throughout the network (Wang et al. 2021). Evaluation across leading generic detectors, displayed in Appendix H.2, shows the highest performing multiscale method to achieve a DOR of 29.1, still within the original range of performance gap between small and large object detection. One limitation that was identified for this strategy was the increased computation complexities. In addition, unnecessary representation of background noise can degrade performance in the fusion process (G. Chen et al. 2020).

Contextual Information uses surrounding neighbourhood information to identify a small object. This strategy is able to improve object classification, however as it focuses on information obtained around the ROI area, redundant contextual information causes information noise. Appendix H.2 shows the most effective contextual information method achieving a DOR of 31.4, again comparable to unaugmented baselines. Similar to multiscale representation, in response to insufficient information at the point of extraction, this method attempts to compensate after image processing. Complexities are introduced as many context methods rely on RNNs or spatial attention, inheriting the scalability constraints identified in Section 2.4 (G. Chen et al. 2020).

Super resolution addresses deficits in feature representation by upscaling low-resolution target regions prior to detection. Methods like Perceptual GAN, SOD-MTGAN, and JCS-Net generate super-resolved representations of the original image to feed into the downstream detector. The implementation of super resolution is revealed to have modest improvements in Appendix H.2, with a DOR of 29. The paper acknowledges that GANs are notoriously difficult to train and susceptible to model collapse. Adding a generator and discriminator networks on top of the detector adds significant overhead (G. Chen et al. 2020). MambaIR demonstrated that Mamba-based architectures are excellent at image restoration, outperforming SwinIR by up to 0.45dB under similar computational costs (Guo et al. 2025). In the case that Mamba is able to preserve information better in the backbone itself, the super-resolution step becomes redundant. Alternatively, in the case images may still not be clear enough for reliable detection, the addition of mamba-based super resolution modules could be more effective than GAN-based approaches (Guo et al. 2025).

Region proposal methods predefine smaller bounding boxes and appropriate sampling strategies to provide the network with more exposure to small object images during training. This method can improve the number of small objects that undergo classification, however, a small target that is poorly represented in the feature map cannot have its information recovered through anchor refinement (G. Chen et al. 2020). This reinforces that the missing information is lost in the backbone, shifting the focus to the architectural cause.

Infrared Small Target Detection

The challenges in existing approaches are further compounded when the imaging modality is infrared. Infrared images display targets as small and faint point sources against thermally complex and dynamic backgrounds. This can produce significant noise, obscuring targets from detection. Additionally, intensities from interferences in the background may initiate false alarms (M. Zhang et al. 2025a). These weaknesses expose the limitations of conventional building blocks. Recent work continues to refine CNN and YOLO-based detectors through multidirectional difference convolution (L. Zhang et al. 2025), gradient decomposition (C. Hu et al. 2026) and lightweight real-time designs (Atrash et al. 2025), which shows the importance of local contrast. CNN-based methods are limited in their ability to capture global context and features of lower intensities. To address this, hybrid solutions combining Vision Transformers (ViTs) with CNNs have been proposed, with significant improvements in long-range detection and capturing global context. However, as identified in Section 2.5, the use of transformers comes at the cost of quadratic complexity. This yields operational limitations, for instance, the inference time of approaches such as RKformer and IAANet are significantly longer than CNN-based methods (M. Zhang et al. 2025a). The GPU memory constraint identified by Guo et al. (2025) directly manifests in hybrid methods such as IAANet, tested to exhaust all available memory at 1024x1024 pixel resolution (M. Zhang et al. 2025a). The CNN component uses downsampling to capture broader semantic information, however, this may reduce target detail during image reconstruction (M. Zhang et al. 2025a). This reiterates the broader trade-off between global receptive field modelling and computation efficiency in both restoration and detection applications (Guo et al. 2025).

As identified in Section 2.6, Mamba's favourable linear complexity disrupts local spatial relationships. IRMamba addresses this by modifying the state space model directly. Rather than appending convolution after the SSM as MambaIR does, the Pixel Difference Mamba Module updates the state space model with local contrast information. This enables the model to be sensitive to pixel-level spatial relationship while remaining globally aware, without the quadratic computational cost. IRMamba goes further with the Layer Restoration Module to restore details that were lost during downsampling. However, recovering target details without reintroducing noise is a fundamental challenge in feature recovery (M. Zhang et al. 2025a). Results on the

NUDT-SIRST benchmark, displayed in Appendix H.3, demonstrate IRMamba surpassing the best hybrid transformer method by over 1 IoU point while achieving a false alarm rate more than three times lower (M. Zhang et al. 2025a). This suggests the Mamba-based approach is not restricted by domain and remains a well-supported architectural direction against conventional building block limitations.

Sequential and Video Object Detection

The detection of high-speed targets across sequential frames introduces temporal modelling demands that extend beyond single frame detection architecture capabilities. Individual frames often deteriorate due to motion blur and out of focus conditions. Additionally, object variations and occlusion make it difficult to match and aggregate feature information across frames. Conventional convolutional operations struggle to capture long range spatial consistency and motion correlation across frames (Yao et al. 2026).

OVSMMFA-Net addresses these challenges with two modules. The first module extracts features from both the key frame and neighbouring support frames to capture object variations. The second is a multidirectional Mamba-based global feature aggregation module, performing scans in four directions simultaneously and dynamic gating to adaptively model long range spatiotemporal relationships across the video sequence (Yao et al. 2026). Appendix H.4 displays its evaluation on the ImageNet VID benchmark, where the network achieves a mAP of 66.9% with SwinB backbone, outperforming all competitors, including recent transformer-based approaches and other Mamba applications (Yao et al. 2026). However, for small object detection, the AP is significantly worse performing than medium and large objects. The authors attribute this to the limited pixel information in small objects, whose spatial features are often lost in deeper networks (Yao et al. 2026). To further evaluate detection accuracy the models were tested under different speeds, where the model with SwinB backbone again outperformed alternative models for slow, medium and fast object speeds, as shown in Appendix H.4. However, detection of fast-moving objects remains weaker, a limitation the authors flag as unresolved future work (Yao et al. 2026). It is important to note that evaluation was limited to ImageNet VID, a single natural scene benchmark and cannot be generalised to other domains, such as infrared surveillance.

2.8 Knowledge Gaps

The evidence accumulated across the prior sections identifies five critical gaps for the small object detection scenario.

1. **Progressive spatial detail loss**
   Down sampling operations in CNN-based backbones sacrifice spatial resolution in order to capture broader semantic information. This critically reduces the model's ability to characterise small objects occupying few pixels, and hence, classifying them. Domain-specific interventions such as multiscale fusion and super-resolution attempt to recover detail loss, but as established in Section 2.7, these strategies intervene after the representational deficit rather than preventing it (G. Chen et al. 2020).

2. **Complex computational cost**
   While transformers are able to achieve the global context CNNs cannot model, they operate at a quadratic scale, exhausting GPU memory at operational resolutions and require significantly longer inference times than CNN-based alternatives (Guo et al. 2025). This architectural limitation has direct measurable consequences in practical deployment scenarios.

3. **Local spatial structure disruption**
   The application of standard Mamba to visual tasks requires flattening 2D images into 1D

sequences, disrupting local spatial relationships that are critical for small features (M. Zhang et al. 2025a). Fixed scanning strategies address this but introduce spatial discontinuities between semantically related pixels (Xiao et al. 2024). Tree topology was addressed in Section 2.6 as a solution, though it does not address local contrast relationships critical for infrared small target detection.

4. **Persistent small object performance gap**
   Experimental results in video object detection (Section 2.7) show the same drop between the AP of large and small objects (Yao et al. 2026; G. Chen et al. 2020). OVSMMFA-Net, a Mamba-based model yields a small object AP much lower than medium and large objects. This suggests that the building block selection of Mamba alone is not sufficient, motivating specific architectural design for extreme small object detection.

5. **Non-existing study for Mamba in hypersonic target detection**
   The current literature reveals no existing study for the application of a Mamba-based building block specifically to the detection of small hypersonic airborne vehicle targets. The intersection of high-velocity motion, extreme spatial scarcity, adverse sensing conditions and real-time computational constraints has not been addressed as a unified research problem.

2.9 Justified Selection for Mamba

The five gaps identified in Section 2.8 collectively expose failures in conventional building blocks and point to an architecture that will simultaneously preserve pixel-level spatial detail without quadratic costs. Mamba is selected as the candidate building block as it addresses the first two gaps, where its selective mechanism enables content-aware information mixing, and its linear scaling overcomes the dilemma between spatial sensitivity and computational efficiency. This selection identifies Mamba's selective state space mechanism as an adaptable architectural foundation. As demonstrated by IRMamba and MambaIR, the core properties of linear scaling and content-aware mixing provide the capabilities upon which domain-specific modifications refine. The absence of existing research connecting Mamba-based building blocks to hypersonic airborne vehicle detection, the combination of high-velocity motion, extreme spatial scarcity, infrared sensing conditions and computational constraints, represents a clear research gap that motivates this work.

Mamba is selected on the basis of its linear scaling in sequence length and content-aware information mixing. Given the availability of an open-source PyTorch implementation, Mamba can be directly integrated for practical evaluation. There remains the question of how it should be adapted for small high-speed airborne target detection.

2.10 MOCID: A Domain-Specific Precedent

M. Zhang et al. (2025b) characterise existing methods of Moving Infrared Small Target Detection (MIRSTD) as typically following a framework that consists of two independent and sequential modules. First, a spatial-only feature extractor processes each frame, after which a separate temporal module models target features using contextual information across frames (Gong et al. 2021; Zhou et al. 2023; R. Li et al. 2023; S. Chen et al. 2024). The authors argue that without incorporating motion and contextual information during spatial modelling, detectors are deprived of discriminative motion cues that separate targets from noise. This structural weakness introduces inaccuracies in temporal modelling in subsequent modules, particularly against complex backgrounds, resulting in error-prone target localisation. MOCID's core design response is to inject temporal motion and displacement information into the feature extraction process itself, at two levels of granularity. Clip-level processing spans the whole input sequence, and frame-level processing compares specific frame pairs.

2.11 Justification of the Candidate Adaptation

MOCID was chosen as the host because it is an existing multi-frame infrared detector with a published baseline. Its Displacement-aware Mamba Module (DAM) was chosen as the point of change because it is the only part of MOCID built from Mamba. Changing that module alone keeps the backbone, pool and head fixed, so each new design can be compared directly with the original. Table 4 summarises the adaptations reviewed and the DAM limitation each one targets.

Table 4: Mamba adaptations considered for the DAM, the weakness of a plain scan each addresses, and its relevance to MOCID. IRMamba and OVSMMFA-Net informed the review but were not implemented.

Adaptation	Weakness addressed	Mechanism	Relevance to the DAM
MambaIR	Local pixel forgetting	Local convolution, channel attention, learned skip scales	Adds spatial detail around the scan, implemented
CFMW	Each frame seen in isolation	Two images share one scanned sequence	Adds scene-wide context, implemented
MambaTree	Fixed reading order	Scan follows a tree built from image content	Replaces the raster order, implemented
IRMamba	Lost local contrast	Pixel differences inside the SSM	MOCID's SDS already uses difference convolution
OVSMMFA-Net	Single scan direction	Four-direction scanning with gating	TIDS already scans in several directions

MambaIR was worth testing because its evidence for recovering local detail is strong, and the DAM's output is otherwise processed only by simple projections. Local convolution and channel attention around the scan offered a way to restore neighbourhood information without altering the comparison between frames (Guo et al. 2025).

CFMW was considered because its Cross-modality Fusion Mamba addresses a structurally similar problem, relating two images of the same scene (H. Li, Q. Hu, et al. 2025). Rather than fusing the two images with attention as seen in Transformer-based architectures, CFM pools both frames to a coarse grid of tokens, places the tokens of both images in one sequence and scans it, so that the state carries information from one image into the other at linear cost. The DAM's own scan, TIDS, compares only spatially corresponding tokens of the target and reference frames, and therefore models local displacement only. A CFM-style branch therefore offers a way to add global cross-frame context, which could help separate a target's own motion from scene-wide change such as camera motion, without replacing the temporal scan on which MOCID relies.

Like most visual Mamba models, MOCID's TIDS flattens each feature map row by row in a raster scan. Vertically adjacent pixels can therefore end up far apart in the sequence, and the reading order is the same whatever the image contains. MambaTree was considered because it addresses this limitation by building a tree topology from the input. The tree scan joins neighbouring pixels with similar features and propagates the state along these links, so information from the whole image reaches every pixel in one pass (Xiao et al. 2024). Since the tree scan can replace the raster scan inside TIDS without altering the frame-difference selection or temporal interleaving, it allows a clean ablation of scan order.

2.12 Conclusion

The literature review evaluated four principal architectural families, RNNs, CNNs, Transformers and SSMs, against the demands of small, high-speed targets and infrared sensing conditions. Sections 2.3 to 2.6 found conventional building blocks satisfying some criteria while failing others. RNNs perform inefficiently due to fixed bottleneck memory and sequential processing. CNNs suffer progressive spatial detail loss through fixed local receptive fields and downsampling.

Transformers have the ability to capture global context but are operationally prohibitive at larger resolutions due to quadratic complexity. Domain-specific interventions surveyed in Section 2.7 find strategies addressing these architectural deficits, however, they occur after feature extraction, where detail loss is irrecoverable. Mamba-based approaches replace the backbone itself, addressing the problem at feature extraction. These architectures have impressive results across image restoration, infrared detection and video object detection, outperforming strong CNN-based and Transformer-based competitors. The reviewed papers provide strong evidence that Mamba's linear scaling and content-aware information mixing overcome identified gaps in traditional building blocks. The specific adaptation of this building block for the small high-speed airborne target detection scenario constitutes the research problem this work investigates.

MOCID applies Mamba to moving infrared small-target detection and serves as this work's base detector. Its displacement-aware module (DAM) aligns earlier frames with the current frame through a selective scan (M. Zhang et al. 2025b). Since the DAM is the Mamba component of MOCID, it was selected as the site of modification. By limiting changes to this module, multiple designs can be compared directly with the original. Three variants were tested to address different limitations of the DAM. MambaIR adds local processing and channel attention to recover spatial detail. CFMW adds global context across frames to separate target motion from scene-wide change. MambaTree replaces the fixed row-by-row order with a tree scan that keeps similar neighbouring pixels together. Section 3 now sets out the theory on which all three depend.

3 Theory and Technical Description of Mamba

3.1 Overview

The systematic literature review described in Section 2 identified Mamba (Gu and Dao 2024) as the modern neural network building block selected for integration into the project's object detection backbone. Mamba is a selective state space model (SSM) that achieves linear-time scaling in sequence length while preserving the long-range modelling capability typically associated with attention. This section develops the theoretical foundations required to understand Mamba's operation and its implications for small-object detection in satellite imagery.

This discussion begins with classical state space models and their continuous-time formulation, before introducing the discretisation step that allows them to operate on sequence data. The dual recurrent and convolutional representations of structured SSMs are then derived, motivating both the linear time-invariance (LTI) assumption used by earlier models and the limitations that this assumption imposes. The selection mechanism that distinguishes Mamba from prior structured SSMs is introduced next, followed by the hardware-aware algorithm that makes the resulting time-varying recurrence computationally tractable on modern GPUs. The section concludes with a description of the Mamba block architecture and a discussion of its suitability for the project's object detection task. A complementary intuitive walk-through using a vehicle-monitoring analogy and a language modelling analogy, together with a fully-worked numerical example, are provided in Appendix A and Appendix B respectively.

For this project, the sequence is built from image features. Its positions are not words, and the state is not a physical estimate of aircraft position or velocity. It is a learned representation used to produce features for the detector. Appendix A begins with a physical analogy, then distinguishes that analogy from the learned model.

3.2 State Space Models

A state space model represents a dynamic system through a latent state vector $h(t) \in \mathbb{R}^N$ that summarises the relevant history of an input signal $x(t) \in \mathbb{R}$, and from which an output $y(t) \in \mathbb{R}$ is

produced. The state itself is not directly observed. Only the input applied to the system and the output produced from it are observable. The role of the model is to learn a state representation that is strong enough to predict future outputs from future inputs.

In the continuous-time, linear formulation used by structured SSMs, the system is described by

$$h'(t) = \mathbf{A}\, h(t) + \mathbf{B}\, x(t), \tag{1}$$

$$y(t) = \mathbf{C}\, h(t), \tag{2}$$

where $h'(t)$ is the first-order derivative of $h(t)$, $\mathbf{A} \in \mathbb{R}^{N\times N}$ is the state transition matrix, $\mathbf{B} \in \mathbb{R}^{N\times 1}$ projects the input into the latent space, and $\mathbf{C} \in \mathbb{R}^{1\times N}$ reads the output from the latent state. Equation (1) is the *state equation* and describes how the latent state evolves over time. Two terms drive this evolution: the term $\mathbf{A}\, h(t)$ describes how the current state propagates forward of its own accord, and the term $\mathbf{B}\, x(t)$ describes how the new input modifies the state. Equation (2) is the *output equation* and describes how the state is read out as an observable quantity through $\mathbf{C}$. A direct feed-through term $\mathbf{D}\, x(t)$ is sometimes included in the output equation (Gu et al. 2021a). It is omitted here for clarity and restored in the selective form of Section 3.4. An architectural diagram of this is exhibited in Figure 4.

The three matrices encode three distinct mechanisms. The matrix $\mathbf{A}$ encodes the natural dynamics of the system in the absence of input. In a physical setting it captures decay, oscillation, or drift. In a sequence setting it controls how strongly past information is retained or attenuated. The matrix $\mathbf{B}$ encodes how new inputs are incorporated into the state, determining which dimensions of the latent representation a given input affects. The matrix $\mathbf{C}$ encodes how the state is decoded into an observable output, determining which combinations of state dimensions are relevant for prediction. A worked numerical example using a vehicle-monitoring system is provided in Appendix A, and a similar treatment for language modelling is given in the same appendix.

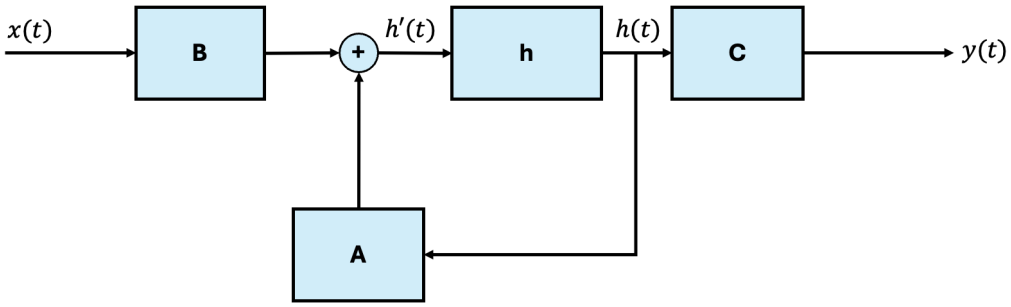


Figure 4: Block diagram of a continuous-time linear state space model. The latent state $h(t)$ is updated through $\mathbf{A}$ from its previous value and through $\mathbf{B}$ from the input $x(t)$. The output $y(t)$ is produced from the latent state through $\mathbf{C}$.

Discretisation

Inputs to deep learning models are discrete tensors rather than continuous signals, so the continuous SSM in Equations (1) and (2) must be discretised. A learned timestep parameter $\Delta \in \mathbb{R}_{>0}$ defines the spacing between samples, and the zero-order hold (ZOH) rule (Gu and Dao 2024) converts the continuous parameters $(\mathbf{A}, \mathbf{B})$ to their discrete counterparts $(\overline{\mathbf{A}}, \overline{\mathbf{B}})$ via

$$\overline{\mathbf{A}} = \exp(\Delta\mathbf{A}), \tag{3}$$

$$\overline{\mathbf{B}} = (\Delta\mathbf{A})^{-1}(\exp(\Delta\mathbf{A}) - \mathbf{I}) \cdot \Delta\mathbf{B}. \tag{4}$$

The resulting discrete recurrence is

$$h_t = \overline{\mathbf{A}}\, h_{t-1} + \overline{\mathbf{B}}\, x_t, \tag{5}$$

$$y_t = \mathbf{C}\, h_t. \tag{6}$$

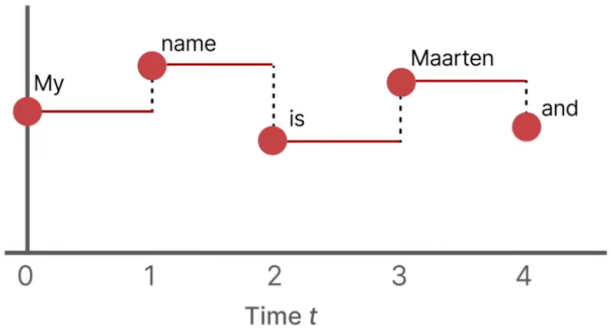
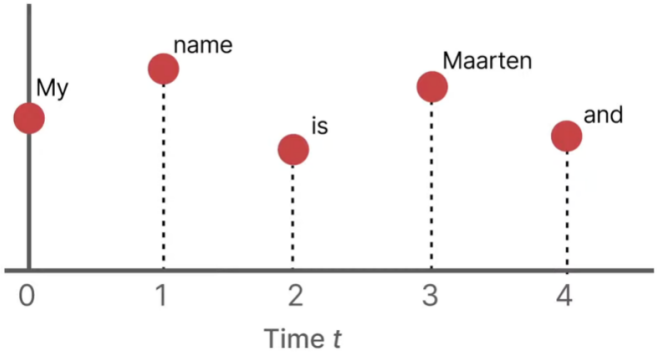
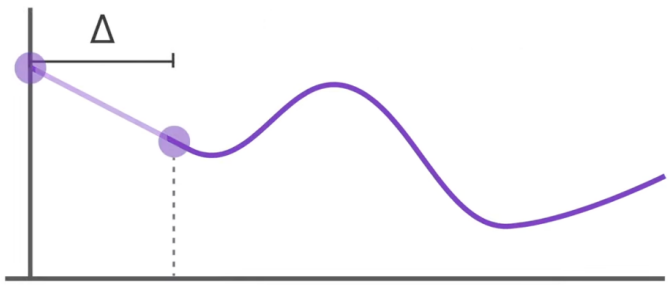


Figure 5: The zero-order hold rule samples a continuous signal at a fixed step size Δ. The learnable parameter Δ controls how finely the underlying continuous dynamics are sampled and therefore how aggressively the state is updated at each step (Grootendorst 2024).



(a) Discrete token sequence (the actual input)(Grootendorst 2024).



(b) The same sequence held as a piecewise-constant signal, as if it were continuous (Grootendorst 2024).

Figure 6: Conceptual view of the zero-order hold. A discrete token sequence (top) is treated as a piecewise-constant continuous signal (bottom) by holding each value over an interval of width Δ. The continuous-time SSM dynamics are integrated over this signal, and the result is sampled to give the discrete recurrence used in practice.

The discretisation step is therefore not just a numerical convenience. Δ controls the effective

time scale on which the state retains information, and is itself a learnable parameter (Gu and Dao 2024). This sampling is shown in Figure 5 and, for a token sequence, in Figure 6.

The recurrence above is written for a single-channel input. In practice the input has multiple feature channels, and the SSM is applied independently to each one. The full hidden state therefore has shape (B, L, D, N), where B is the number of training examples processed at once, L is the number of positions in the sequence, D is the number of feature channels per position, and N is the SSM's internal state dimension. With $N \approx 16$, this hidden state is an order of magnitude larger than the input itself, which is the central concern of the hardware-aware algorithm described in Section 3.5. Following Gu and Dao (2024), D here denotes the channel count; elsewhere in this report, where feature maps are written with shape (B, C, H, W), the same quantity is written C. The value $N \approx 16$ is the common default; the recorded MOCID module uses $N = 32$. A full explanation of these dimensions, with the project's satellite-imagery setting as a worked example is provided in Appendix D.

The continuous-time parameterisation provides a convenient way to construct the discrete transition. For a diagonal state matrix $\mathbf{A}$ and a positive step parameter Δ_t,

$$\overline{\mathbf{A}}_t = \exp(\Delta_t \mathbf{A}). \tag{7}$$

The learned Δ_t controls an internal memory scale. It is not the camera frame interval. Images enter the implementation as discrete arrays, so the model does not reconstruct a continuous video before processing it. Appendix C distinguishes the exact zero-order-hold input term from the simplified input update commonly used in selective-scan implementations.

3.3 Recurrent and Convolutional Representations

Structured SSMs can be computed in two equivalent ways (Gu et al. 2021b). The recurrent form is given directly by Equations (5) and (6), and is the natural method for generating outputs one token at a time, where each new token depends on those that came before. Computing each output requires only a fixed amount of arithmetic (independent of how long the sequence is so far), since only the most recent hidden state needs to be carried forward. The recurrent form is, however, inherently sequential. The hidden state at position t cannot be computed until the hidden state at position $t-1$ is known. This is a problem during training, when the entire input sequence is available at once and ought to be processed in parallel across the GPU. Iterating through the sequence one position at a time leaves most of the hardware idle.

Derivation of the Convolutional Form

The convolutional form arises by unrolling the recurrence. Initialising $h_{-1} = 0$ and substituting Equation (5) successively, the first three outputs are

$$h_0 = \overline{\mathbf{B}}\, x_0, \qquad y_0 = \mathbf{C}\overline{\mathbf{B}}\, x_0, \tag{8}$$

$$h_1 = \overline{\mathbf{A}}\, h_0 + \overline{\mathbf{B}}\, x_1, \qquad y_1 = \mathbf{C}\overline{\mathbf{A}}\,\overline{\mathbf{B}}\, x_0 + \mathbf{C}\overline{\mathbf{B}}\, x_1, \tag{9}$$

$$h_2 = \overline{\mathbf{A}}\, h_1 + \overline{\mathbf{B}}\, x_2, \qquad y_2 = \mathbf{C}\overline{\mathbf{A}}^{\,2}\overline{\mathbf{B}}\, x_0 + \mathbf{C}\overline{\mathbf{A}}\,\overline{\mathbf{B}}\, x_1 + \mathbf{C}\overline{\mathbf{B}}\, x_2. \tag{10}$$

The pattern that emerges is that each output is a weighted sum of past inputs, where the weight applied to an input j steps in the past is $\mathbf{C}\overline{\mathbf{A}}^{\,j}\overline{\mathbf{B}}$. In general,

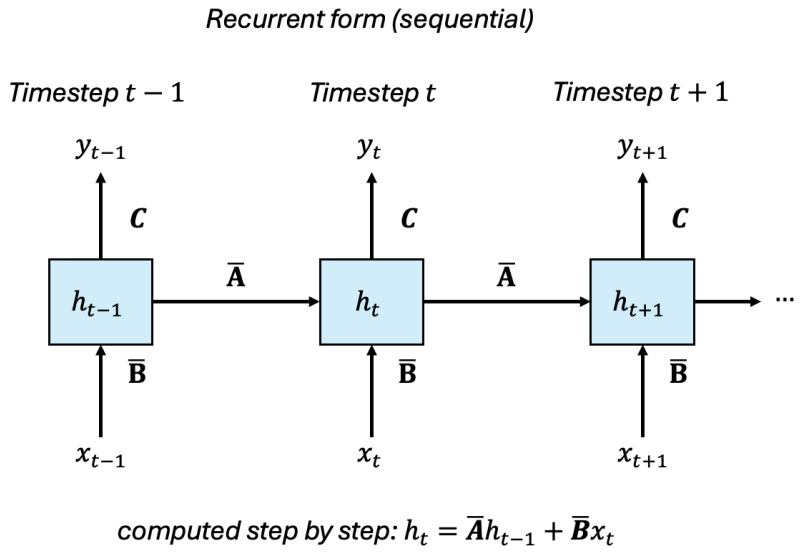
$$y_t = \sum_{k=0}^{t} \mathbf{C}\,\overline{\mathbf{A}}^{\,k}\,\overline{\mathbf{B}}\, x_{t-k}. \tag{11}$$

Equation (11) has the structure of a one-dimensional discrete convolution. Defining the SSM convolution kernel

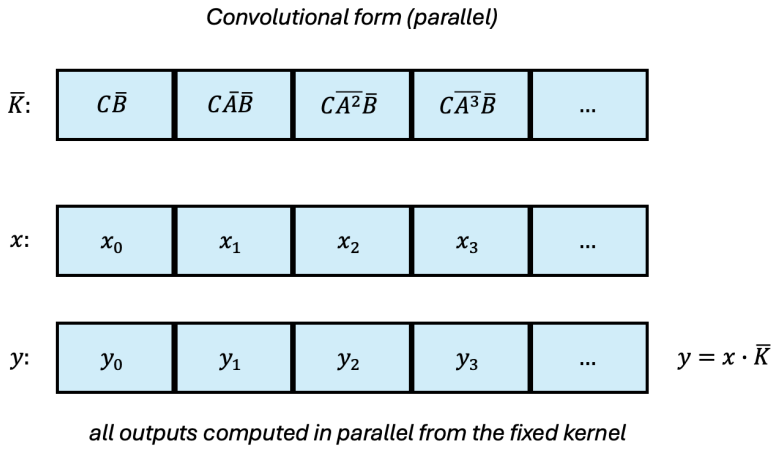
$$\overline{\mathbf{K}} = \left(\mathbf{C}\overline{\mathbf{B}},\ \mathbf{C}\overline{\mathbf{A}}\,\overline{\mathbf{B}},\ \mathbf{C}\overline{\mathbf{A}}^{\,2}\overline{\mathbf{B}},\ \ldots,\ \mathbf{C}\overline{\mathbf{A}}^{\,L-1}\overline{\mathbf{B}}\right), \tag{12}$$

the entire output sequence can be computed as a single one-dimensional convolution $y = x * \overline{\mathbf{K}}$. The kernel has length equal to the sequence length, and its entries decay (or grow) according to the eigenvalues of $\overline{\mathbf{A}}$, controlling how strongly past inputs contribute to the present output.

The convolutional form is parallelisable across the length dimension, since every output position can be computed independently once the kernel is known. It is therefore the natural method for training, where the entire input is available at once. A fully-worked numerical example showing the equivalence between the recurrent and convolutional forms is provided in Appendix B. Both forms are shown in Figure 7.



(a) Recurrent form: the hidden state is computed sequentially through repeated application of $\overline{\mathbf{A}}$ and $\overline{\mathbf{B}}$.



(b) Convolutional form: the output is computed as a one-dimensional convolution of the input with a fixed kernel $\overline{\mathbf{K}}$, whose entries are powers of $\overline{\mathbf{A}}$ multiplied by $\overline{\mathbf{B}}$ and $\mathbf{C}$.

Figure 7: The two equivalent representations of a linear time-invariant SSM.

Initialisation of the Transition Matrix

The convolution kernel $\overline{\mathbf{K}}$ in Equation (12) contains powers of $\overline{\mathbf{A}}$, so the choice of $\mathbf{A}$ at initialisation determines how strongly past inputs contribute to the current output before any training has taken place. If the eigenvalues of $\overline{\mathbf{A}}$ are small in magnitude, the kernel decays rapidly. An input from many positions in the past contributes negligibly to the present output, and the model effectively cannot learn long-range dependencies. If the eigenvalues are too close to one, the kernel decays too slowly and gradients become unstable during training. A poorly-chosen initialisation therefore makes long-range modelling difficult regardless of how the model is trained.

A range of structured initialisations have been proposed to address this. The most influential is the High-order Polynomial Projection Operator (HiPPO) framework of Gu et al. (2020), which derives a specific form for $\mathbf{A}$ such that the latent state acts as a continuously-updating compression of the input history onto a basis of orthogonal polynomials. Variants used in practice include the diagonal HiPPO initialisations of Gu et al. (2021a), denoted S4D-Lin and S4D-Real, which retain the long-range properties of the original HiPPO matrix while reducing computational cost. Gu and Dao (2024) report that for selective SSMs, simpler real-valued initialisations such as $\mathbf{A}_n = -(n+1)$ perform comparably to the more complex HiPPO forms, and that random initialisations also work well in the large-data regime. The detailed structure of $\mathbf{A}$ matters less than the general principle that the eigenvalues must be chosen so that the convolution kernel covers a useful range of decay rates.

Limitations of Linear Time-Invariance

The dual recurrent–convolutional formulation rests on a critical assumption. The parameters $(\Delta, \mathbf{A}, \mathbf{B}, \mathbf{C})$ are constant across time, making the model linear time-invariant. This invariance is what allows the recurrence to be expressed as a fixed convolution, but it also means that the model cannot adapt its dynamics to the content of the input. In practice, an LTI SSM treats every token in a sequence with the same transition rule, regardless of whether the token carries useful information or is irrelevant noise. Gu and Dao (2024) demonstrate this limitation on synthetic tasks such as Selective Copying, where the spacing between informative tokens is randomised. LTI models cannot solve the task because doing so requires content-dependent gating of the recurrence.

In a linear time-invariant model, these coefficients do not change with sequence position. Under a zero initial state, the recurrence can then be written as a convolution with a fixed kernel. In a selective model, coefficients depend on the input, so a single fixed convolution no longer describes the whole sequence. Appendix B works through both forms numerically, and Appendix C explains the selection operations.

3.4 The Selection Mechanism

Mamba addresses the LTI limitation by making the parameters Δ, $\mathbf{B}$, and $\mathbf{C}$ functions of the input, so that the recurrence becomes time-varying. Specifically,

$$s_B(x) = \text{Linear}_N(x), \tag{13}$$

$$s_C(x) = \text{Linear}_N(x), \tag{14}$$

$$s_\Delta(x) = \text{Broadcast}_D(\text{Linear}_1(x)), \tag{15}$$

with Δ further passed through a softplus to ensure positivity (Gu and Dao 2024). The discretised parameters $(\overline{\mathbf{A}}, \overline{\mathbf{B}})$ now depend on the position t in the sequence, and the model can selectively propagate or forget information based on the content of each token. The transition matrix $\mathbf{A}$ itself is left input-independent, since selectivity in Δ already produces selectivity in $\overline{\mathbf{A}}$ through the discretisation rule. Figure 8 contrasts S4 and S6.

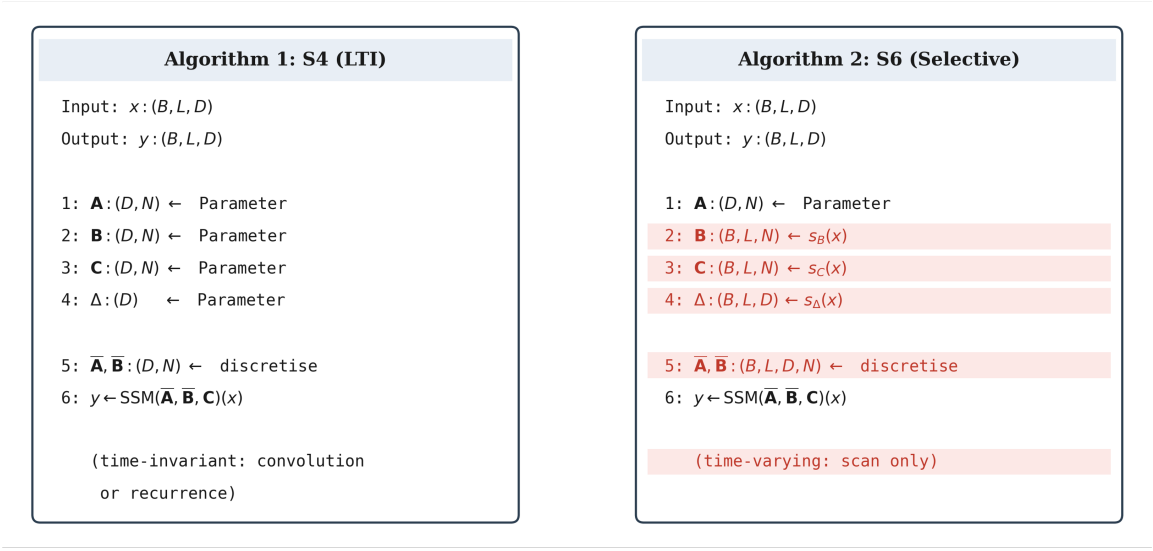


Figure 8: Comparison of the S4 and S6 (selective) algorithms. In S4, $\mathbf{B}$, $\mathbf{C}$, and Δ are static parameters, allowing the recurrence to be expressed as a global convolution. In S6, $\mathbf{B}$, $\mathbf{C}$, and Δ are produced by linear projections of the input and therefore vary across the sequence length, making the model content-aware at the cost of breaking the convolutional form.

The selection mechanism implements a form of input-dependent information mixing along the sequence dimension. Gu and Dao (2024) relate this mechanism to the gating mechanisms of classical recurrent neural networks. When $N = 1$, $\mathbf{A} = -1$, and $\mathbf{B} = 1$, the selective SSM reduces exactly to the gated recurrence

$$g_t = \sigma(\mathrm{Linear}(x_t)), \qquad h_t = (1 - g_t)\, h_{t-1} + g_t\, x_t, \tag{16}$$

recovering the form of an LSTM-style gate. The selection mechanism therefore generalises the principle of content-dependent gating to a structured high-dimensional recurrence. A complete numerical walk-through of the selection mechanism, including a from-first-principles explanation of each operation appearing in Equations (13)–(15) is presented in Appendix C.

With input-dependent parameters, the discrete recurrence of Equations (5) and (6) becomes time-varying. Restoring the direct term, it is written

$$\mathbf{h}_t = \overline{\mathbf{A}}_t \mathbf{h}_{t-1} + \overline{\mathbf{B}}_t x_t, \tag{17}$$

$$y_t = \mathbf{C}_t \mathbf{h}_t + D x_t. \tag{18}$$

The input x_t is the feature value at position t. The state $\mathbf{h}_t$ contains N memory values. The matrix $\overline{\mathbf{A}}_t$ controls how the previous state carries forward, $\overline{\mathbf{B}}_t$ writes the input into the state, and $\mathbf{C}_t$ reads the state into the output. The term Dx_t is a direct input path. It is distinct from the residual connection around the full neural network block.

The cost of this generalisation is that the convolutional form is no longer available, since the kernel in Equation (12) is defined only when $\overline{\mathbf{A}}$ and $\overline{\mathbf{B}}$ are constant. Selective SSMs must therefore be evaluated through the recurrence directly, which presents a parallelisation challenge addressed in the next section.

3.5 Hardware-Aware Implementation

Now that the recurrence depends on the input, the convolutional form is no longer available. The recurrence must be evaluated step by step, and on a modern GPU this raises two problems. First, the steps cannot be done at the same time, because each step needs the result of the

previous one. Second, the hidden state takes up a lot of memory: with $N \approx 16$, the full hidden state tensor of shape (B, L, D, N) is sixteen times larger than the input. Moving such a tensor in and out of GPU memory is slow, and on most modern hardware this movement, rather than the arithmetic itself, is the bottleneck (Dao et al. 2022). Mamba's selective scan implementation uses three techniques to overcome these problems, all of which are described in detail in Appendix E.

Parallel scan. Although the recurrence is sequential as written, it can be reorganised into a tree of partial computations and evaluated in roughly $\log_2 L$ parallel steps using a work-efficient parallel scan algorithm (Blelloch 1990). This recovers the parallelism that was lost when the convolutional form was given up.

Kernel fusion. A naive implementation would shuttle the large hidden state tensor between fast and slow regions of GPU memory many times. Mamba avoids this by performing all of the SSM's intermediate computations within the GPU's small but fast on-chip memory, and only writing the final output back to the larger but slower main memory. This reduces memory traffic by a factor of N and yields a measured speedup of 20–40× over the naive implementation (Gu and Dao 2024).

Recomputation. The intermediate hidden states are required when computing gradients during training, but storing them would undo the savings of kernel fusion. Mamba instead discards them after the forward pass and recomputes them when needed. The extra arithmetic is cheap compared to the memory traffic that has been avoided. This traffic is summarised in Figure 9.

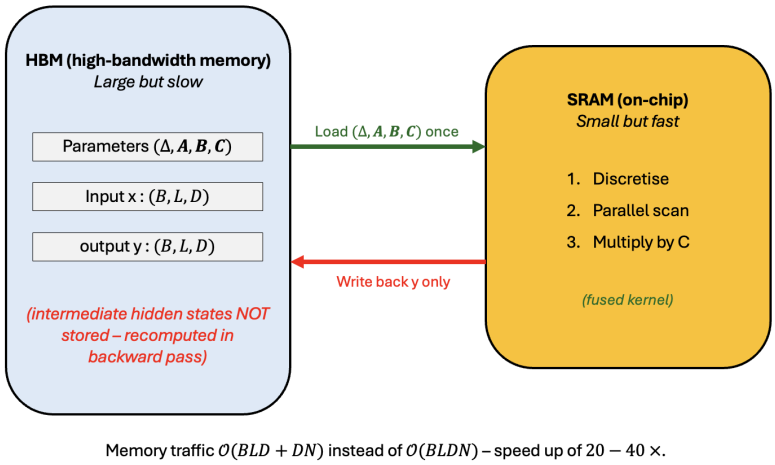


Figure 9: Memory traffic in the fused selective scan. The SSM parameters are loaded from HBM into SRAM once. Discretisation, scan, and output projection are all performed in SRAM. Only the output tensor is written back to HBM. The intermediate hidden states are not stored and are recomputed during backpropagation.

The combined algorithm of input-dependent parameters, parallel scan, kernel fusion, and recomputation is referred to as the *selective scan* or S6 layer.

Implementation efficiency in this project

A recurrence written as a Python loop can be slow even when its arithmetic cost is linear in sequence length. A GPU benefits from many operations running together and from avoiding unnecessary transfers of intermediate values. A fused selective-scan kernel addresses those implementation costs. The repository depends on a VMamba scan implementation, and the

project records describe building a compatible CUDA kernel before practical stage-2 training could proceed.

Linear scaling also needs a precise scope. It describes the scan with feature width and state size fixed. It does not mean the entire detector is linear in every design variable, nor that it is faster than every convolutional detector. The model still contains convolutions, feature pyramids, Fourier operations, and box processing. Appendix D.2 explains the memory dimensions and parallel scan, and Section 7.8 reports the measured timings.

3.6 The Mamba Block

The S6 layer transforms one sequence into another and must be embedded into a neural network block for use in a deeper architecture. The Mamba block, illustrated in Figure 10, combines the design of the H3 block from earlier SSM architectures with the gated multilayer perceptron (MLP) block used in modern Transformers (Gu and Dao 2024). The input is first passed through a linear projection that expands its feature dimension by a factor E (set to 2 by default), producing two branches. The first branch is passed through a short one-dimensional convolution, a SiLU activation, and the S6 layer. The second branch is passed through a SiLU activation alone, and acts as a gating signal that multiplies the output of the first branch element by element, controlling how much of it is allowed through. A final linear projection contracts the feature dimension back to its original size, and a residual connection together with a normalisation layer completes the block.

The block is stacked many times to form the full Mamba architecture. Unlike the H3 architecture, which alternates SSM blocks with separate MLP blocks, the Mamba block already contains both a sequence-mixing component (the S6 layer) and a gated channel-mixing component (the two branches and their multiplication), so it can be stacked uniformly. This homogeneous stacking makes the block easy to integrate into existing detection backbones, since each block has the same input and output shape and therefore behaves like a standard residual unit.

3.7 From a sequence to an image block

An image feature map contains a vector at each spatial location. A scan arranges these vectors into a sequence, processes the sequence, and returns the results to their spatial locations. The arrangement matters because nearby positions in the sequence exchange state directly. Two neighbouring image pixels need not be neighbours under a simple row-by-row ordering.

The project's MambaIR adaptation keeps the temporal scan already used by MOCID and changes the processing around it. Local convolutions provide a route for nearby spatial information, while channel attention changes the relative contribution of different feature channels. These additions are motivated by the MambaIR design, but the resulting module is an adaptation for detection rather than the complete published restoration network. Its actual structure is described in Section 5.7. The CFMW adaptation changes what the scan combines rather than its order. A branch modelled on the Cross-modality Fusion Mamba scans the target and reference feature maps together, so that each position in the target frame can draw on the whole of the reference frame and not only the position it overlaps. This allows the displacement module to judge each local change against the motion of the rest of the scene. Its structure is described in Section 5.9. The MambaTree adaptation replaces the scan order itself. Instead of a fixed row-by-row ordering, it links each pixel to its most similar neighbours in a tree structure. The pixels exchange state along these links, so adjacent pixels with similar features remain close in the scan. Its structure is described in Section 5.11.

3.8 Suitability for Small-Object Detection

Three properties of Mamba motivate its selection for the satellite imagery detection task described in Section 1. First, the selection mechanism allows the model to suppress background regions while preserving fine-grained features associated with small targets. Background suppression is particularly relevant in satellite imagery, where targets occupy a small fraction of the image area and the surrounding context is dominated by clutter (G. Chen et al. 2020). Second, the linear scaling of compute and memory with sequence length means that high-resolution feature maps, which are valuable for small-object detection but expensive under attention, can be processed without the quadratic cost of self-attention (Fournier et al. 2023). Third, recent work on Mamba-based image restoration (Guo et al. 2025) has demonstrated that the architecture can be adapted to two-dimensional spatial inputs by serialising spatial tokens with appropriate scan orderings, providing a precedent for use within a convolutional or hybrid detection backbone.

The integration of the Mamba block into a fixed object detection backbone is the subject of Objective 4 and is treated in Sections 4 and 6. Of particular note for the integration is that the block produces and consumes tensors of the same shape, which permits direct substitution for an existing residual unit without modification of the surrounding pipeline.

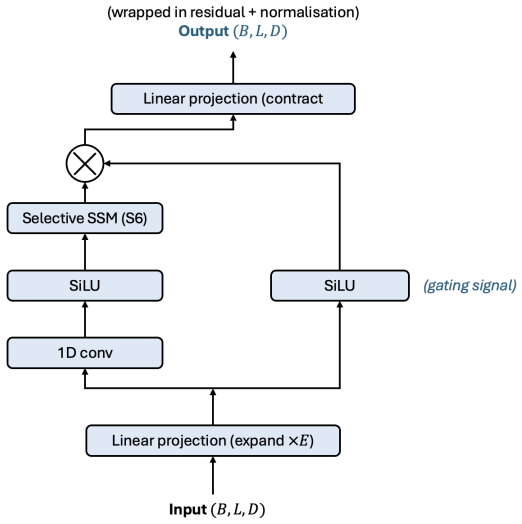


Figure 10: The Mamba block. The input projection expands the model dimension by a factor E and produces two branches. The first branch applies a one-dimensional convolution, a SiLU activation, and the selective SSM (S6). The second branch produces a gating signal through a SiLU activation. The branches are combined by element-wise multiplication and projected back to the model dimension. The block is wrapped in a residual connection and normalisation layer (not shown).

3.9 Interface requirements

The module consumes target and reference feature maps with equal shape (B, C, H, W) and returns one feature map of that shape. Preserving the shape permits replacement within the displacement network, but does not by itself preserve the detector's behaviour. The output must represent the intended reference contribution and must preserve the stage-1 signal at initialisation.

3.10 Summary

This section developed the theoretical foundations of Mamba in three stages. The classical state space model and its discretisation establish the linear recurrence on which all structured SSMs are built. The dual recurrent–convolutional representation explains how earlier LTI models

such as S4 achieved efficient training, and motivates the selection mechanism that distinguishes Mamba from its predecessors by making the recurrence content-aware. The hardware-aware algorithm restores parallelism and memory efficiency to the resulting time-varying recurrence through parallel scan, kernel fusion, and recomputation. The Mamba block embeds the selective SSM into a homogeneous residual unit suitable for stacking, and its properties of selective information mixing, linear scaling, and shape-preserving interface make it a strong candidate for the small-object detection problem investigated in this project. The implementation of the block in PyTorch and the verification of its functional correctness are described in Sections 5 and 6.

These requirements connect the theory to the engineering work. Selection provides a way to mix information, scan ordering determines which information is adjacent, and the residual path determines what the rest of the detector sees when a new module is introduced. Section 4 now places those mechanisms inside the complete detector.

4 MOCID and the Implemented Detection Architecture

4.1 Overview

Section 3 explained how a selective scan works on a single sequence. MOCID shows how such a scan can serve a real detector. MOCID combines information across a short infrared clip before predicting boxes in its final frame. The published design introduces Fourier-inspired spatio-temporal attention, or FISTA, for clip-level processing and a displacement-aware Mamba module, or DAM, for frame-pair processing (M. Zhang et al. 2025b). The implementation used in this project follows that design, with its differences noted below.

Figure 11 shows the complete detector and the block replaced in the MambaIR, CFMW and MambaTree experiments. The backbone, pooling, feature pyramid and detection head remain in place while the DAM changes, so any difference in the results can be attributed to the DAM. The figure also shows where each design acts inside the block.

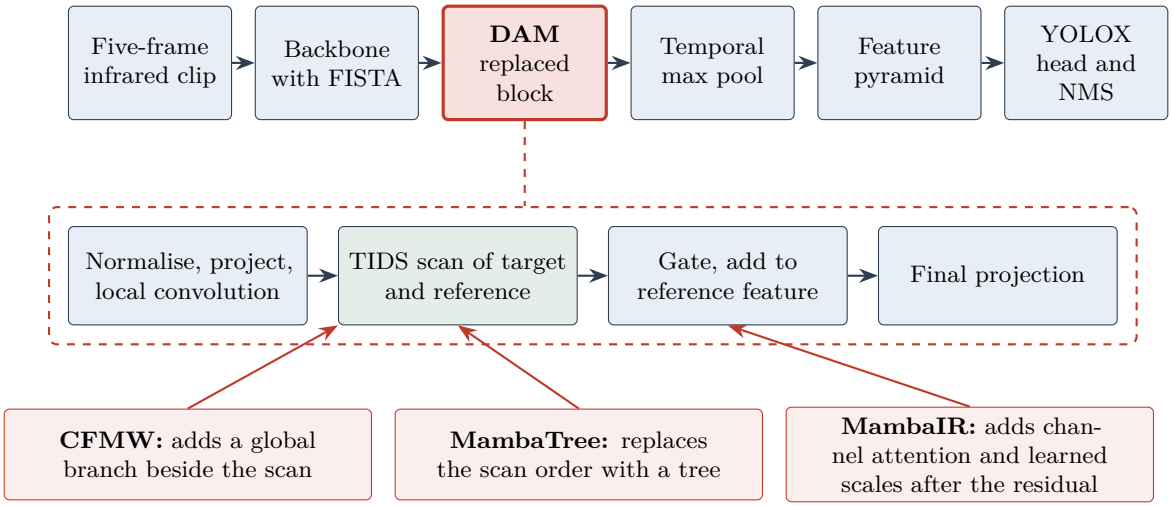


Figure 11: The MOCID detector and the three designs tested in this project. The top row is the complete inference path, in which only the highlighted DAM changes between experiments. The dashed box expands the DAM block, and the lower row marks where each design acts. In stage 1, raw backbone features bypass the DAM and enter the same temporal pool.

4.2 Clip-level feature extraction

The backbone first extracts spatial features from every frame. Three subsequent FISTA stages operate at strides 8, 16, and 32, producing 128, 256, and 512 channels. At a 512×512 input size, their spatial sizes are 64×64, 32×32, and 16×16. The implementation uses four, four, and one FISTA blocks at these stages. Section 4.5 explains the Fourier processing with small examples.

Each FISTA block proceeds in two stages, following the approach of GFNet (Rao et al. 2021) for the spatial stage.

Spatial stage: a 2D Discrete Fourier Transform (DFT) is applied to each frame independently, the result is multiplied by a learnable spatial filter, and an inverse DFT restores the spatial domain. This step is intended to emphasise high-frequency content, where small-target edge and shape information is concentrated.

Temporal stage: a 1D DFT is applied across the temporal axis at each spatial location, multiplied by a learnable temporal filter, and inverted. The amplitude of the resulting signal is high where a pixel location exhibits distinctive temporal dynamics, theoretically corresponding to target motion rather than a static background.

The L_2-normalised amplitude of this temporal signal is multiplied element-wise with the original features to produce M, the clip-level motion context. M is then used to modulate the backbone's convolution kernels for each frame individually, via a squeeze-and-excitation-style calibration: M is spatially pooled, passed through a fully connected layer operating across the temporal dimension, and used as a per-frame scalar weight on a shared base convolution kernel (M. Zhang et al. 2025b, Eqs. 5–6). By stacking three FISTA layers, motion context is progressively refined and re-applied through the backbone.

The implementation uses a real-input Fourier representation and a residual addition around the FISTA block. It also uses channel bottlenecks whose detailed widths are not specified by the paper. These are explicit implementation choices. Similar parameter counts provide a useful structural check, but cannot establish that all internal operations match the authors' model.

4.3 Frame-pair displacement processing

The DAM is the component of MOCID responsible for modelling frame-to-frame displacement (M. Zhang et al. 2025b). The DAM is the only part of MOCID built from the Mamba architecture, and it operates on the feature maps produced by the spatio-temporal backbone (STB) (M. Zhang et al. 2025b). It adapts the selective state space formulation of Gu and Dao (2024) to operate on frame pairs, each formed from the target frame and one reference frame. DAM does not process the whole clip at once. It runs $T-1$ times, once per reference frame, with shared weights, and the outputs are combined by temporal pooling (M. Zhang et al. 2025b). Unlike the STB, which captures clip-level motion context, the DAM targets subtle pixel shifts between frames. Displacement is a distinct signal from broad motion context, and detecting it helps separate a moving target from misleading noise, whether that noise is static or drifting.

DAM achieves this through Temporal Interpolation and Difference-aware Selective Scan (TIDS), which consists of two parts.

Spatio-temporal Difference Selection (SDS): The target and reference frames are concatenated along the temporal axis, combining two frames into a single input x. The scan parameters B, C and Δ are generated by applying 3D central difference convolution (Yu et al. 2021) to the input:

$$B,\ C,\ \Delta = \mathrm{3DCDC}(x). \tag{19}$$

This departs from the original Mamba, where scan parameters B, C and Δ are generated by linear projections, each applying a learned weight matrix to a single token (Gu and Dao 2024). The authors argue that Mamba alone cannot effectively model fine-grained spatio-temporal differences. SDS makes the scan parameters at each step responsive to local changes between frames, not just to the content of a single token.

Temporal Interpolation Scan (TIS): The features of both frames are pooled along the width dimension and interleaved one token at a time, alternating between the reference and target frame, to produce a single sequence, X_W. Let $F_T, F_R \in \mathbb{R}^{C\times H\times W}$ be the target and reference frame features, let $L = H \times W$, and let SP be spatial pooling along the width with a 1×2 kernel. After pooling, each frame is reshaped into a sequence of $L/2$ tokens, where f^i is the i-th token:

$$\bar{F}_T = \mathrm{SP}(F_T) = \left[f_T^1, f_T^2, \ldots, f_T^{L/2}\right] \in \mathbb{R}^{L/2\times C}, \tag{20}$$

$$\bar{F}_R = \mathrm{SP}(F_R) = \left[f_R^1, f_R^2, \ldots, f_R^{L/2}\right] \in \mathbb{R}^{L/2\times C}. \tag{21}$$

The two separate sequences are then interleaved, to give the width-interpolated sequence:

$$X_W = \mathrm{Interpolation}(\bar{F}_T, \bar{F}_R) = \left[f_R^1, f_T^1, f_R^2, f_T^2, \ldots, f_R^{L/2}, f_T^{L/2}\right] \in \mathbb{R}^{L\times C}. \tag{22}$$

The same steps are repeated along the height dimension, with a 2×1 pooling kernel, to give X_H. In both sequences, each target token sits next to the reference token from the same pooled position, so X_W captures displacement information between the two frames along the width, and X_H does the same along the height.

Each sequence is scanned in two directions, from top left to bottom right and the reverse, using the expanding scan method of VMamba (Liu et al. 2024). The two scan outputs are merged by addition and reshaped into one feature map. The feature maps from X_W and X_H are merged also via addition to give the final output features. This scanning method was selected as multi-directional scans help aggregate representations of small targets (M. Zhang et al. 2025b).

4.4 Fusion, detection, and model variants

The displacement outputs and the target feature are stacked and combined by an elementwise maximum over time. The FPN adds this pooled contribution to the target features and combines spatial scales. The YOLOX head returns box coordinates, objectness scores, and class scores on three grids. A decoding step converts the grid outputs into boxes, and NMS removes sufficiently overlapping predictions.

Two simpler models provide comparisons. Base is a single-frame detector that uses the YOLOX-S backbone and PAFPN. Base+FISTA keeps the YOLOX front end, replaces later stages with FISTA, and uses a plain FPN. MOCID has its own front end and a raw-frame pooling path, so moving between these models changes more than the component in the model's name.

4.5 How the FISTA and DAM operations work

The operations above can be understood through small examples, which also explain the blocks used by the variants in Section 5.

Why transform a feature map. Learned global filtering is also used in image architectures such as GFNet (Rao et al. 2021). A spatial Fourier transform expresses a feature map as a combination of spatial frequencies. Slowly varying patterns occupy different frequencies from abrupt changes. Multiplying the transformed values by learned weights changes the mixture before the inverse transform returns to image coordinates. The operation gives each output

access to information spread across the spatial map, without explicitly comparing every pair of pixels.

The local implementation uses `rfft2`, which stores the non-redundant half of the spectrum for real-valued input. Its learned weights are represented by real and imaginary tensors. The inverse transform returns a real-valued feature map. The filters are allocated for particular feature-map sizes, so changing input resolution requires more than changing a data-loader flag.

The temporal transform acts along T, not along the channel dimension. Its local implementation takes a norm across the transformed-back sequence to produce a temporal amplitude weighting. A norm is not the same as dividing a vector by its norm. The paper's terminology left room for interpretation here, so the report identifies the operation actually implemented rather than treating the wording alone as proof of equivalence.

A small temporal-interleaving example. Suppose one pooled reference row is (r_1, r_2) and the corresponding target row is (t_1, t_2). The width-interpolation path constructs (r_1, t_1, r_2, t_2). The scan now encounters information from the two frames in alternating positions. The height path applies an analogous arrangement along the other spatial axis.

The preceding average pooling is part of this construction. A width of four becomes two before interleaving restores a sequence of width four. Thus interleaving does not simply double the length of the unpooled image. It also changes which spatial locations are represented by each token. Parameter maps supplied to the scan must correspond to that ordering, which is why the scan's parameter maps are built in the same order.

What the difference convolution measures. An ordinary convolution forms a weighted sum of nearby values. The local central-difference convolution subtracts a centre-based term from that sum. For one input channel, its operation can be written as

$$y(p) = \sum_{q \in \mathcal{N}} w_q x(p+q) - \theta x(p) \sum_{q \in \mathcal{N}} w_q. \tag{23}$$

Here p is the current position, q indexes the neighbourhood, and θ sets the strength of the subtraction. The implemented neighbourhood spans time and two spatial dimensions. Multiple input channels contribute additional sums.

The same family of difference operators is used in other detection work (Z. Su et al. 2025). A one-dimensional illustration makes the subtraction easier to see. Give a three-value neighbourhood equal weights of $1/3$. For values $(2, 5, 2)$, the ordinary response is 3. With $\theta = 0.7$, the corrected response is $3 - 0.7(5) = -0.5$. For a flat neighbourhood $(2, 2, 2)$, it is $2 - 0.7(2) = 0.6$. A negative response can therefore carry useful contrast information. It is not automatically an error or a target probability.

At $\theta = 1$, a constant interior neighbourhood gives zero. At the implemented $\theta = 0.7$, the operation retains some ordinary intensity response. Padding and learned weights also affect the result. The subtraction therefore introduces sensitivity to differences, but does not by itself identify motion or guarantee that clutter will be rejected.

The SDS module applies this operation to the stacked reference and target features, averages its output over the two time positions, and projects it into three parameter maps. At the finest scale, with $C = 128$ and $N = 32$, the projection produces $128 + 32 + 32 = 192$ channels. They are split into the step-parameter map and the two state input/output maps. This explains why the parameter generator is distinct from the scan that consumes its output.

Local processing and channel attention. The lightweight CAB first reduces the channel count with a 1×1 convolution. A depthwise 3×3 convolution processes each resulting channel separately. A GELU activation introduces nonlinearity, a second 1×1 convolution restores the channel count, and a dilated depthwise convolution covers a wider neighbourhood. These operations retain the feature-map height and width.

Channel attention then averages each channel across the image, giving one descriptor per channel. Two small learned transformations and a sigmoid turn those descriptors into channel weights between zero and one. Each weight multiplies every spatial location of its channel. This mechanism can change the relative importance of feature channels, but it does not produce a separate attention weight at every pixel.

The branch is added to its input through a residual connection. Multiplication by a sigmoid inside the CAB does not imply that the full residual block can only reduce amplitudes. The processed features can have either sign, and the surrounding learned projections and residual scales also change the output.

Why two normalisation layers can behave differently. Per-pixel LayerNorm compares the channels at one spatial location. GroupNorm with one group compares all channels and spatial locations of one example. Both produce normalised features, but the values used to calculate their mean and variance are different.

For a small example, take two pixels whose channel vectors are $(1, 3)$ and $(101, 103)$. Ignoring the small numerical stabiliser, per-pixel LayerNorm transforms both into $(-1, 1)$. One-group GroupNorm instead uses mean 52 and variance 2501 across all four values. Its normalised values are approximately $(-1.020, -0.980, 0.980, 1.020)$. The strong spatial brightness difference remains visible in that second result.

Neither calculation is universally better. They present different signals to the next layer. This is why changing the first GroupNorm to per-pixel LayerNorm should be tested as its own architectural choice rather than described as a cosmetic correction. The recorded MambaIR run retained GroupNorm first and used channel LayerNorm after the scan and before the CAB.

4.6 Two-stage training

MOCID is trained in two stages, as in the published work (M. Zhang et al. 2025b). In stage 1, the backbone, temporal pool, feature pyramid and head learn together while the DAM is bypassed, so raw backbone features enter the pool directly. In stage 2, the backbone is frozen and the DAM is switched on and trained, so that the module learns on top of features that no longer change. In the code used here, stage 2 also continues training the pool, feature pyramid and head. The change from stage 1 to stage 2 is called the handover.

Each training stage follows a recipe, which is the complete set of learning rate, input scaling, decay and scheduling settings. The team's first complete recipe is called R0. It was kept unchanged as the common reference, so that all displacement modules were compared under identical conditions. Stage 1 and stage 2 each run for 100 epochs.

4.7 Summary

MOCID provides a practical location for comparing displacement modules, since the DAM can be replaced while the rest of the detector stays fixed. Details such as the front end, temporal pool, parameter generation and module width are not fully specified by the paper, so the local model is a reconstruction and not an exact copy. Section 5 now turns to what happened when this detector was first trained.

5 Rebuilding MOCID and Developing the Displacement Modules

5.1 Starting point and investigation strategy

Section 4 identified the DAM as the place to test new building blocks, but such a test is only meaningful if the detector around it works. At the start of the project there was no working implementation capable of reproducing the published results. The inherited MOCID prototype trained unstably and scored far below its reported accuracy, so the team first set out to find out why.

The investigation moved from checking the complete model to testing smaller explanations for its behaviour. Parameter counts checked model size. A plain YOLOX-S baseline tested whether the training problem required MOCID's new modules at all. Per-video evaluation identified where accuracy was lost. Short ablations tested individual training settings, and a direct comparison with the DAM on and off tested the transition between training stages. This sequence separated three distinct faults, and it is summarised in Table 5.

Table 5: The three faults found in the inherited implementation, how each was identified, and its correction. Together they turned an unstable prototype into a dependable reference for comparing displacement modules.

Fault	Evidence	Correction
Weight decay applied to BatchNorm scales and biases	Plain Base also failed, and short tests rose from 28.94% to 73.35% once these parameters were excluded	Exclude them from decay in new training recipes
Stale BatchNorm statistics in the evaluated model	One checkpoint rose from 28.87% to 75.18% with no change to its weights	Recalibrate the statistics before every evaluation
Incorrect DAM handover into stage 2	Switching on the untrained DAM dropped accuracy from 89.52% to 67.44%	Start each DAM output from its own reference feature

Claude Code assisted the team with implementing and debugging the three DAM variants and with analysing training logs. The team reviewed this code and verified it through the checks described in this section, and all training runs and measurements were carried out by the team.

5.2 Structural checks and the first complete run

The Base model has approximately 8.94 million parameters, matching the published total. The inherited MOCID has approximately 9.49 million parameters outside the DAM and 12.52 million in total, against published totals of 9.45 and 13.05 million. The local DAM is therefore smaller than the module implied by the paper. Widening the network that generates its scan parameters would close this gap, but different internal designs can share one parameter count, so doing so would not establish the unpublished design. The team therefore kept the existing width as the common reference.

The first complete stage-1 run, R0, reached 88.29% VOC AP50 at epoch 4, fell to 0.68% at epoch 20, and recovered to 88.65% by epoch 100, as shown in Figure 12. A detector that loses and regains nearly all of its accuracy is not a sound basis for comparison, and this curve set the direction of the investigation. The initial code also tracked its best checkpoint only after epoch 40, which could discard a useful early result, so later configurations track the best result from the start.

Stage 2 also exposed a practical dependency. Without the compiled selective-scan kernel, the Python fallback was too slow for a full run, and building a compatible CUDA kernel made stage-2

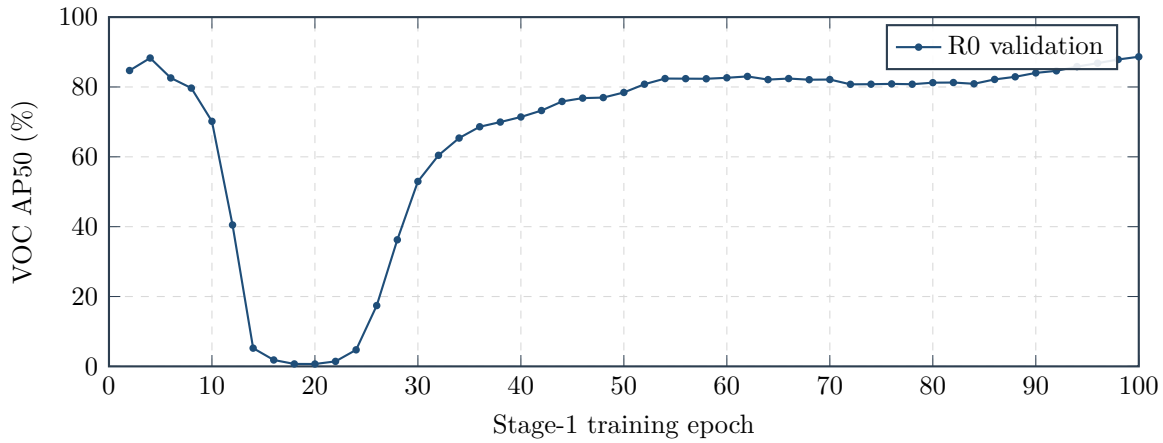


Figure 12: The R0 stage-1 validation curve on 4,767 clips, as logged before BatchNorm recalibration was introduced. The collapse around epoch 20 prompted the investigation in this section, which later showed that part of it came from the evaluation and not from the learned weights.

training feasible.

5.3 Separating training faults from evaluation faults

The plain Base detector also lost accuracy under the inherited recipe, finishing at 75.93% VOC AP50. Since Base contains neither FISTA nor the DAM, those modules were not required for the failure. The fault had to lie in the training recipe or the evaluation.

Weight decay. Weight decay pulls parameters towards zero during training. That is appropriate for convolution weights, but BatchNorm scales control feature amplitude, and the objectness bias sets the head's initial preference for background. In short Base experiments the control reached 28.94% AP50 at epoch 14. Excluding BatchNorm and bias parameters from decay gave 73.35%, and adding ImageNet input normalisation gave 85.80%. Inspection of a collapsed checkpoint agreed with this mechanism, showing a BatchNorm scale that had fallen from 1.000 to 0.231 and an objectness bias that had moved from about -4.6 to $+0.61$. Section 5.4 explains why these changes matter. An early explanation blaming the learning rate was withdrawn, because a short run at the original rate of 0.01 reached 89.93% AP50 by epoch 10 once the decay scope was corrected.

BatchNorm recalibration. The second fault lay in evaluation. The model being scored is an exponential moving average (EMA) of the training weights, and BatchNorm layers also store running statistics of the features they have seen. The EMA averages weights and statistics separately, so when weights move quickly the stored statistics stop matching the averaged weights, and the scored model behaves poorly even though training is healthy. BatchNorm recalibration corrects this by passing training images through the evaluated model and recomputing the stored statistics, leaving every weight unchanged. One checkpoint improved from 28.87% to 75.18% AP50 with this step alone, and R0's final stage-1 checkpoint moved from 88.65% to 89.55%. Much of the collapse in Figure 12 was therefore a fault in measurement, not a loss of learned ability. The pipeline now recalibrates from 300 training batches before every evaluation. In stage 2 this covers only the layers after the frozen backbone.

5.4 Why these faults arise

The faults above follow from how the loss, the optimiser and BatchNorm interact.

What the detector learns from the loss. The training loss measures disagreement between predictions and labelled objects. It is not AP50. The inspected implementation combines box overlap loss, objectness loss, and class loss as

$$\mathcal{L} = \frac{5\mathcal{L}_{\text{box}} + \mathcal{L}_{\text{obj}} + \mathcal{L}_{\text{cls}}}{\max(n_{\text{fg}}, 1)}. \tag{24}$$

Here n_{fg} is the number of grid predictions assigned to labelled objects during training. Objectness loss is applied across the candidate grid locations. Box and class losses are applied to assigned foreground predictions. The default box term is $1 - \text{IoU}^2$, while objectness and class terms use binary cross-entropy on logits.

Backpropagation calculates how changing a trainable parameter would change this loss. The optimiser uses those gradients to update the parameters. Assignment, differentiable loss, and the AP evaluator serve different purposes. A falling training loss therefore does not guarantee a rising validation AP50, especially when the data distribution or evaluation statistics differ.

Why the parameter group matters. Weight decay adds a pull towards zero during optimisation. That pull can be appropriate for convolution weights, but a normalisation scale initialised to one serves a different function. BatchNorm produces a standardised value and then applies a learned scale γ and shift β. If γ shrinks, the signal on that channel can shrink even when the convolution still extracts useful information.

An objectness bias is another example. A bias of approximately -4.595 has sigmoid value 0.01 when all other logit contributions are zero. A bias of zero gives 0.5 under that same condition. Weight decay changes this prior tendency, but actual predictions depend on the complete feature-dependent logit. Figure 13 illustrates the isolated bias relationship.

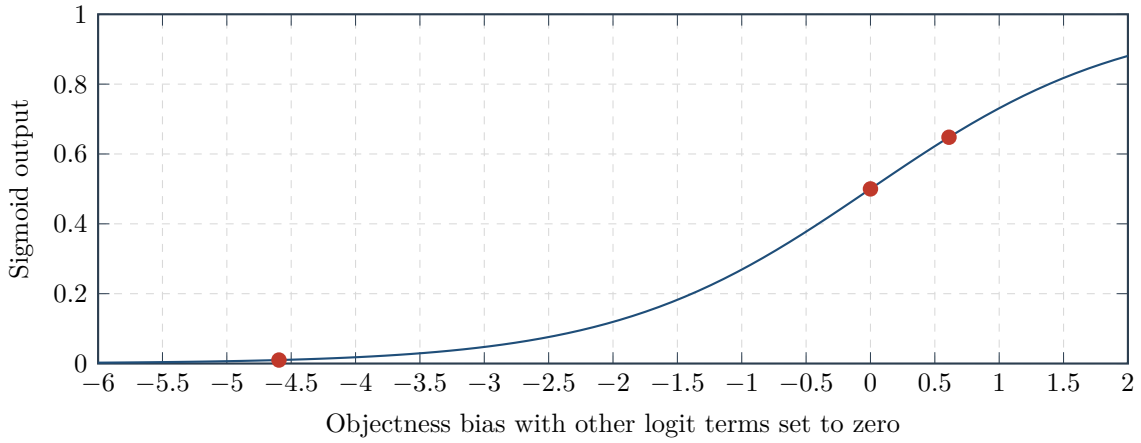


Figure 13: The mathematical relationship between a bias and sigmoid output when other logit terms are zero. Marked values illustrate the initial bias, zero, and the inspected bias. The curve is not a measured confidence distribution.

Momentum accumulates repeated gradient contributions. For a constant scalar contribution, the geometric accumulation factor approaches $1/(1-m)$, which is approximately 15.9 at momentum $m = 0.937$. This is a useful intuition for why a repeated decay term can matter. It is not an exact replacement of the optimiser by a larger decay coefficient, because parameters, gradients, learning rate, and Nesterov updates vary during training.

Why evaluation can fail while loss falls. BatchNorm uses statistics estimated from training batches, while evaluation typically uses stored means and variances. EMA forms a moving average

of model parameters and buffers. Averaging weights and averaging normalisation statistics separately does not guarantee that the two averages describe the same feature distribution.

Recalibration feeds training data through the evaluated model to refresh its stored statistics. It uses no validation labels to update the model. In the current code, only the eligible BatchNorm layers are updated, while other components remain in their intended evaluation state. Recalibration can therefore change validation accuracy without changing the learned weights.

The recorded recovery from 28.87% to 75.18% AP50 shows why a low evaluation score alone was insufficient to diagnose complete training collapse. Correcting the optimiser and correcting the evaluated statistics address different mechanisms. A final experiment should record both policies explicitly and save the exact weights and statistics used to produce the reported score.

5.5 Establishing a stronger baseline

With the decay scope corrected, Base reached 88.83% COCO AP50 at epoch 51 and held 87.99% at epoch 100, with a stable curve throughout late training. This run used the training procedure of SSTNet (S. Chen et al. 2024), which the MOCID authors follow, including ImageNet normalisation and sampling one fifth of the training images per epoch. Section 7 gives its settings.

This single-frame result exceeds the 83.59% reported for Base in the MOCID paper (M. Zhang et al. 2025b). Tests of confidence threshold, checkpoint selection and batch size did not reproduce the lower figure as a converged result, although the training curves pass near it early in training. A batch-8 run reached 86.77%, and the checkpoint selected by lowest validation loss in the batch-4 experiment scored 88.51%. The local curves pass near the paper's reported scores earlier in training, which is compatible with differences in training duration, initialisation, implementation or checkpoint selection. The cause cannot be established without the authors' code. The practical lesson is that a new architecture must be judged against a well-trained baseline, because a weak baseline inflates the apparent benefit of any addition.

5.6 Correcting the transition into stage 2

The final fault appeared at the moment the DAM was switched on. A new module should start as a pass-through, so that switching it on leaves the detector's output exactly as stage 1 trained it, and training can then improve from that point. The inherited DAM instead returned a copy of the target feature for every reference frame. All reference information vanished from the temporal pool before any learning took place, and AP50 dropped from 89.52% to 67.44% on one checkpoint.

The correction makes each untrained DAM output equal its own reference feature, which restored 89.52%. A separate R0 diagnostic recorded a larger drop, from 88.65% to 24.90%, under a different pipeline, so these two measurements describe different checkpoints and are not combined.

A single position shows why this matters. Consider one feature channel at one spatial location. Let the target value be 2 and the four reference values be $(1, 5, 3, 4)$. In stage 1, the temporal maximum is 5. Adding the target feature gives a value of 7 entering the corresponding fusion path.

If an untrained DAM returns the target for every reference slot, the pool receives $(2, 2, 2, 2, 2)$. Its maximum becomes 2, and the fused value becomes 4. A zero-initialised correction branch therefore does not ensure an unchanged detector if the wrong feature is used as its residual. Returning each reference unchanged restores the pool maximum of 5 and the fused value of 7.

The equality property should be tested on features and raw head outputs before the first update.

A relative error can be recorded as

$$\epsilon = \frac{\|Y_{\text{on}} - Y_{\text{off}}\|_2}{\|Y_{\text{off}}\|_2 + \varepsilon}, \tag{25}$$

where ε prevents division by zero. The code uses a tolerance rather than requiring bitwise identity because floating-point operation order can change very small values. A full validation comparison can then confirm that the feature-level property is preserved through box decoding.

The initialisation test should be followed by training checks. A finite first loss is useful but does not prove that every intended branch learns. Record non-zero gradients after the residual projections have updated, check that trainable weights move, and confirm that frozen modules remain unchanged. The current sanitisation of non-finite scan values also needs attention because replacing NaNs before a later finite check can hide a failure. A verification tool now confirms the pass-through directly, by comparing features and head outputs with the DAM off and on before every new experiment.

A stage-2 run from the R0 checkpoint with this correction reached 90.12% VOC AP50 and finished at 89.20%. This run, called the corrected original DAM or R0-DAM-fixed, became the reference for all three new designs. It kept the R0 recipe, including decay on all parameters, so that every variant could start from one shared stage-1 checkpoint.

5.7 Developing the MambaIR-inspired DAM

The first design asked whether restoring local image detail around the scan would improve detection. MambaIR showed that a selective scan performs better when combined with local convolution, channel attention and learned residual scales (Guo et al. 2025). MambaIR's own scan processes a single image, so replacing the DAM's scan with it would have removed the comparison between frames. The temporal scan was therefore retained and the MambaIR residual structure was built around it.

The design adds a LayerNorm after the scan, learned scales on the residual connections, and a second residual branch containing a lightweight channel attention block (CAB). In Figure 11 this is the addition after the residual step. Section 4.5 explains the CAB and the normalisation layers. The additions raised the full model from 12.52 to 12.91 million parameters. Their final layers start at zero, so the module still begins as a pass-through. Some internal parameters receive zero gradients at the first step because their output sits behind a zero-initialised projection, and they become active as those projections move away from zero. The CAB and channel attention code was confirmed to match the published MambaIR code numerically, and the verification tool confirmed the handover and stable training on real clips.

For reference feature F_R, the implemented block is

$$U = s_1 \odot F_R + P\big(\text{LN}(\text{TIDS}(X_T, X_R)) \odot Z_T\big), \tag{26}$$

$$V = s_2 \odot U + \text{CAB}\big(\text{LN}(U)\big), \tag{27}$$

$$F_R^{\text{out}} = Q(V), \tag{28}$$

where X_T and X_R are the locally processed projected features, Z_T is the target-derived SiLU gate, P and Q are output projections, and s_1 and s_2 are learned channel scales. The first normalisation remained the original whole-map GroupNorm, so the block draws on MambaIR's residual structure but is not identical to its published RSSB. Figure 14 shows the two residual stages.

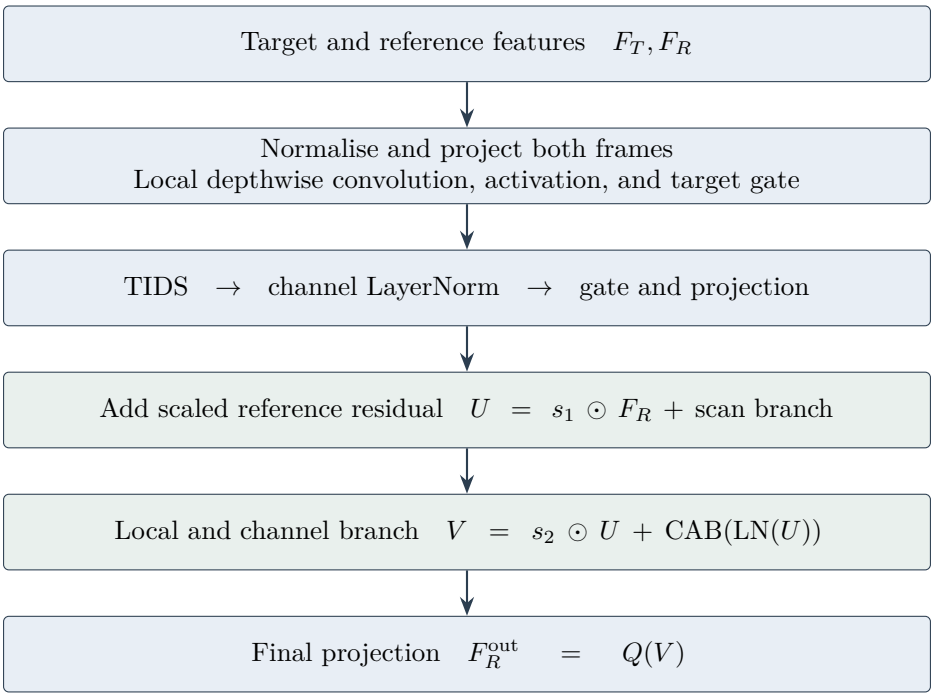


Figure 14: The MambaIR-inspired DAM . TIDS retains the temporal scan. The scan projection and final CAB convolution start at zero, residual scales start at one, and the final projection starts as the identity. The complete block therefore initially returns the reference feature.

Table 6: Stage-2 validation results of the MambaIR-inspired DAM on DAUB (%). Both runs start from the R0 stage-1 checkpoint and are evaluated every second epoch on 4,767 clips. The better value in each row is in bold.

Metric	MambaIR-inspired DAM	Corrected original DAM
Best AP50 (epoch)	89.53 (32)	**90.12** (52)
Final AP50 (epoch 100)	88.53	**89.20**
Mean AP50, epochs 2–100	88.87	**89.49**
Mean AP50, epochs 80–100	88.72	**89.37**
Best F1 (epoch)	**90.96** (78)	90.50 (12)
Final F1 (epoch 100)	**90.68**	89.66

5.8 MambaIR experiments and results

The module was trained in stage 2 only, from the R0 stage-1 checkpoint and under the R0 recipe. Table 6 compares it with the corrected original DAM.

The module scored 0.59 percentage points below the reference at its best checkpoint and 0.67 at the final epoch, and it was lower on every AP50 summary, so the ranking is not an artefact of a single epoch. Its F1 was higher, because F1 reflects one operating point while AP50 summarises the whole ranking of detections. The added local processing therefore shifted the balance between precision and recall without improving the ranking overall. This design did not achieve its aim under the R0 recipe.

5.9 Developing the CFMW-inspired DAM

The second design asked whether the DAM would benefit from a view of the whole scene. CFMW is an object detector for paired visible and infrared images in adverse weather (H. Li, Q. Hu, et al. 2025). It contains two components. A Perturbation-Adaptive Diffusion Model (PADM) reconstructs features degraded by weather, and a Cross-modality Fusion Mamba (CFM) exchanges information between the two image streams. In the authors' released code, CFM pools each stream to 8×8 tokens, swaps half of the channels between them, and places both in one sequence for a Mamba layer. Because the two streams share one sequence, the scan state carries information between them at linear cost.

CFM relates two images of one scene, which mirrors the DAM's pairing of a target frame with a reference frame. TIDS, by contrast, compares only corresponding tokens of the two frames, so each position sees local displacement alone. A CFM branch was therefore added beside TIDS, as shown in Figure 11, to help separate target motion from scene-wide change such as camera motion. Only CFM was adopted, because PADM addresses weather degradation in visible images and has no counterpart in single-modality infrared video.

Three changes were made to the released code. The Mamba layer scans in both directions, so that each frame's tokens receive information from the other frame. The depthwise convolution acts on each frame's own 8×8 grid. The scan's inputs are RMS-normalised and its parameters are excluded from weight decay, because without these measures the state contribution collapsed to near zero under the SGD settings used. The branch increased the DAM from 3.034 to 4.291 million parameters and the full model from 12.52 to 13.78 million, slightly above the 13.05 million reported for MOCID. Table 7 gives the parameter breakdown.

For reference feature F_R, the implemented block is

$$Y = \mathrm{TIDS}(X_T, X_R) + \mathrm{CFM}(X_T, X_R), \tag{29}$$

$$U = F_R + P(Y \odot Z_T), \tag{30}$$

$$F_R^{\mathrm{out}} = Q(U), \tag{31}$$

with both branches receiving the same projected features. Figure 15 shows the block.

Table 7: Parameter count of the DAM blocks with and without the CFM branch (millions).

Scale (channels)	DAM block without CFM	CFM branch	DAM block with CFM
1/8 (128)	0.305	0.077	0.382
1/16 (256)	0.737	0.257	0.993
1/32 (512)	1.993	0.923	2.916
DAM total	3.034	+1.256	4.291

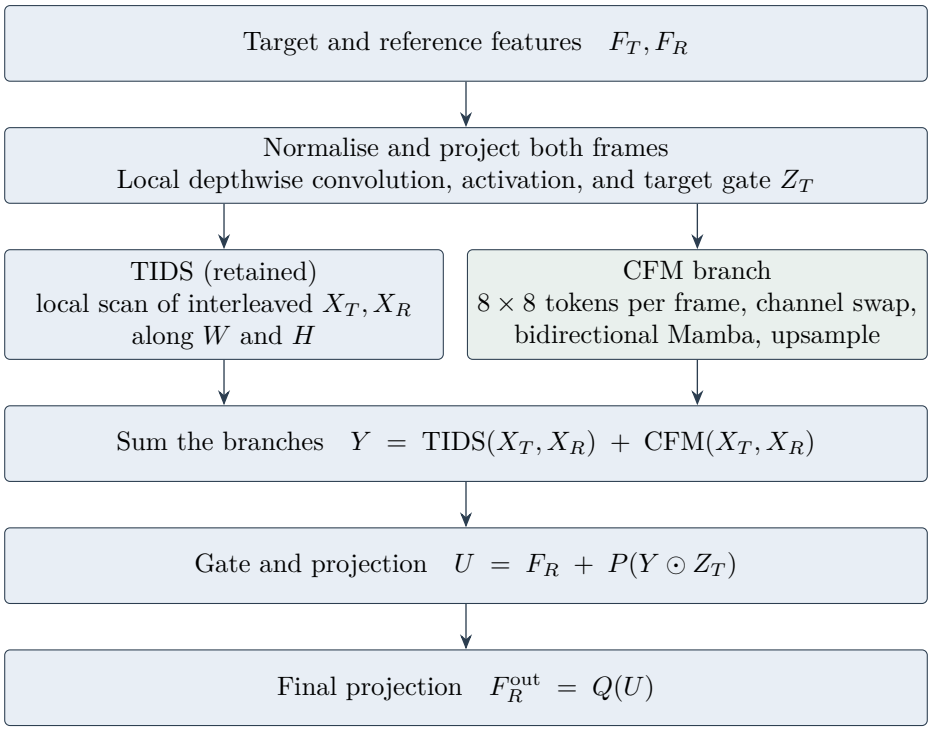


Figure 15: The CFMW-inspired DAM . TIDS is retained from the original DAM, and the CFM branch (shaded) runs beside it on the same projected features. The scan-output projection P starts at zero and the final projection Q starts as the identity. The complete block therefore initially returns the reference feature.

5.10 CFMW experiments and results

The module was trained in stage 2 only, from the R0 stage-1 checkpoint and under the R0 recipe, with its new scan parameters excluded from weight decay. This run also scored every evaluation a second time with the DAM switched off, which tests whether the module contributes to detection at all. Table 8 and Figure 16 give the results.

Table 8: Stage-2 validation results of the CFMW-inspired DAM on DAUB (%). All columns start from the R0 stage-1 checkpoint and use 4,767 clips. The DAM-off column scores the CFMW model at each evaluation with its DAM bypassed. The best value in each row is in bold.

Metric	CFMW-inspired DAM	CFMW model, DAM off	Corrected original DAM
Best AP50 (epoch)	90.27 (70)	**91.99** (32)	90.12 (52)
Final AP50 (epoch 100)	89.44	**89.56**	89.20
Mean AP50, epochs 2–100	89.64	**91.09**	89.49
Mean AP50, epochs 80–100	89.49	**90.09**	89.37
Final F1 (epoch 100)	**90.37**	89.25	89.66

The module reached 90.27% AP50, which is 0.15 percentage points above the reference at the best checkpoint and 0.24 at the final epoch. Both differences are smaller than the variation of about ±0.3 percentage points between consecutive evaluations of a single run, so one run cannot separate the two modules.

The DAM-off comparison is the more important result. With the DAM bypassed, the same model scored higher at 48 of the 50 evaluations, peaking at 91.99% at epoch 32 and averaging 91.09%, against 89.64% with the DAM on. The added global context therefore did not make the DAM contribute to AP50 under this recipe. The rise above the stage-1 score appears to

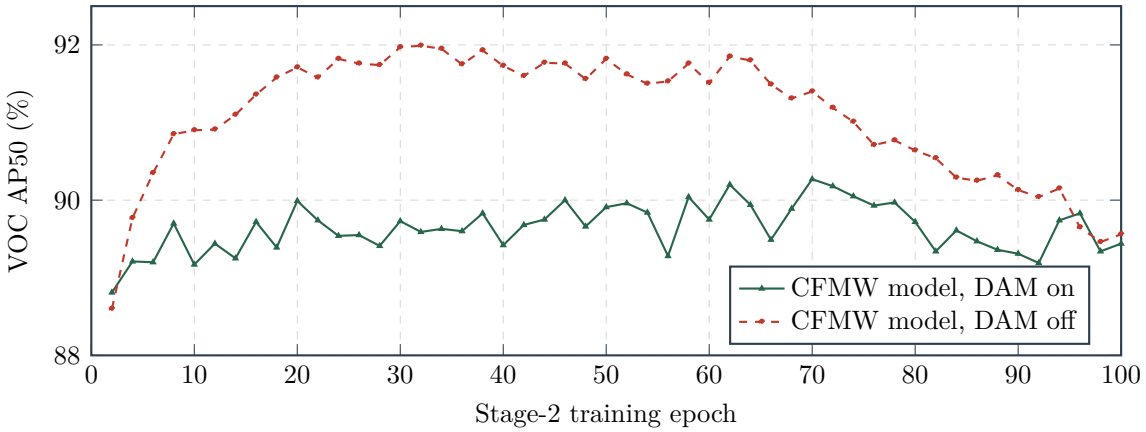


Figure 16: The same CFMW-inspired model scored with its DAM on and off at every evaluation on 4,767 clips. Bypassing the DAM scored higher at 48 of 50 evaluations, so the module reduced AP50 for most of training. The DAM-off curve falls after epoch 60, which suggests that the feature pyramid and head begin to overfit.

come mainly from continued training of the feature pyramid and head. The DAM-off score then declines late in training, which suggests that those layers begin to overfit. The head and pyramid were trained with the DAM on, so the DAM-off path was never the one being optimised, which makes its higher score more striking.

5.11 Developing the MambaTree-inspired DAM

The third design asked whether the scan's reading order could follow the image. The DAM's raster scan flattens each map into a line, which separates vertically adjacent pixels by W positions regardless of content. Xiao et al. (2024) argue that such fixed paths ignore spatial structure. MambaTree instead links each pixel to its most similar neighbours through a minimum spanning tree over a four-connected grid, and it passes state along that tree at the same linear complexity as Mamba.

Two integration options were considered. The first replaces only the scan, retaining SDS and the temporal interleaving and building the tree over each interleaved map. The second removes the interleaving and builds a single tree across both frames. The second option would change the temporal handling as well as the scan, so accuracy differences could not be attributed to the scan alone. The first option was therefore implemented, as marked in Figure 11.

The tree scan replaces only the scan within TIDS. Edges between neighbouring pixels are weighted by $\exp(-\cos(\mathbf{x}_i, \mathbf{x}_j))$, where $\cos(\mathbf{x}_i, \mathbf{x}_j)$ is the cosine similarity of their features, so the tree joins the most similar pixels first. State is aggregated from the leaves to the root and back, so a single pass gives each pixel a contribution from every other pixel. The scan parameters come from SDS, so the block follows MambaTree's tree scan without reproducing the complete published method. A LayerNorm is applied to the aggregated state before read-out. Figure 17 shows where the tree scan sits within the block.

5.12 MambaTree experiments and results

The MambaTree-inspired DAM was trained and compared with the corrected original DAM and a dedicated control run, to isolate the effect of the scan. Like the other variants, these runs trained stage 2 only from the R0 stage-1 checkpoint, following the setup in Section 7.2. MOCID's default state size, the number of values the scan stores in memory for each channel, is $N = 32$. At this size, the tree scan increased the training time to 1.43 s per step, amounting to

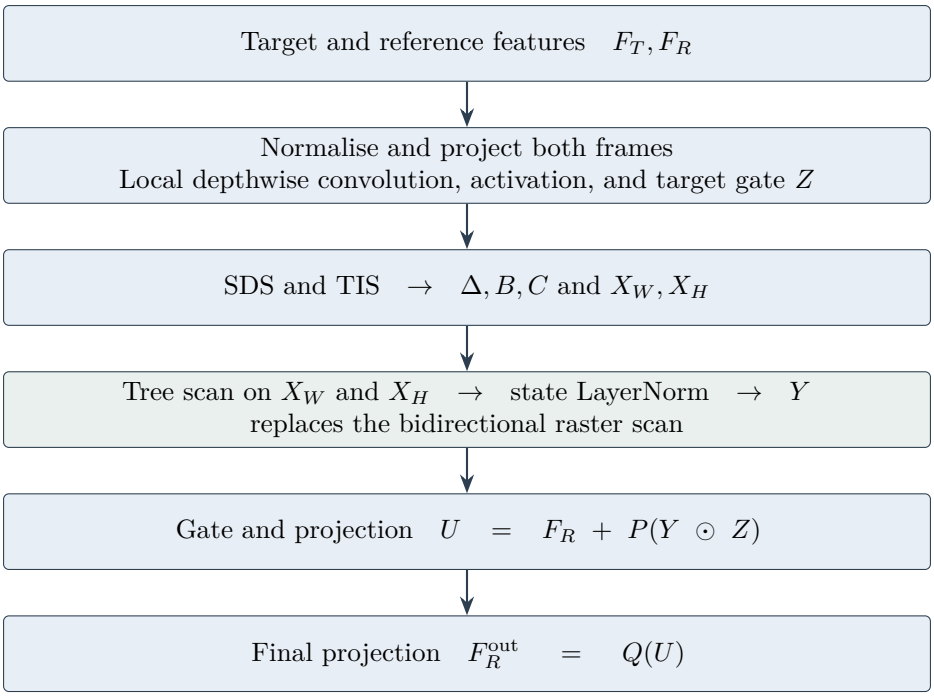


Figure 17: The MambaTree-based DAM . SDS and TIS are retained from the original DAM, and the tree scan (shaded) replaces the bidirectional raster scan. The scan-output projection starts at zero and the final projection starts as the identity. The complete block therefore initially returns the reference feature.

approximately 3.7 days for a completed run. The tree scan was therefore trained with $N = 1$, matching the setup of MambaTree's released implementation (Xiao et al. 2024), which meant that state size became a variable alongside the scan.

Four configurations were compared in Tables 9 and 10, with the corrected original DAM at $N = 32$ as the reference. The tree scan was first trained with $N = 1$, followed by a raster-scan control with $N = 1$ to separate the effect of the state size from the effect of the scan. The tree scan was then trained with $N = 8$ to test a larger state size. This gave no improvement in AP50 (Table 9), so the cost of increasing the state size further, and ultimately of completing a full run at $N = 32$, was not justified.

Table 9: Stage-2 validation AP50 (%) of the MambaTree-inspired DAM on DAUB. All runs start from the R0 stage-1 checkpoint (88.65%) and are evaluated every second epoch. *Best* is the highest score of the run, with its epoch in parentheses. *Final* is the score at epoch 100. Training speed is in steps per second on one NVIDIA L40S. The best value in each row is in bold.

	Tree, N=1	Raster, N=1 (control)	Tree, N=8	Corrected original (raster, N=32)
Best (epoch)	**90.19** (48)	90.12 (52)	90.13 (58)	90.12 (52)
Final (epoch 100)	89.41	**89.57**	88.95	89.20
Mean, epochs 2–100	**89.70**	89.56	89.54	89.49
Mean, epochs 2–50	**89.72**	89.45	89.68	89.49
Mean, epochs 52–100	**89.68**	**89.68**	89.40	89.49
Mean, epochs 80–100	89.52	**89.63**	89.17	89.37
DAM parameters (M)	2.595	2.593	2.603	3.034
Training speed (steps/s)	6.7	10.0	3.0	9.6

At $N = 1$, the tree scan improved on the corrected original DAM by 0.07 percentage points at

Table 10: Stage-2 validation F1 (%) for the MambaTree runs in Table 9. F1 is the highest value over all confidence thresholds at an IoU of at least 0.5. *Best*, *Final* and the *Means* are defined as in Table 9. The best value in each row is in bold.

	Tree, N=1	Raster, N=1 (control)	Tree, N=8	Corrected original (raster, N=32)
Best (epoch)	90.59 (22)	90.74 (98)	**90.98** (30)	90.50 (12)
Final (epoch 100)	90.10	**90.70**	89.99	89.66
Mean, epochs 2–100	90.23	**90.38**	90.31	90.07
Mean, epochs 2–50	90.31	90.26	**90.49**	90.29
Mean, epochs 52–100	90.15	**90.51**	90.12	89.86
Mean, epochs 80–100	90.07	**90.63**	90.05	89.72

best, 0.21 at the final epoch and 0.21 on average, and it was ahead at 38 of the 50 matching evaluations. Taken alone, this would suggest that the tree scan helps.

The control shows that most of this gain did not come from the tree. The tree scan was ahead over the first half of training (89.72% against 89.45% mean AP50), but the two scans tied over the second half at 89.68%, and the control finished higher (89.57% against 89.41%). The control also improved on the corrected original DAM late in training, which attributes the gain mainly to the change from $N = 32$ to $N = 1$. Because the SDS hidden width is $\max(2N, C/8)$, this change also narrows SDS at the two finer scales, so the gain reflects the smaller state and narrower SDS together. On this evidence the tree scan learns faster early but does not reach a higher or more stable accuracy than the raster scan, while costing about 50% more time per training step.

With $N = 8$, the tree scan reached the highest F1 of any run (90.98% at epoch 30), but it declined most after epoch 60 and finished lowest at 88.95% AP50, only 0.30 percentage points above the stage-1 starting point. A larger state therefore did not raise the peak. It changed how quickly the peak was reached and how much accuracy was lost afterwards. All runs also overfitted. Between epochs 50 and 92 the training loss fell by about 18% in every run while validation AP50 stayed flat or declined.

5.13 Summary

Three faults were found and corrected, turning an unstable prototype into a working detector. Three new displacement modules then each tested a different idea from the literature, namely local detail, global context and content-aware reading order. Each was implemented, verified to begin as a pass-through, and trained from one shared checkpoint. Their best scores fell within 0.74 percentage points of one another, and none clearly beat the corrected original DAM. The DAM-off measurement suggests why, since the detector scored higher without any DAM at all. Section 6 describes how these modules fit into the shared pipeline, and Section 7 assesses them side by side.

6 Integration into the MOCID Software Pipeline

6.1 Input and output contract

A replacement module must fit a fixed interface so that nothing else in the detector changes. The detector accepts a batch of clips with shape $(B, T, 3, H, W)$, in which the last frame is the target and the preceding frames provide context. All recorded runs use $T = 5$ and $H = W = 512$. The source imagery is infrared, and the loader reads it into three channels to match the network interface.

During training, each example has a variable-length list of target-frame labels, and the forward method returns a single training loss. During evaluation, it returns three raw prediction grids, which are decoded into boxes and filtered with non-maximum suppression. Table 11 and Appendix D trace these dimensions through the model.

Table 11: Tensor dimensions for the default five-frame, 512×512 configuration.

Tensor	Shape
Input clip	$(B, 5, 3, 512, 512)$
Stride-8 backbone volume	$(B, 5, 128, 64, 64)$
Stride-16 backbone volume	$(B, 5, 256, 32, 32)$
Stride-32 backbone volume	$(B, 5, 512, 16, 16)$
Pooled stride-8 feature	$(B, 128, 64, 64)$
Stride-8 raw head output, one class	$(B, 6, 64, 64)$
All decoded grid candidates	$(B, 5376, 6)$

6.2 Module replacement

The DAM wrapper applies one block per feature scale, sharing that block across the four reference frames. A replacement must return one feature map per reference, at the same spatial size and channel count. Under this rule the backbone, pool, feature pyramid and head stayed identical across all experiments, and the MambaIR, CFMW and MambaTree modules each plugged into the same position. The handover check in Section 5.6 guards this boundary, confirming that each new module begins as a pass-through.

6.3 Configuration

Named configuration profiles select each experiment from the command line, so that a run can be repeated exactly and no training code changes between variants. Table 12 lists them. Each checkpoint must be loaded with the architecture and input settings used to train it.

Table 12: Configuration profiles used to select each experiment.

Profile or setting	Purpose	Code branch
`r0`	The common R0 recipe, with its learning rate, input scaling and decay settings	`main`
`wdfix`, `freeze`	Add-ons that exclude exempt parameters from decay, and limit stage-2 learning to the DAM	`main`
`ln`, `dtbias`, `noout`, `cabstd`	MambaIR options for per-pixel LayerNorm, a learned delta bias, removal of the final projection, and the standard CAB	`dam-mambair`
`r0-cfm`	R0 with the CFM branch (`DAM_CFM`) and DAM-off scoring (`EVAL_DAM_OFF`). `CFM_GRID` and `CFM_LAYERS` set the token grid and depth	`cfmw-test`
`r0-tree`	R0 with the scan type set to the tree scan. `MOCID_D_STATE` sets the state size	`mamba-tree`

The profiles compose from left to right, so an option can be added without editing the recipe. The results in this report use the `r0`, `r0-cfm` and `r0-tree` profiles. The additional MambaIR options and add-ons are provided for future experiments, and no completed result for them is reported here.

6.4 Summary of integration

The MambaIR, CFMW and MambaTree modules were all integrated into the complete MOCID pipeline, and each was trained and evaluated through the same entry point, data loader, stage-1 checkpoint and evaluation routine. A swappable module, a configurable experiment and a repeatable pipeline were the outcomes required by Objective O4. The repository also contains a registered adapter, `mocid_module.py`, as the route into the industry partner's framework. A complete run inside that benchmark was not carried out, because reconstructing MOCID had to come first and the GPU server was lost late in the year. It is therefore the first recommendation in Section 8.

7 Training and Evaluation

7.1 Dataset and preprocessing

All experiments use DAUB (Hui et al. 2019), a public infrared video dataset of dim, small aircraft targets against ground and air backgrounds. It is a standard benchmark for moving infrared small target detection and was used to evaluate the published MOCID. Its targets are small and low in contrast and its videos contain real target motion, which makes it a practical public stand-in for the problem motivating this project, although it does not model the viewing geometry of a satellite. Following the split used by SSTNet and MOCID (S. Chen et al. 2024; M. Zhang et al. 2025b), ten videos are used for training and seven for validation.

The split files contain 8,982 training frames and 4,795 validation frames, and the two sets share no sequences. The clip loader builds five consecutive frames and rejects windows that cross a gap in frame numbering, giving 8,942 training clips and 4,767 validation clips. A five-frame clip cannot end on any of the first four frames of a video, so the seven validation videos account for the difference of 28 frames. Table 13 lists the counts for each video. The Base experiments evaluated all 4,795 frames with the SSTNet-style loader.

Table 13: Validation frame and clip counts from the local DAUB split.

Sequence	Listed frames	Valid five-frame clips
data6	399	395
data11	745	741
data12	1,500	1,496
data15	751	747
data18	500	496
data20	400	396
data21	500	496
Total	4,795	4,767

The repository loader resizes images directly to 512×512, applies the same random horizontal flip to every frame during training, and scales the pixels according to the selected profile. The R0 profile divides pixel values by 255. The Base experiments instead used bicubic letterboxing, ImageNet channel normalisation, no augmentation, and a random fifth of the training set per epoch.

7.2 Training configurations

The experiments proceed in order. The Base baseline establishes what a well-trained single-frame detector can achieve. Stage 1 then trains MOCID without its DAM, and stage 2 trains each

DAM from the resulting checkpoint. Table 14 sets out the settings for the baseline and for stage 2. Both use stochastic gradient descent (SGD) on DAUB, but the recipes differ. In particular, 100 epochs of one-fifth sampling represent far fewer optimisation steps than 100 epochs over the complete training set.

Table 14: Training settings for the Base baseline and for the stage-2 displacement module runs.

Setting	Base baseline	Stage-2 DAM runs (R0 recipe)
Input	ImageNet normalisation, bicubic letterbox	Divide by 255, direct resize
Training data per epoch	Random fifth of training set	All valid five-frame clips
Augmentation	None	Shared horizontal flip
Batch size	4, with a separate batch-8 trial	4
Precision	FP32	AMP, with scan and loss in FP32
Optimiser	SGD, Nesterov, momentum 0.937	SGD, Nesterov, momentum 0.937
Learning rate	6.25×10^{-4} to 6.25×10^{-6} at batch 4	10^{-3} to 10^{-4}
Warm-up	3 epochs	6 epochs
Weight decay	5×10^{-4}, BN and bias excluded	5×10^{-4} on all parameters
Gradient clipping	None	Norm limit 10
Stage-2 trainable parts	Not applicable	DAM, FPN, and head
Evaluation interval	Every epoch	Every 2 epochs
Scored frames	4,795	4,767
Primary AP convention	COCO 101-point AP50	VOC all-points AP50

All four stage-2 runs use the R0 recipe, so that a single stage-1 checkpoint could be shared and the DAM would be the only change. The MambaIR, CFMW and MambaTree runs recalibrated BatchNorm statistics from 300 training batches before each evaluation. The corrected original DAM run was logged before recalibration was added, but re-scoring its best and final checkpoints moved AP50 by less than 0.1 percentage points, so this difference does not affect the comparison. The CFMW run also excludes its own scan parameters and position embedding from weight decay, following the convention of the Mamba and VMamba implementations.

7.3 Evaluation methodology

A prediction counts as a correct detection when it overlaps an unmatched labelled box with an IoU of at least 0.50. Each detection's score is the product of objectness and class confidence, with a confidence floor of 0.001 and an NMS IoU threshold of 0.65. Detections are then ranked by confidence to form a precision–recall curve.

The main metric is VOC AP50 (Everingham et al. 2010), the area under that curve at the fixed overlap requirement of 0.50,

$$\mathrm{AP50} = \int_0^1 p(r)\,\mathrm{d}r, \tag{32}$$

where $p(r)$ is the precision at recall r, taken from the interpolated precision envelope. The Base experiments report the COCO form, which samples the same curve at 101 recall values. The two forms differ by about 1 percentage point in this project, so they are always labelled. Neither should be confused with COCO AP averaged over IoU thresholds from 0.50 to 0.95. Section 7.4 gives worked examples of these metrics.

F1 is the harmonic mean of precision and recall. The repository reports the highest F1 over its confidence sweep. The Base logs instead contain an SSTNet-style F1 calculated from that evaluator's precision and recall summaries, so the two conventions are not ranked together.

The validation videos serve both for choosing checkpoints and for reporting performance, as in the published work, so the figures describe validation performance and not an independent test. Repeated epochs from one run are correlated measurements, so no statistical significance is claimed for differences between single runs.

7.4 Worked metric examples

Overlap and small targets. The intersection over union of two boxes is

$$\mathrm{IoU} = \frac{\text{intersection area}}{\text{area of box 1} + \text{area of box 2} - \text{intersection area}}. \tag{33}$$

For equal square boxes of side s, displaced horizontally by d pixels with no vertical displacement, the IoU for $0 \le d \le s$ is $(s-d)/(s+d)$. A one-pixel shift gives IoU 0.50 for a three-pixel box, but approximately 0.94 for a 32-pixel box. This is a geometrical sensitivity, even before considering classification confidence.

Figure 18 shows this effect. It assumes continuous box coordinates and axis-aligned equal boxes. Real annotations and predictions may also differ in width and height, so the figure is an illustrative special case.

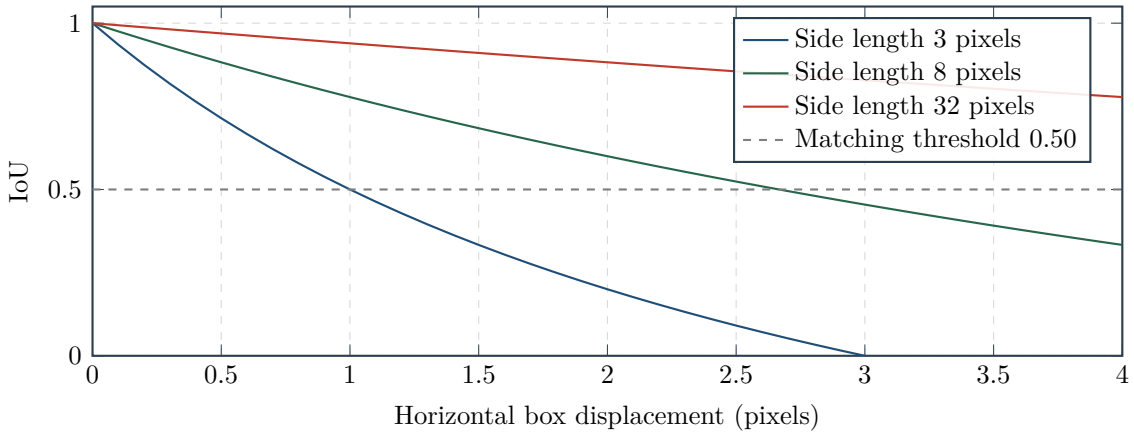


Figure 18: Illustrative sensitivity of equal square boxes to horizontal localisation error. The smaller box reaches the AP50 matching boundary after a one-pixel shift.

Precision, recall, and F1. If a threshold yields 18 correctly matched detections, two unmatched detections, and five missed labelled targets, then precision is $18/20 = 0.90$ and recall is $18/23 \approx 0.783$. Their harmonic mean is

$$F_1 = \frac{2PR}{P+R} = \frac{36}{43} \approx 0.837. \tag{34}$$

Lowering a confidence threshold usually admits more detections. Some recover missed targets and others introduce false positives. A threshold therefore selects one operating point. The best F1 on a validation set is a threshold-tuned result, not automatically a deployable operating point.

Average precision. Consider three labelled objects and four confidence-ranked detections that are, in order, correct, incorrect, correct, and correct. Cumulative precision is $(1, 1/2, 2/3, 3/4)$ and recall is $(1/3, 1/3, 2/3, 1)$. The monotonically interpolated precision envelope is 1 through the first third of recall and 3/4 over the remaining two thirds.

VOC all-points integration gives $1/3 + (2/3)(3/4) = 5/6 \approx 0.8333$. COCO-style sampling at 101 evenly spaced recall values gives $(34 + 67 \times 0.75)/101 \approx 0.8342$ for this simplified example. These

are two calculations of AP at one matching threshold. COCO's broader AP metric averages across several IoU thresholds as well, which is a separate operation.

A high AP50 does not specify an acceptable number of false alarms per minute. That quantity depends on the confidence threshold, frame rate, scene duration, and temporal handling of repeated detections. The final application evaluation should report an agreed false-alarm measure rather than relabel a box-area proxy as an operational alarm rate.

7.5 Baseline results

Table 15 compares the baseline runs, and Figure 19 shows their full trajectories. The corrected batch-4 baseline kept most of its best score through epoch 100. The COCO-pretrained run reached a strong early result and then deteriorated, with training loss falling while validation loss rose, which indicates a widening generalisation gap. That run also changed the input scaling, so its decline cannot be attributed to pretraining alone. Stepping from Base through FISTA to the full model gave 88.83%, 89.52% and an initial 87.07% on the COCO measure, as recorded in Appendix G. The 87.07% was recorded before the handover correction of Section 5.6.

Table 15: Base baseline validation results on all 4,795 frames, using COCO AP50 and SSTNet-style F1 (%). Batch is the number of images in each training step.

Configuration	Best AP50	Epoch	Final AP50	Final F1
Corrected Base, batch 4	88.83	51	87.99	94.08
Corrected Base, batch 8	86.77	48	86.39	93.27
COCO-pretrained, raw-pixel input	89.82	5	72.10	84.43

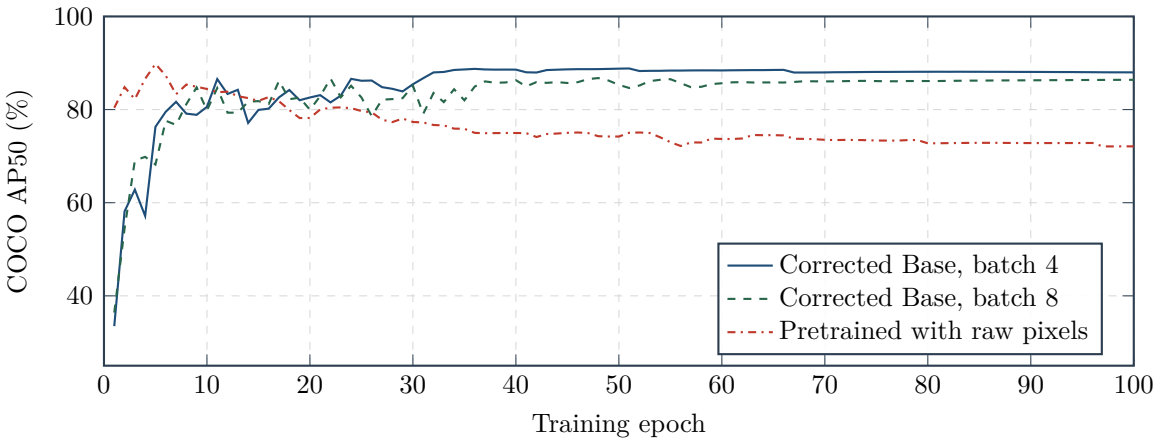


Figure 19: Base baseline validation curves on 4,795 frames, using COCO AP50 without smoothing. The corrected batch-4 run stays stable late in training, whereas the pretrained run declines. The pretrained run also changes input scaling, so its curve does not isolate the effect of pretraining.

7.6 Comparison of the displacement modules

The corrected original DAM and the MambaIR-, CFMW- and MambaTree-inspired DAMs all started from the same R0 stage-1 checkpoint. Table 16 brings their VOC AP50 results together, and Figure 20 shows every evaluation. Each mean is over 50 evaluations of one run, not over independent repetitions. The MambaTree column is the $N = 1$ configuration, the strongest of that family.

The CFMW-inspired DAM reached a best AP50 of 90.27% at epoch 70 and 89.44% at the final epoch, with a final best-threshold F1 of 90.37%. Relative to the original DAM, its best, final

Table 16: Comparison of the four displacement modules on 4,767 validation clips under the R0 recipe. AP50 and F1 are in %. F1 is the best value over the confidence sweep at the final epoch. The best value in each row is in bold.

Metric	Corrected original DAM	MambaIR-inspired DAM	CFMW-inspired DAM	MambaTree-inspired DAM
Approximate full-model parameters (M)	12.52	12.91	13.78	12.09
Best AP50	90.12	89.53	**90.27**	90.19
Epoch of best AP50	52	32	70	48
Final AP50	89.20	88.53	**89.44**	89.41
Mean AP50 over recorded evaluations	89.49	88.87	89.64	**89.70**
Mean AP50 at epochs 80 to 100	89.37	88.72	89.49	**89.52**
Final best-threshold F1	89.66	**90.68**	90.37	90.10

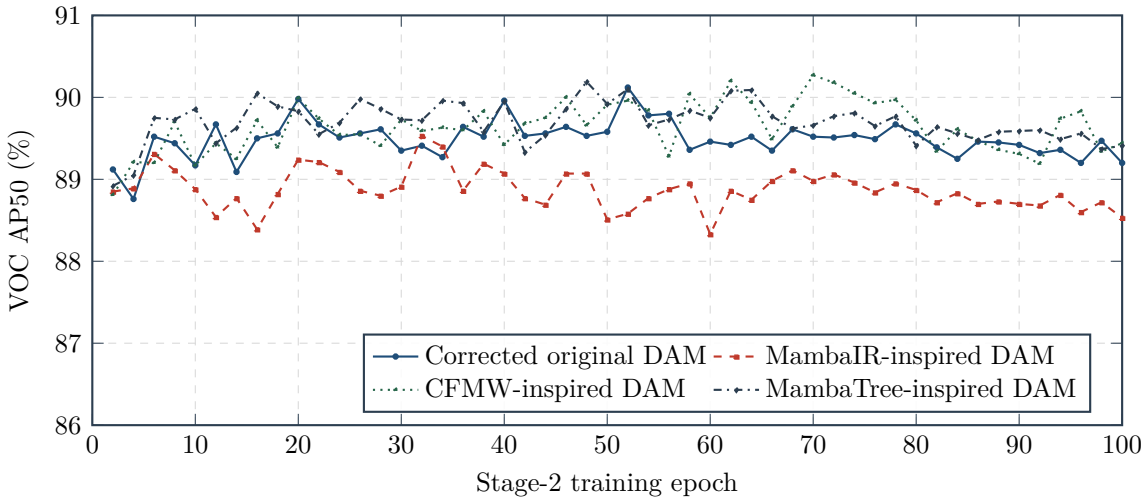


Figure 20: Every evaluation of the four stage-2 runs on 4,767 clips, under the R0 recipe. The vertical axis is restricted to 86–91% to make the small differences visible. The curves overlap throughout, which shows that no module separates clearly from the corrected original DAM once normal run-to-run variation is considered. These are single runs, not seed averages.

and mean AP50 were 0.15, 0.24 and 0.15 percentage points higher. These differences are smaller than the variation of about ±0.3 percentage points between consecutive evaluations of a single run, so one run cannot distinguish the two modules. The margin over the MambaIR-inspired DAM is larger, at 0.74 percentage points at the best checkpoint and 0.91 at the final epoch. This ordering would still need repeated runs to confirm, and the CFMW variant's final F1 was 0.31 percentage points below that of the MambaIR variant. The DAM-off score was higher at 48 of the 50 evaluations, peaking at 91.99% at epoch 32 and averaging 91.09%, against 89.64% with the DAM on.

The MambaTree-inspired DAM, using the tree scan with state size $N = 1$, reached a best AP50 of 90.19% at epoch 48 and a final AP50 of 89.41%, with a final F1 of 90.10%. It was 0.07 percentage points above the corrected original DAM at its best checkpoint and 0.21 above at the final checkpoint, and 0.66 and 0.88 above the MambaIR variant. The matched raster-scan control changed only the state size to $N = 1$, yet it improved on the original DAM by a similar amount at the end of training, with 0.37 percentage points at the final checkpoint and 0.26 over epochs 80 to 100. The advantage is therefore attributed mainly to the smaller state size and the correspondingly narrower SDS, not to the tree scan.

Three observations stand out. First, the CFMW- and MambaTree-inspired modules are effectively level with the corrected original DAM. Their leads of 0.15 and 0.07 percentage points at the best checkpoint are smaller than the variation of about ±0.3 percentage points between consecutive evaluations. Second, the MambaIR-inspired module is the only one consistently below the reference, and it trails the other two by 0.74 and 0.66 percentage points at the best checkpoint. It also has the highest final F1, so the ordering depends on the metric, and a final application comparison should use a preselected operating requirement as well as AP50. Third, every variant's best score lies within about 1.6 percentage points of the 88.65% stage-1 starting point, so no DAM produced a substantial gain.

The size of the gain from adding a DAM is also uncertain. The figure of 88.65% is R0's logged stage-1 score, whereas BatchNorm recalibration raises that checkpoint to 89.55%. Measured from the recalibrated score, the improvement from adding the corrected original DAM falls to 0.57 percentage points. The stage-2 routine recalibrates only the layers after the backbone, so the two numbers are not perfectly matched, but both point to a small effect.

7.7 Per-video behaviour and demonstrations

An overall score can conceal failures concentrated in a few scenes. In the corrected Base checkpoint, recall varies from 69.6% on data21 to 100% on data12, and high recall can coexist with a much lower AP50, as on data15. Figure 21 presents both quantities.

The demonstration tool provides another view, drawing labelled and predicted boxes on every frame of a video so that behaviour can be inspected directly. At a fixed confidence threshold of 0.3 and IoU of at least 0.5, the target-found rate on data15, the harder sequence, was 45.6% for stage 1, 53.5% for the corrected original DAM and 53.1% for the MambaIR variant. On data6 the rates were 95.4%, 97.0% and 96.2%. These results show the displacement modules helping on a difficult sequence, even though the overall AP50 gain is small. They are not AP50 values and describe one fixed threshold. The CFMW and MambaTree variants could not be rendered, for the reason given in Section 7.9.

7.8 Computational efficiency

Table 17 gives the measured cost of each model on one NVIDIA L40S. Inference time covers transfer to the GPU, the forward pass, box decoding and NMS, after warm-up and with GPU synchronisation, and it excludes image loading. The DAM adds about 10 ms per clip and the

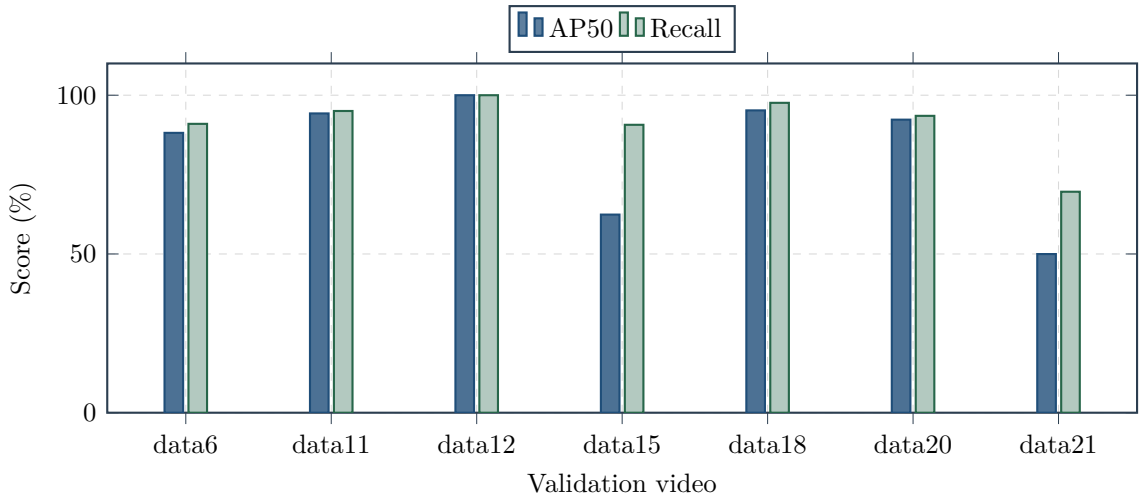


Figure 21: Per-video scores at epoch 51 of the corrected batch-4 Base run. A few scenes account for most of the lost accuracy, especially data15 and data21. Video lengths differ, so the overall AP50 is not the simple average of these seven values.

MambaIR additions a further 4 ms. All measured models run faster than the 25 frames per second of typical video on this GPU, although that GPU is far larger than satellite hardware, so no operational real-time claim follows.

Table 17: Measured computational cost on one NVIDIA L40S. Inference is per five-frame clip. Training cost is relative to the corrected original DAM, from the steps per second in Table 9. Dashes mark values that were not measured.

Model	Inference (ms)	Frames per second	Full-model parameters (M)
Stage 1, no DAM	14.0	71	9.49
Corrected original DAM	24.4	41	12.52
MambaIR-inspired DAM	28.2	35	12.91
CFMW-inspired DAM	–	–	13.78
MambaTree-inspired DAM, $N = 1$	–	–	12.09

The CFMW and MambaTree variants have training measurements in place of inference times. A CFMW stage-2 training step took 78 ms against 76 ms without the branch, an increase of about 3%. This includes the backward pass, so it cannot be compared with the inference times above. The MambaTree scan at $N = 1$ took about 50% longer per step than its raster control.

Timing method matters as much as the result. GPU work runs asynchronously, so a timer stopped without synchronising measures only the launch of the work. Appendix F gives the timing loop recommended for a final comparison, which times the network forward pass alone on real images, after warm-up, with synchronisation either side of the call, and reports median and 95th percentile values on an otherwise idle GPU. Published speeds for MOCID were measured on other hardware and are not compared here.

7.9 Limitations and interpretation

The clearest contribution of this work is a MOCID framework that works. The team began without any implementation able to approach the published results, and it finished with a stable detector, a corrected training and evaluation procedure, and tools that verify each new module before training. Without that foundation, the faults in Section 5 would have distorted any comparison of building blocks.

Against that foundation, each variant gave a clear answer. The MambaIR result is a bounded negative finding, as local processing and channel attention did not improve AP50 under the R0 recipe. The CFMW result shows that adding global context beside the temporal scan leaves AP50 within run-to-run variation, so the lack of benefit from the DAM is not explained by missing global context. The MambaTree result shows that at equal state size the tree scan does not clearly exceed the raster scan, so reading order alone is not the limiting factor. Three different changes to the scan and its surroundings produced the same outcome, which suggests that the limit lies elsewhere.

The DAM-off measurement indicates where. All DAM-on runs peaked between 89.53% and 90.27%, which could suggest that the frozen backbone sets a ceiling. However, the same frozen backbone reached 91.99% with the DAM bypassed, so the backbone features can support a higher score. The evidence instead points to how DAM outputs enter the temporal pool, where a modified reference feature takes the place of a raw one that was already useful. A scan that is initialised as a pass-through and then trained may degrade that raw feature more than it adds to it. This remains a hypothesis, because the DAM-off peak was selected from 50 evaluations and has not been repeated with a separate training run.

Several limitations apply to the wider conclusions. MOCID is not reproduced exactly, and all results remain below the reported 95.93%. Each configuration was trained once, so differences of 0.1 to 0.3 percentage points need repeated runs to confirm. All four stage-2 runs use the R0 recipe, including decay on all parameters, which keeps the comparison fair but means that no variant has been tested under the fully corrected settings. The tree scan could not be trained at the original state size of $N = 32$ because of its cost. The Base and stage-2 results use different frame coverage and AP calculations, so they are not directly subtracted.

The project GPU server became unavailable on 21 October 2026 following a system update, and restoring access was beyond the team's control. Inference timing, per-video results and demonstration hit rates for the CFMW and MambaTree variants were therefore not completed. All AP50 and F1 results for these variants were recorded before the fault and are unaffected. Appendix I lists these measurements and their protocols.

There is also a domain boundary. DAUB contains moving small targets, but it does not quantify the effects of hypersonic speed, viewing geometry from orbit, atmospheric conditions, or specific sensor characteristics. A useful module on DAUB is therefore a candidate for further evaluation and not a validated solution to the full application.

7.10 Summary

The results establish a stable baseline at 88.83% COCO AP50 and a complete MOCID at 90.12% VOC AP50. The MambaIR-inspired DAM reached 89.53%, the CFMW-inspired DAM 90.27%, and the MambaTree-inspired DAM 90.19%. None improved reliably on the corrected original DAM, and the CFMW run scored higher still with its DAM switched off. Section 8 builds these findings into recommendations for future work.

8 Recommendations for Future Work

8.1 Priority recommendations

Redesign how displacement information is combined. The most valuable next step follows directly from the DAM-off result. Rather than replacing each reference feature in the temporal pool, a future design could add DAM output alongside the raw reference features, or use it to weight them, so that a module can only add information. Each such design should be tested against a DAM-off control scored in the same run.

Repeat the comparison across seeds. Each of the MambaIR, CFMW and MambaTree variants, together with the corrected original DAM, should be trained with at least three prespecified seeds, reporting individual results, mean and spread. Epochs from one run cannot substitute for independent repetitions, and this step would settle whether differences of 0.1 to 0.3 percentage points are real.

Evaluate under corrected training settings. All variants should be repeated from a shared stage-1 checkpoint trained with the corrected weight-decay scope and ImageNet normalisation, with recalibration applied throughout. The `wdfix` and `freeze` profiles of Section 6 already provide this, and they make it possible to test whether stage 2 should train the DAM alone, as the MOCID paper describes.

Move to the intended application. Each variant should be run through the industry partner's benchmark using the registered adapter, after confirming that the adapter and standalone model give identical outputs on a shared checkpoint. Evaluation should then extend to IRDST (M. Zhang et al. 2025b) and to representative high-speed imagery, with datasets fixed before results are inspected, and failure analysis should cover false alarms, missed targets and localisation error.

8.2 Further experiments

Several controlled experiments would sharpen the conclusions. A single evaluation of all saved checkpoints with one frame list, AP calculation and recalibration setting would remove the remaining differences between records, and saving predictions would avoid repeating inference for later analysis. Base with MOCID's plain FPN, followed by a consistent Base+FISTA+DAM ladder, would separate neck changes from backbone changes and give a cleaner measure of the DAM's contribution. A DAM widened to the published parameter budget would test whether the size difference matters. For the MambaIR variant, per-pixel LayerNorm should be tested separately from the other options. For the CFMW variant, the token grid and branch depth remain untested. For the MambaTree variant, a single tree spanning both frames would test the integration option not pursued here, and a faster tree implementation would allow a run at $N = 32$.

8.3 Decision criteria

Future experiments should be chosen for the uncertainty they resolve rather than for the likelihood of a higher headline score. A clean result, meaning one obtained with a matched control and a single changed factor under an agreed protocol, is sufficient to reject a configuration even when it shows no improvement. An improvement should be supported by that same protocol, by variation across seeds, and by a measured computational cost. Appendix I sets out the evidence each recommendation should produce.

9 Conclusion

This project set out to determine whether Mamba-based building blocks could improve the detection of small moving targets in infrared imagery, as a step towards monitoring high-speed airborne vehicles from space. A systematic review selected Mamba, a formal technical description established how it works, and MOCID provided a detector in which to test it.

The project's first achievement was to make that detector work. Three separate faults in weight-decay scope, BatchNorm evaluation statistics and the handover into stage 2 were found, explained and corrected. The result was a stable baseline at 88.83% COCO AP50 and a complete MOCID at 90.12% VOC AP50, supported by tools that verify every new module before it is trained.

On that foundation, three modules each tested a different idea from the literature. The MambaIR-inspired DAM added local detail and reached 89.53%. The CFMW-inspired DAM added global context and reached 90.27%. The MambaTree-inspired DAM changed the reading order and reached 90.19%, a gain that a matched control attributed mainly to its smaller state size. None improved reliably on the corrected original DAM, and every addition carried a cost in parameters, inference time or training time. Measured on accuracy alone, the corrected original DAM therefore remains the preferred design, and none of the new variants is recommended for adoption.

The lesson of the project lies in why. Three different modifications reached the same outcome, while one model scored 91.99% with its DAM switched off. A strong building block cannot help if the path carrying its output into the detector works against it, and its benefit cannot be judged at all unless the training, evaluation and baseline around it can be trusted.

Objectives O1, O2 and O3 were achieved in full, and O4 and O5 were achieved within the MOCID pipeline on DAUB, with benchmark integration and the remaining timing measurements passed on as recommendations. The team leaves behind a working detector, three evaluated modules, a repeatable comparison procedure and a clear next question. Future work can now ask not which scan to use, but how displacement information should reach the detector, and that question brings the goal of detecting hypersonic vehicles from space one step closer.

References

Atrash, Abdulkarim, Seyda Ertekin, Omur Ugur, Omar Moured, Yufan Chen, and Jiaming Zhang (Oct. 2025). "TY-RIST: Tactical YOLO Tricks for Real-time Infrared Small Target Detection". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops*, pp. 2201–2210.

Blelloch, Guy E. (1990). *Prefix Sums and Their Applications*. Tech. rep. CMU-CS-90-190. School of Computer Science, Carnegie Mellon University.

Chen, Guang, Haitao Wang, Kai Chen, Zhijun Li, Zida Song, Yinlong Liu, Wenkai Chen, and Alois Knoll (2020). "A Survey of the Four Pillars for Small Object Detection: Multiscale Representation, Contextual Information, Super-Resolution, and Region Proposal". In: *IEEE Transactions on Systems, Man, and Cybernetics: Systems* 52.2, pp. 936–953. DOI: 10.1109/TSMC.2020.3005231.

Chen, Shengjia, Luping Ji, Jiahao Zhu, Mao Ye, and Xiaoyu Yao (2024). "SSTNet: Sliced Spatio-Temporal Network With Cross-Slice ConvLSTM for Moving Infrared Dim-Small Target Detection". In: *IEEE Transactions on Geoscience and Remote Sensing*.

Dahlgren, Masao (2023). *Getting on Track: Space and Airborne Sensors for Hypersonic Missile Defense*. Tech. rep. Accessed: 2 March 2026. Washington, D.C.: Center for Strategic and International Studies. URL: https://www.csis.org/analysis/getting-track-space-and-airborne-sensors-hypersonic-missile-defense.

Dao, Tri, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré (2022). "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh. Vol. 35. Curran Associates, Inc.

Everingham, Mark, Luc Van Gool, Christopher K. I. Williams, John Winn, and Andrew Zisserman (2010). "The Pascal Visual Object Classes (VOC) Challenge". In: *International Journal of Computer Vision* 88.2, pp. 303–338. DOI: 10.1007/s11263-009-0275-4.

Fournier, Quentin, Daniel Aloise, and Gilles Caron (2023). "A Practical Survey on Faster and Lighter Transformers". In: *ACM Computing Surveys* 55.14s, pp. 1–40. DOI: 10.1145/3586074.

Gong, Tao, Kai Chen, Xinggang Wang, Qian Chu, Feng Zhu, Dahua Lin, Nenghai Yu, and Huamin Feng (2021). *Temporal RoI Align for Video Object Recognition*. arXiv: 2109.03495. URL: https://arxiv.org/abs/2109.03495.

Grootendorst, Maarten (2024). *A Visual Guide to Mamba and State Space Models*. Personal blog. Accessed: April 2026. URL: https://newsletter.maartengrootendorst.com/p/a-visual-guide-to-mamba-and-state.

Gu, Albert and Tri Dao (2024). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. DOI: 10.48550/arXiv.2312.00752. arXiv: 2312.00752 [cs.LG]. URL: https://arxiv.org/abs/2312.00752.

Gu, Albert, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré (2020). "HiPPO: Recurrent Memory with Optimal Polynomial Projections". In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin. Vol. 33. Curran Associates, Inc., pp. 1474–1487. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/102f0bb6efb3a6128a3c750dd16729be-Paper.pdf.

Gu, Albert, Karan Goel, and Christopher Ré (2021a). "Efficiently Modeling Long Sequences with Structured State Spaces". In: *CoRR* abs/2111.00396. DOI: 10.48550/arXiv.2111.00396. arXiv: 2111.00396. URL: https://arxiv.org/abs/2111.00396.

Gu, Albert, Isys Johnson, Karan Goel, Khaled Saab, Tri Dao, Atri Rudra, and Christopher Ré (2021b). "Combining Recurrent, Convolutional, and Continuous-time Models with Linear State Space Layers". In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan. Vol. 34. Curran Associates, Inc., pp. 572–585. URL: https://proceedings.neurips.cc/paper_files/paper/2021/file/05546b0e38ab9175cd905eebcc6ebb76-Paper.pdf.

Guo, Hang, Jinmin Li, Tao Dai, Zhihao Ouyang, Xudong Ren, and Shu-Tao Xia (2025). "MambaIR: A Simple Baseline for Image Restoration with State-Space Model". In: *Computer Vision – ECCV 2024*. Ed. by Aleš Leonardis, Elisa Ricci, Stefan Roth, Olga Russakovsky, Torsten Sattler, and Gül Varol. Cham: Springer Nature Switzerland, pp. 222–241. DOI: 10.1007/978-3-031-72649-1_13.

Hu, Chen, Mingyu Zhou, Shuai Yuan, Hongbo Hu, Zhenming Peng, Tian Pu, and Xiying Li (2026). "STGBD-Net: Spatio-Temporal Gradient Basis Decomposition Network for Infrared Small Target Detection". In: *IEEE Transactions on Geoscience and Remote Sensing* 64, pp. 5006714–5006714. DOI: 10.1109/TGRS.2026.3701189.

Hui, Bingwei, Zhiyong Song, Hongqi Fan, Ping Zhong, Weidong Hu, Xiaofeng Zhang, Jianguang Lin, Hongying Su, Wei Jin, Yongjun Zhang, et al. (2019). "A Dataset for Infrared Image Dim-Small Aircraft Target Detection and Tracking under Ground/Air Background". In: *Science Data Bank* 5.12, p. 4.

Li, Haoyuan, Qi Hu, et al. (2025). "CFMW: Cross-modality Fusion Mamba for Robust Object Detection under Adverse Weather". In: *IEEE Transactions on Circuits and Systems for Video Technology* 35.12, pp. 12066–12081. DOI: 10.1109/TCSVT.2025.3587918.

Li, Ruojing, Wei An, Chao Xiao, Boyang Li, Yingqian Wang, Miao Li, and Yulan Guo (2023). "Direction-Coded Temporal U-Shape Module for Multiframe Infrared Small Target Detection". In: *IEEE Transactions on Neural Networks and Learning Systems*.

Lin, Tsung-Yi, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C. Lawrence Zitnick (2014). "Microsoft COCO: Common Objects in Context". In: *Computer Vision – ECCV 2014*. Cham: Springer International Publishing, pp. 740–755. DOI: 10.1007/978-3-319-10602-1_48.

Liu, Yue, Yu Tian, Yuzhong Zhao, Hongtian Yu, Lingxi Xie, Yaowei Wang, Qixiang Ye, and Yun Liu (2024). *VMamba: Visual State Space Model*. arXiv: 2401.10166 [cs.CV]. URL: https://arxiv.org/abs/2401.10166.

Mienye, Ibomoiye, Theo Swart, and George Obaido (2024). "Recurrent Neural Networks: A Comprehensive Review of Architectures, Variants, and Applications". In: *Information* 15, p. 517. DOI: 10.3390/info15090517.

Rao, Yongming, Wenliang Zhao, Zheng Zhu, Jiwen Lu, and Jie Zhou (2021). "Global Filter Networks for Image Classification". In: *Advances in Neural Information Processing Systems*. Vol. 34. Curran Associates, Inc., pp. 980–993.

Serrano, Luis (2024). *State Space Models and Mamba*. Serrano Academy, YouTube. Accessed: April 2026. URL: https://www.youtube.com/@SerranoAcademy.

Su, Zhuo, Li Liu, Matthias Müller, Jiehua Zhang, Diana Wofk, Ming-Ming Cheng, and Matti Pietikäinen (2025). "Rapid Salient Object Detection With Difference Convolutional Neural Networks". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 47.10, pp. 9061–9077. DOI: 10.1109/TPAMI.2025.3583968.

Van Wie, David M (2021). "Hypersonics: Past, present, and potential future". In: *Johns Hopkins APL Technical Digest* 35.4, pp. 335–341.

Wang, Jingdong, Ke Sun, Tianheng Cheng, Borui Jiang, Chaorui Deng, Yang Zhao, Dong Liu, Yadong Mu, Mingkui Tan, Xinggang Wang, Wenyu Liu, and Bin Xiao (2021). "Deep High-Resolution Representation Learning for Visual Recognition". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.10, pp. 3349–3364. DOI: 10.1109/TPAMI.2020.2983686.

Xiao, Yicheng, Lin Song, Shaoli Huang, Jiangshan Wang, Siyu Song, Yixiao Ge, Xiu Li, and Ying Shan (2024). "MambaTree: Tree Topology is All You Need in State Space Model". In: *Advances in Neural Information Processing Systems*. Ed. by A. Globerson, L. Mackey, D. Belgrave, A. Fan, U. Paquet, J. Tomczak, and C. Zhang. Vol. 37. Curran Associates, Inc. DOI: 10.52202/079017-2398. URL: https://proceedings.neurips.cc/paper_files/paper/2024/file/89b89c04f55ea7c7ca989992bb6a98c0-Paper-Conference.pdf.

Yao, Tingting, Jianzhi Yu, Xinyu Gu, Meiwen Zhu, Fuxiao Cao, and Qing Hu (2026). "OVSMMFA-Net: An object variation sensitive and multi-direction mamba based feature aggregation network for video object detection". In: *Digital Signal Processing* 175, p. 106006. ISSN: 1051-2004. DOI: 10.1016/j.dsp.2026.106006.

Yu, Zitong, Benjia Zhou, Jun Wan, Pichao Wang, Haoyu Chen, Xin Liu, Stan Z. Li, and Guoying Zhao (2021). "Searching Multi-Rate and Multi-Modal Temporal Enhanced Networks for Gesture Recognition". In: *IEEE Transactions on Image Processing* 30, pp. 5626–5640. DOI: 10.1109/TIP.2021.3087348.

Zang, Dongyuan, Weihua Su, Bonan Zhang, and Huixin Liu (2025). "DCANet: Dense Convolutional Attention Network for Infrared Small Target Detection". In: *Measurement* 240. DOI: 10.1016/j.measurement.2024.115595.

Zhang, Luping, Junhai Luo, Yian Huang, Fengyi Wu, Xingye Cui, and Zhenming Peng (2025). "MDIGCNet: Multidirectional Information-Guided Contextual Network for Infrared Small Target Detection". In: *IEEE Journal of Selected Topics in Applied Earth Observations and Remote Sensing* 18, pp. 2063–2076. DOI: 10.1109/JSTARS.2024.3508255.

Zhang, Mingjin, Xiaolong Li, Fei Gao, and Jie Guo (2025a). "IRMamba: Pixel Difference Mamba with Layer Restoration for Infrared Small Target Detection". In: *The Thirty-Ninth AAAI Conference on Artificial Intelligence (AAAI-25)*. Vol. 39. 9, pp. 10003–10011. DOI: 10.1609/aaai.v39i9.33085.

Zhang, Mingjin, Yuanjun Ouyang, Fei Gao, Jie Guo, Qiming Zhang, and Jing Zhang (Apr. 2025b). "MOCID: Motion Context and Displacement Information Learning for Moving Infrared Small Target Detection". In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 39. 10, pp. 10022–10030. DOI: 10.1609/aaai.v39i10.33087.

Zhou, Qianyu, Xiangtai Li, Lu He, Yibo Yang, Guangliang Cheng, Yue Tong, Lizhuang Ma, and Dacheng Tao (2023). "TransVOD: End-to-End Video Object Detection With Spatial-Temporal Transformers". In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 45.6, pp. 7853–7869.

Statement of AI Tool Use

This statement details the use of AI tools in this assignment, in line with the course AI Tool Use Guide. The team reviewed all AI output, checked every reported number against the saved evaluation logs, and takes responsibility for the content of this report. No model training result or measurement was generated by an AI tool.

Category 1: Copy Editing

Grammarly (https://www.grammarly.com) and Claude Code (https://claude.com/claude-code) were used to check spelling, grammar, tense and punctuation in text written by the team. All suggestions were reviewed before being accepted.

Category 3: Visuals

Claude Code and Codex (https://openai.com/codex) helped prepare the LaTeX code of the block diagrams and result plots in this report, using data from the saved logs. The team specified the content of each figure and checked it against the source data.

Category 4: Substantive Editing

Claude Code and Codex were used to suggest structure, reword and condense team drafts, and improve the flow of the report. This included assistance in drafting the Executive Summary, Introduction, Sections 5 to 9, and the appendices from the team's notes, results and supervisor feedback. The authors reviewed, edited and approved this text.

Category 5: Idea Generation

Claude (https://claude.ai) and ChatGPT (https://chatgpt.com) were used early in the project to explain concepts related to the Mamba architecture and state space models. Claude Code was also used to explain code and results to team members.

Category 7: Research Technical Support

Claude Code assisted with implementing and debugging the MambaIR-inspired, CFMW-inspired and MambaTree-inspired DAMs, the verification and demonstration tools, and with analysing training logs, as described in Section 5. The repository's prompt log records this assistance for the MambaIR implementation. Codex was used to inspect code and records and to recalculate summaries of saved logs. Neither tool generated experimental results.

Categories 2 and 6

No AI tools were used for literature review support (Category 2) or translation (Category 6).

Copies of the material submitted to these tools and of their outputs have been retained.

A Understanding State and Memory

A.1 A physical example

Consider a vehicle moving along a straight line. A simple state contains its position p and velocity v. If acceleration a is constant during a time interval δ, the update is

$$\begin{bmatrix} p_{t+1} \\ v_{t+1} \end{bmatrix} = \begin{bmatrix} 1 & \delta \\ 0 & 1 \end{bmatrix} \begin{bmatrix} p_t \\ v_t \end{bmatrix} + \begin{bmatrix} \delta^2/2 \\ \delta \end{bmatrix} a_t. \tag{35}$$

For $p_t = 10$ m, $v_t = 2$ m/s, $a_t = 1$ m/s^2, and $\delta = 1$ s, the next state is $p_{t+1} = 12.5$ m and $v_{t+1} = 3$ m/s. The previous state carries information forward, while the input changes that state. Reading only the position corresponds to an output matrix [1 0].

A learned SSM uses the same state-update structure but does not assign these physical meanings to its memory values. Each value can represent a learned combination of earlier features. The physical example explains the roles of state, input, and output. It is not an assertion that the detector estimates acceleration or maintains a trajectory.

A.2 A scalar memory

An even simpler example is

$$h_t = a h_{t-1} + (1 - a) x_t, \qquad 0 < a < 1. \tag{36}$$

If a is close to one, much of the previous state survives. If a is small, the current input has greater influence. Figure 22 shows how an existing unit state decays with no further input. The curves are mathematical illustrations, not measurements from the detector.

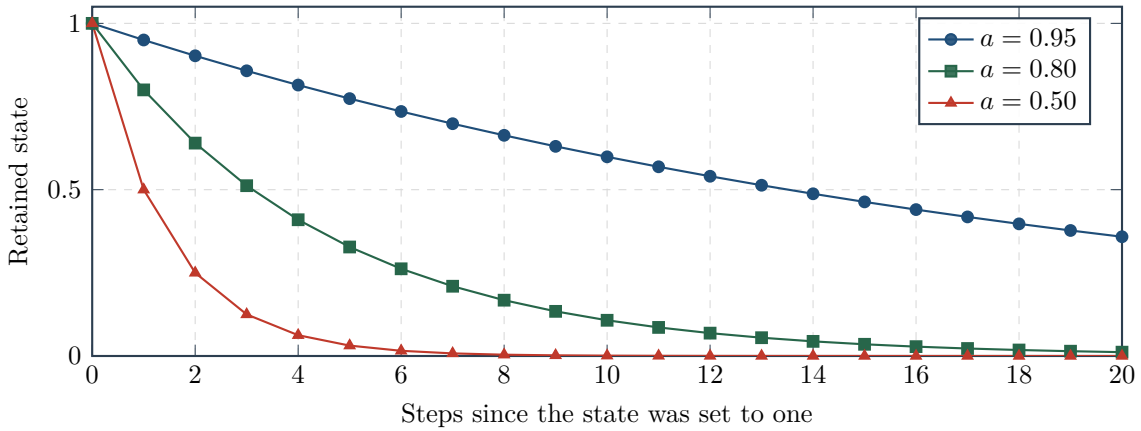


Figure 22: Illustrative retention of a unit state under $h_t = a^t h_0$, with $h_0 = 1$ and zero subsequent inputs. Different transition values produce different memory lengths.

One scalar state has limited capacity. Several state variables can retain information on different scales. That is the purpose of the state dimension N. In the recorded MOCID module, $N = 32$. Increasing N changes capacity and computational cost, so a comparison that also changes N is not a scan-only ablation.

The following two extended analogies, a vehicle-monitoring system and a language model, were used by the project team during the early stages of the literature review. They draw on the visual treatment of Serrano (2024) and Grootendorst (2024) to build understanding of the underlying concepts before engaging with the original Mamba paper (Gu and Dao 2024).

A.3 The State Space of a Race Car

Consider monitoring the operational state of a race car day by day. There are quantities that can be *set* by an operator (maintenance actions), quantities that describe the *condition* of the vehicle (its internal state), and quantities that can be *measured* (its performance). These three roles map directly onto the input, latent state, and output of a state space model.

State, Input, and Output

Let the state of the vehicle on day t be described by four variables: the gas level, the oil level, the tyre condition, and the motor condition. The state is therefore a vector

$$h_t = \begin{pmatrix} \text{gas} \\ \text{oil} \\ \text{tyres} \\ \text{motor} \end{pmatrix}. \tag{37}$$

The input on day t is the maintenance applied to the car, here taken to be two scalar quantities: the volume of fluid added and the amount of repair work undertaken,

$$x_t = \begin{pmatrix} \text{fluids} \\ \text{repairs} \end{pmatrix}. \tag{38}$$

The output is the measured top speed of the vehicle on day t, written as a scalar y_t. The aim of the model is, given the previous state h_{t-1} and the input x_t, to compute the new state h_t and the resulting output y_t via the relations

$$h_t = \overline{\mathbf{A}}\, h_{t-1} + \overline{\mathbf{B}}\, x_t, \tag{39}$$

$$y_t = \mathbf{C}\, h_t. \tag{40}$$

The State Transition Matrix $\overline{\mathbf{A}}$

The matrix $\overline{\mathbf{A}}$ encodes the natural evolution of the vehicle state in the absence of any maintenance: in other words, the wear and tear that accumulates from one day to the next. Suppose that, on a typical day, the vehicle loses 10% of its gas, 5% of its oil, 20% of its tyre condition, and 15% of its motor condition. Suppose further that good tyres slightly reduce gas consumption (so well-maintained tyres add a small amount to the gas reading) and that high oil levels slightly improve motor condition. A possible $\overline{\mathbf{A}}$ matrix consistent with these dynamics is

$$\overline{\mathbf{A}} = \begin{pmatrix} 0.90 & 0 & 0.05 & 0 \\ 0 & 0.95 & 0 & 0 \\ 0 & 0 & 0.80 & 0 \\ 0 & 0.05 & 0 & 0.85 \end{pmatrix}, \tag{41}$$

where the rows and columns are ordered as gas, oil, tyres, and motor. The diagonal entries describe the day-to-day persistence of each component, and the off-diagonal entries describe the interactions between components.

The Input Matrix $\overline{\mathbf{B}}$

The matrix $\overline{\mathbf{B}}$ encodes how the maintenance input affects the state. Suppose that adding fluids increases the gas and oil readings by one unit each, and that repairs increase the tyre and motor condition by one unit each. The corresponding $\overline{\mathbf{B}}$ matrix is

$$\overline{\mathbf{B}} = \begin{pmatrix} 1 & 0 \\ 1 & 0 \\ 0 & 1 \\ 0 & 1 \end{pmatrix}, \tag{42}$$

where the columns correspond to the fluids and repairs components of the input vector and the rows correspond to the four state variables.

The Output Matrix C

The matrix $\mathbf{C}$ is a row vector that converts the state vector into the scalar output (the top speed). Suppose the contributions to top speed are 0.4 per unit of gas, 0.2 per unit of oil, 0.3 per unit of tyre condition, and 0.5 per unit of motor condition,

$$\mathbf{C} = \begin{pmatrix} 0.4 & 0.2 & 0.3 & 0.5 \end{pmatrix}. \tag{43}$$

A Single Step of the Model

Suppose the previous state is $h_{t-1} = (1, 1, 1, 1)^\top$ (each component fully replenished) and the maintenance applied today is $x_t = (0.4, 0.3)^\top$ (a small top-up of fluids and a small amount of repair work). The transition produces

$$\overline{\mathbf{A}}\, h_{t-1} = \begin{pmatrix} 0.90 + 0.05 \\ 0.95 \\ 0.80 \\ 0.05 + 0.85 \end{pmatrix} = \begin{pmatrix} 0.95 \\ 0.95 \\ 0.80 \\ 0.90 \end{pmatrix}, \tag{44}$$

and the input contribution is

$$\overline{\mathbf{B}}\, x_t = \begin{pmatrix} 0.4 \\ 0.4 \\ 0.3 \\ 0.3 \end{pmatrix}. \tag{45}$$

The new state is therefore $h_t = (1.35, 1.35, 1.10, 1.20)^\top$, and the output is

$$y_t = \mathbf{C}\, h_t = 0.4 \cdot 1.35 + 0.2 \cdot 1.35 + 0.3 \cdot 1.10 + 0.5 \cdot 1.20 = 1.74. \tag{46}$$

The example illustrates the three mechanisms encoded by the SSM. $\overline{\mathbf{A}}$ governs how the system would evolve on its own, $\overline{\mathbf{B}}$ governs how external action enters the state, and $\mathbf{C}$ governs how the state is translated into an observable measurement.

A.4 Application in Language

The same concept applies to language modelling, where the state describes the context of a piece of text, the input is the most recent token, and the output is the predicted next token. Consider the sentence

> *"Last week we saw a TV documentary about wild animals. We saw giraffes with long. . . "*

A language model attempts to predict the next token, in this case most likely *"necks"*.

Mapping the Three Quantities

The latent state h_t is a high-dimensional vector that summarises everything the model has read so far: the topic (zoology), the tense (past), the entities mentioned recently (TV, documentary, animals, giraffes), and many other latent features that the model has learned to track. The input x_t is the embedding of the most recent token (*"long"*), typically represented as a sparse one-hot vector that is then projected into a dense embedding space. The output y_t is a probability distribution over the vocabulary indicating what the model believes the next token to be. For the example above, the output might assign high probability to *"necks"* and lower probability to *"legs"*, *"tails"*, and other plausible continuations.

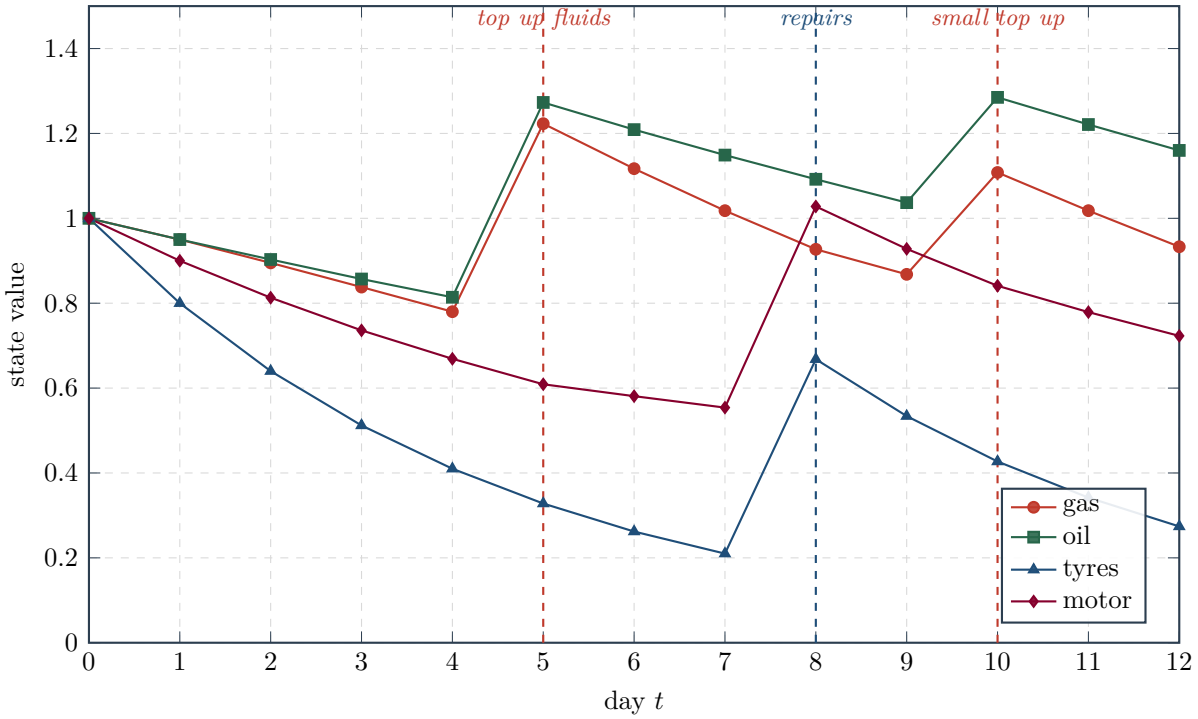


Figure 23: Evolution of the race car's four state components over twelve days. Between maintenance events, each component decays according to the transition matrix $\overline{\mathbf{A}}$, with the rate determined by the diagonal entries (gas at 0.90, oil at 0.95, tyres at 0.80, motor at 0.85). Maintenance inputs x_t produce upward spikes through the matrix $\overline{\mathbf{B}}$: fluid top-ups raise gas and oil simultaneously, while repairs raise tyres and motor.

Interpretation of the Matrices

The matrix $\overline{\mathbf{A}}$ describes how the context drifts as the model moves through the sequence in the absence of new input. Older entities slowly fade from the relevant context, while recently mentioned entities remain prominent. In the example, the matrix encodes the fact that "TV" was mentioned a few tokens ago and is still moderately active, while "last week" was mentioned earlier and has begun to fade.

The matrix $\overline{\mathbf{B}}$ describes how the most recent token modifies the context. The token *"long"* introduces a new feature into the state, namely that the next noun is likely to be something with a long dimension. Different tokens activate different features in the latent state, and $\overline{\mathbf{B}}$ encodes the mapping from token embedding to feature update.

The matrix $\mathbf{C}$ describes how the context is translated into a prediction over the vocabulary. Given the active features (zoology, giraffe, long), $\mathbf{C}$ produces logits that, after softmax, place high mass on tokens that match those features.

Why This Motivates Selectivity

In the formal SSM, the matrices $\overline{\mathbf{A}}$, $\overline{\mathbf{B}}$, and $\mathbf{C}$ are fixed for the entire sequence. This is acceptable for a physical system whose dynamics genuinely do not change, but it is a poor assumption for language. The token *"the"* should update the state much less than the token *"giraffe"*, and the token *"um"* should ideally not update the state at all. A linear time-invariant SSM cannot make this distinction. The selection mechanism described in Appendix C addresses this limitation by making $\overline{\mathbf{B}}$, $\mathbf{C}$, and the timestep Δ functions of the current input, allowing the model to update the state more or less aggressively depending on what the token is.

B Recurrence and Convolution Worked Example

B.1 Calculating one step at a time

Take $h_t = 0.5h_{t-1} + x_t$, $y_t = h_t$, and $h_{-1} = 0$. For inputs $(1, 2, 0, 1)$, the first output is 1. The next state retains half of that value and adds 2, giving 2.5. The remaining outputs follow in the same way.

Table 18 records each step. The example uses a fixed coefficient and no direct input term beyond the state update. It demonstrates the linear time-invariant case described in Section 3.3.

Table 18: A complete scalar recurrence calculation.

t	x_t	h_{t-1}	$h_t = y_t$	Calculation
0	1	0	1	$0.5(0) + 1$
1	2	1	2.5	$0.5(1) + 2$
2	0	2.5	1.25	$0.5(2.5) + 0$
3	1	1.25	1.625	$0.5(1.25) + 1$

B.2 Calculating the same result as a convolution

Unrolling the recurrence gives a kernel $(1, 0.5, 0.25, 0.125, \ldots)$. The fourth output is therefore

$$y_3 = 1(1) + 0.5(0) + 0.25(2) + 0.125(1) = 1.625. \tag{47}$$

The first term uses the current input, the second uses the preceding input, and so on. In matrix form, all four outputs can be calculated together as

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0.5 & 1 & 0 & 0 \\ 0.25 & 0.5 & 1 & 0 \\ 0.125 & 0.25 & 0.5 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 0 \\ 1 \end{bmatrix}. \tag{48}$$

The repeated diagonal pattern is what allows one fixed convolution kernel. If the transition changes with the input, the weight on an earlier input instead depends on a product of the intervening transitions. The recurrence remains valid, but the repeated diagonal pattern generally disappears. A non-zero initial state would also add a separate state-response term.

B.3 Step-by-step Unrolling of the Recurrence

The scalar example above generalises directly to matrix parameters. Consider the discrete recurrence in Equations (5) and (6) with initial state $h_{-1} = 0$. Each successive state and output can be written explicitly in terms of the input sequence by repeated substitution.

At time step $t = 0$, there is no previous state, so the recurrence reduces to

$$h_0 = \overline{\mathbf{B}}\, x_0, \tag{49}$$

$$y_0 = \mathbf{C}\, h_0 = \mathbf{C}\overline{\mathbf{B}}\, x_0. \tag{50}$$

At time step $t = 1$, the previous state is $h_0 = \overline{\mathbf{B}}\, x_0$, so

$$h_1 = \overline{\mathbf{A}}\, h_0 + \overline{\mathbf{B}}\, x_1 = \overline{\mathbf{A}}\,\overline{\mathbf{B}}\, x_0 + \overline{\mathbf{B}}\, x_1, \tag{51}$$

$$y_1 = \mathbf{C}\, h_1 = \mathbf{C}\overline{\mathbf{A}}\,\overline{\mathbf{B}}\, x_0 + \mathbf{C}\overline{\mathbf{B}}\, x_1. \tag{52}$$

At time step $t = 2$, the previous state is $h_1 = \overline{\mathbf{A}}\,\overline{\mathbf{B}}\,x_0 + \overline{\mathbf{B}}\,x_1$, so

$$h_2 = \overline{\mathbf{A}}\,h_1 + \overline{\mathbf{B}}\,x_2 = \overline{\mathbf{A}}^2\overline{\mathbf{B}}\,x_0 + \overline{\mathbf{A}}\,\overline{\mathbf{B}}\,x_1 + \overline{\mathbf{B}}\,x_2, \tag{53}$$

$$y_2 = \mathbf{C}\,h_2 = \mathbf{C}\overline{\mathbf{A}}^2\overline{\mathbf{B}}\,x_0 + \mathbf{C}\overline{\mathbf{A}}\,\overline{\mathbf{B}}\,x_1 + \mathbf{C}\overline{\mathbf{B}}\,x_2. \tag{54}$$

The pattern is clear: the input x_j contributes to the output y_t with coefficient $\mathbf{C}\overline{\mathbf{A}}^{t-j}\overline{\mathbf{B}}$. The general expression is

$$y_t = \sum_{j=0}^{t} \mathbf{C}\overline{\mathbf{A}}^{t-j}\overline{\mathbf{B}}\,x_j = \sum_{k=0}^{t} \mathbf{C}\overline{\mathbf{A}}^{k}\overline{\mathbf{B}}\,x_{t-k}. \tag{55}$$

B.4 Recognising the Convolution

Equation (55) matches exactly the definition of a one-dimensional discrete convolution. Define the kernel

$$\overline{\mathbf{K}} = \left(\overline{K}_0, \overline{K}_1, \overline{K}_2, \dots\right) = \left(\mathbf{C}\overline{\mathbf{B}},\ \mathbf{C}\overline{\mathbf{A}}\,\overline{\mathbf{B}},\ \mathbf{C}\overline{\mathbf{A}}^2\overline{\mathbf{B}},\ \dots\right). \tag{56}$$

Then $y_t = \sum_{k=0}^{t} \overline{K}_k\, x_{t-k}$, which is the formula for a one-dimensional convolution truncated to past inputs only. Equivalently, the output sequence is $y = x * \overline{\mathbf{K}}$. The kernel is fixed (independent of t) precisely because $\overline{\mathbf{A}}$, $\overline{\mathbf{B}}$, and $\mathbf{C}$ are constant in time. This is the formal reason why time-invariance is required for the convolutional form.

B.5 Numerical Verification

To confirm the equivalence in a concrete case, take scalar parameters $\overline{\mathbf{A}} = 0.9$, $\overline{\mathbf{B}} = 1$, $\mathbf{C} = 1$, and the input sequence $x = (1, 0, 1)$.

Computation via the recurrence. With $h_{-1} = 0$,

$$\begin{aligned} h_0 &= 1 \cdot 1 = 1, & y_0 &= 1 \cdot 1 = 1, \\ h_1 &= 0.9 \cdot 1 + 1 \cdot 0 = 0.9, & y_1 &= 1 \cdot 0.9 = 0.9, \\ h_2 &= 0.9 \cdot 0.9 + 1 \cdot 1 = 1.81, & y_2 &= 1 \cdot 1.81 = 1.81. \end{aligned}$$

The recurrent form gives $y = (1.00, 0.90, 1.81)$.

Computation via the convolution. The kernel is

$$\overline{\mathbf{K}} = \left(\mathbf{C}\overline{\mathbf{B}},\ \mathbf{C}\overline{\mathbf{A}}\,\overline{\mathbf{B}},\ \mathbf{C}\overline{\mathbf{A}}^2\overline{\mathbf{B}}\right) = (1, 0.9, 0.81)\,. \tag{57}$$

Convolving x with $\overline{\mathbf{K}}$ gives

$$\begin{aligned} y_0 &= \overline{K}_0 \cdot x_0 = 1 \cdot 1 = 1.00, \\ y_1 &= \overline{K}_0 \cdot x_1 + \overline{K}_1 \cdot x_0 = 1 \cdot 0 + 0.9 \cdot 1 = 0.90, \\ y_2 &= \overline{K}_0 \cdot x_2 + \overline{K}_1 \cdot x_1 + \overline{K}_2 \cdot x_0 = 1 \cdot 1 + 0.9 \cdot 0 + 0.81 \cdot 1 = 1.81. \end{aligned}$$

The convolutional form gives the same output sequence $y = (1.00, 0.90, 1.81)$, verifying the equivalence.

B.6 What Selectivity Changes

The kernel $\overline{\mathbf{K}}$ used above has a fixed exponential decay shape, $1, 0.9, 0.81, \ldots$, that is applied identically to every input regardless of its content. If the SSM were instead made selective by allowing $\overline{\mathbf{A}}$ to depend on the input, for example $\overline{\mathbf{A}}_t = 0.9$ when $x_t \neq 0$ and $\overline{\mathbf{A}}_t = 0.1$ when $x_t = 0$, the model would effectively reset its state when an irrelevant input is encountered. Repeating the calculation above with this rule gives $h_0 = 1$, $h_1 = 0.1 \cdot 1 + 0 = 0.1$ (because $x_1 = 0$ now triggers the reset), $h_2 = 0.9 \cdot 0.1 + 1 = 1.09$, so $y = (1.00, 0.10, 1.09)$. The intermediate output is much smaller than under the LTI rule, reflecting the model's decision to discard the transient zero input. The same calculation cannot be expressed as a convolution with a single fixed kernel, illustrating concretely why the selection mechanism breaks the convolutional form and motivates the hardware-aware scan algorithm of Appendix E.

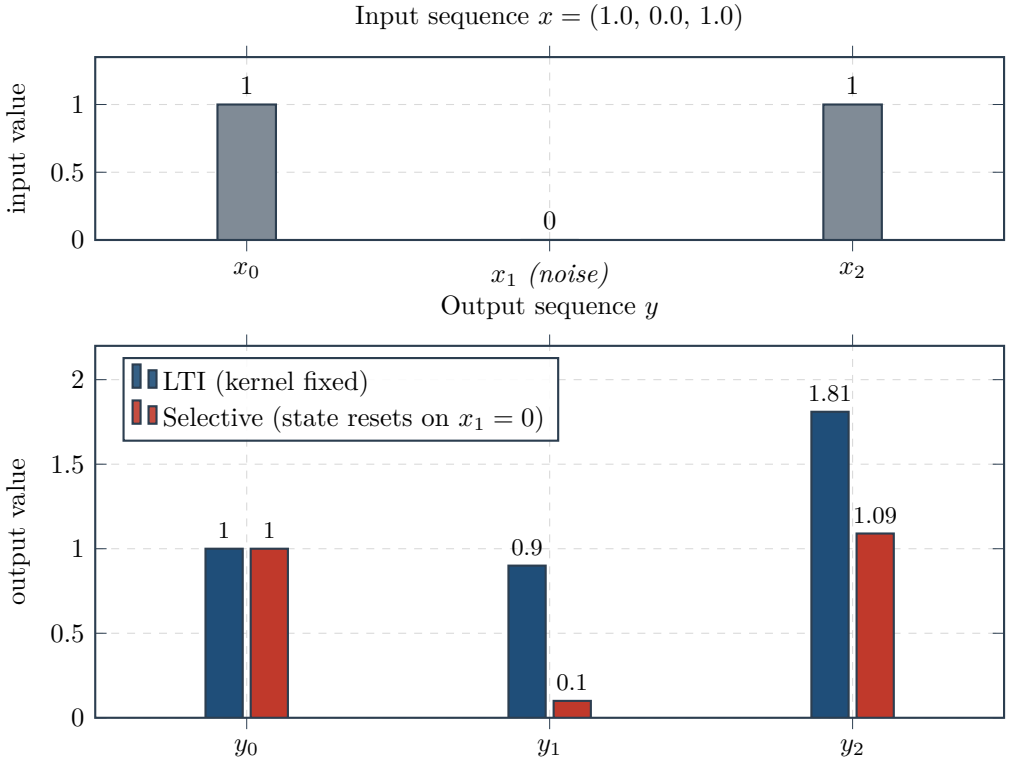


Figure 24: Effect of selectivity on a sequence with a noise token. The LTI model applies the same fixed kernel at every step, so the noise input $x_1 = 0$ still leaves a strong contribution at $y_2 = 1.81$. The selective model detects the noise input and effectively resets the state, suppressing both y_1 and the noise's contribution to y_2.

C Selection and Discretisation Explained

C.1 From continuous parameters to a discrete update

The continuous-state formulation follows the state-space literature (Gu et al. 2021a). For a scalar continuous model $h'(t) = Ah(t) + Bx(t)$ with the input held constant over an interval Δ, exact integration gives

$$h_t = e^{\Delta A} h_{t-1} + \left(\int_0^{\Delta} e^{sA}\, ds \right) B x_t. \tag{58}$$

For $A \neq 0$, the input coefficient is $(e^{\Delta A} - 1)B/A$. The integral form also covers $A = 0$ without an inverse. This is the zero-order-hold interpretation. The held input is a mathematical device for deriving the update, not an extra continuous signal that the image loader must create.

For $A = -1$ and $B = 1$, the exact scalar update becomes

$$h_t = e^{-\Delta_t} h_{t-1} + (1 - e^{-\Delta_t}) x_t. \tag{59}$$

A small positive Δ_t retains more of the old state. A larger value lets the current input replace more of it. With $\Delta_t = \log(1 + e^{z_t})$, the new-input weight is $\sigma(z_t)$, where σ is the logistic sigmoid. This special case provides a useful intuition for input-dependent selection.

Many practical selective-scan implementations use $\overline{A}_t = e^{\Delta_t A}$ with a simplified input coefficient $\Delta_t B_t$. That update does not in general have exactly the same complementary weights as the exact scalar zero-order-hold example. The detector delegates this calculation to an external kernel. Its precise source version should be archived with the final experiment. The conceptual derivation must not be substituted for checking the implemented update.

C.2 What the elementary operations do

A linear layer forms weighted sums of the input channels, usually adding a bias. It changes a vector of features into the number of values required by the next operation. A projection is therefore a learned transformation, not necessarily a reduction in dimension.

Broadcasting reuses a value across an axis without changing what it means. For example, a learned scale with shape (C) can multiply every spatial position in a tensor of shape (B, C, H, W). The scale is shared across the image, but different channels can have different scales.

Softplus is $\log(1 + e^z)$ and is positive, which makes it suitable for a positive step parameter. Sigmoid is bounded between zero and one and is used in the CAB's channel attention. SiLU is $z\sigma(z)$ and is not restricted to that interval. Calling all three operations a gate without distinguishing them can lead to incorrect interpretations of the block.

C.3 A deliberately selective example

Consider inputs $(1, 0, 0, 2)$ and the illustrative update $h_t = (1 - g_t)h_{t-1} + g_t x_t$. Use $g_t = 0.8$ for the non-zero inputs and $g_t = 0.1$ for the zero inputs. Starting from zero gives states $(0.8, 0.72, 0.648, 1.7296)$. A fixed gate of 0.5 gives $(0.5, 0.25, 0.125, 1.0625)$.

Figure 25 compares the two. The gates were chosen by hand to explain retention and writing. The example does not show that the trained detector recognises targets or rejects clutter in this way. That claim would require an experiment on its learned parameters or outputs.

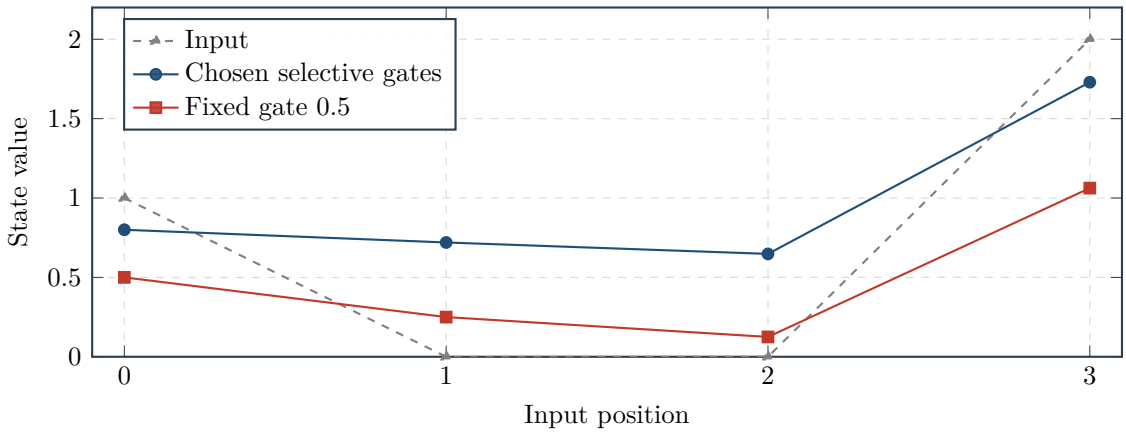


Figure 25: A hand-calculated illustration of selective retention. Small gates retain state through the two intervening zero inputs. No experimental detection performance is represented.

The remainder of this appendix unpacks the selection mechanism operation by operation, before showing how the operations combine on a small numerical example. It is intended for readers without a deep learning background.

C.4 Why a Selection Mechanism is Needed

Recall from Section 3.3 and Appendix B that an LTI state space model uses the same matrices $\overline{\mathbf{A}}$, $\overline{\mathbf{B}}$, $\mathbf{C}$, and the same step size Δ, at every position in the sequence. This is convenient mathematically but limiting in practice. Consider a sequence of words such as

> *"the cat um sat on the mat"*

A useful sequence model should treat the filler word *"um"* differently from the content words *"cat"* and *"sat"*. An LTI model cannot, because the rule it applies to update its state is fixed in advance and does not depend on the word it is currently reading.

The selection mechanism solves this by making the parameters $\overline{\mathbf{B}}$, $\mathbf{C}$, and Δ depend on the current input x_t, through Equations (13)–(15) in Section 3.4. The model now applies a different update rule at every position, and that rule is computed from the input itself. The matrix $\overline{\mathbf{A}}$ is left fixed. Selectivity in Δ already produces selectivity in $\overline{\mathbf{A}}$ through the discretisation rule $\overline{\mathbf{A}} = \exp(\Delta\mathbf{A})$.

C.5 What "$\text{Linear}_N(x)$" Means

The expression $\text{Linear}_N(x)$ is shorthand for a *linear projection*: a small piece of the network that takes an input vector and produces an output vector by multiplying it by a learned matrix and (optionally) adding a learned bias. Concretely, if $x \in \mathbb{R}^d$ is the input and $\mathbf{W} \in \mathbb{R}^{N \times d}$ and $\mathbf{b} \in \mathbb{R}^N$ are learned parameters, then

$$\text{Linear}_N(x) = \mathbf{W}x + \mathbf{b} \in \mathbb{R}^N. \tag{60}$$

The subscript N records the dimension of the output. Three things are worth noting. First, $\mathbf{W}$ and $\mathbf{b}$ are learned during training; they are not specified in advance. Second, every Linear_N that appears in Equations (13)–(15) has its *own* weight matrix and bias; the function names s_B, s_C, s_Δ identify three distinct learned projections. Third, a linear projection is the simplest possible operation that lets the network reshape an input into an output of a different size while still depending on the input's content.

In the selection mechanism, $\text{Linear}_N(x)$ produces a vector in $\mathbb{R}^N$ that the model uses as a content-aware version of $\mathbf{B}$ (or $\mathbf{C}$). The vector changes as x changes, which is precisely the property an LTI model lacks.

C.6 What "Broadcast_D" Means

Broadcasting is a mechanical operation that copies a vector across an additional dimension to make its shape match another tensor. In Equation (15), $\text{Linear}_1(x)$ produces a single scalar per token. The model applies the SSM independently to each of the D channels of the input, and each channel needs its own Δ. Rather than learning a separate scalar for each channel, the model learns one scalar and copies it D times,

$$\text{Broadcast}_D(s) = (s, s, \ldots, s) \in \mathbb{R}^D. \tag{61}$$

This is purely a shape-matching step. No new computation is involved, and no new parameters are introduced. The result is that Δ becomes a length-D vector that depends on x_t but is identical in every channel.

C.7 What "Softplus" Does and Why

The step size Δ in the SSM must be positive. The output of Linear_1 can be any real number, so a function that maps the real line to the positive reals is needed. The softplus function,

$$\text{softplus}(z) = \log(1 + e^z), \tag{62}$$

is a smooth approximation of $\max(0, z)$ that is always strictly positive. It returns approximately z when z is large and positive, approximately zero when z is large and negative, and a smooth interpolation in between. Passing $\text{Linear}_1(x)$ through softplus before treating it as a step size guarantees that Δ is positive while keeping the operation differentiable, so the network can be trained with gradient descent.

C.8 What "Gating" Means

Gating is a mechanism by which a network multiplies one signal by another to control how much of the first signal is allowed to pass through. A typical gate is a number between zero and one. A value of zero blocks the signal entirely, a value of one passes it unchanged, and intermediate values pass a fraction of it. In the selective SSM, the role of the step size Δ is exactly this. When Δ is small, the discretised $\overline{\mathbf{A}} = \exp(\Delta\mathbf{A})$ is close to the identity and the input contribution $\overline{\mathbf{B}}\, x_t$ is small, so the new input has little effect on the state and the past is preserved. When Δ is large, $\overline{\mathbf{A}}$ shrinks the previous state aggressively and $\overline{\mathbf{B}}\, x_t$ is large, so the new input dominates and the past is largely discarded.

The relationship to classical recurrent neural network gates becomes exact in a special case. Setting $N = 1$, $\mathbf{A} = -1$, and $\mathbf{B} = 1$, the discretised parameters reduce to

$$\overline{A} = \exp(-\Delta), \qquad \overline{B} = 1 - \exp(-\Delta). \tag{63}$$

Defining $g_t = 1 - \exp(-\Delta_t)$, the recurrence becomes

$$h_t = (1 - g_t)\, h_{t-1} + g_t\, x_t, \tag{64}$$

which is precisely the form of a one-dimensional gated update. When $g_t \to 0$, the state is preserved (the gate is closed to new input). When $g_t \to 1$, the state is replaced by the new input (the gate is fully open). This is the same mechanism used in LSTMs and gated recurrent units, embedded inside the more general structured recurrence of an SSM.

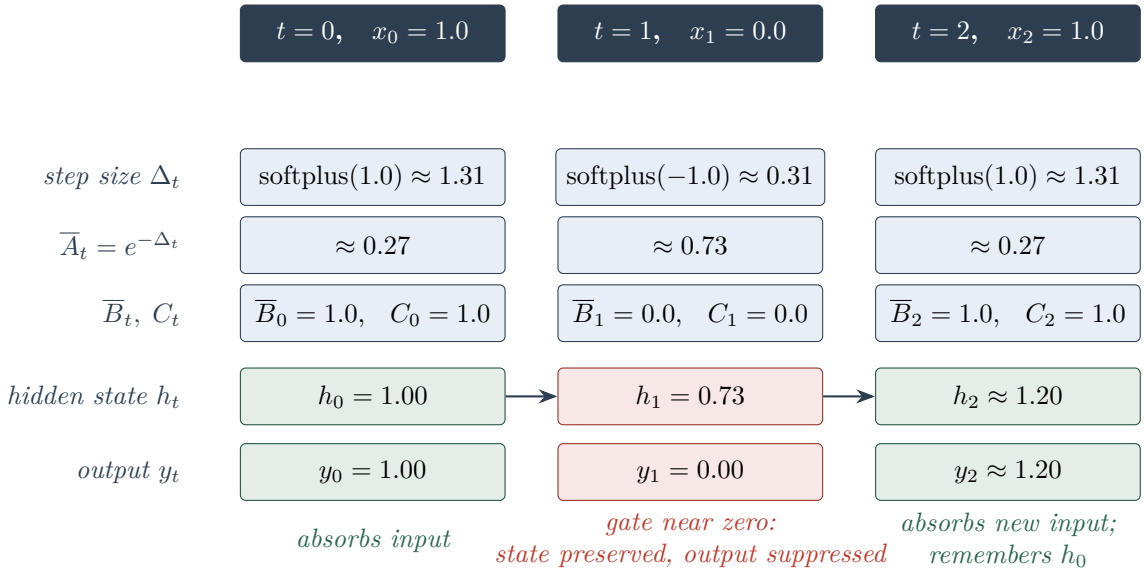


Figure 26: Step-by-step trace of the selective SSM on the input sequence $x = (1.0, 0.0, 1.0)$ with weights $\mathbf{W}_B = \mathbf{W}_C = 1$, $\mathbf{W}_\Delta = 2$, $b_\Delta = -1$, and $\mathbf{A} = -1$. At $t = 0$ the large step size $\Delta_0 \approx 1.31$ opens the gate and the input is absorbed. At $t = 1$ the noise input produces a small step size $\Delta_1 \approx 0.31$, so the state is preserved and the output suppressed. At $t = 2$ the gate reopens, the new input is absorbed, and the output reflects both the new input and the retained memory of h_0.

C.9 A Worked Numerical Example

To make the moving parts concrete, consider a small selective SSM with state dimension $N = 1$ and a one-dimensional input. The fixed parameter is $\mathbf{A} = -1$. The learned weights of the projections s_B, s_C, and s_Δ are taken (for illustration) to be

$$\mathbf{W}_B = 1, \quad b_B = 0, \qquad \mathbf{W}_C = 1, \quad b_C = 0, \qquad \mathbf{W}_\Delta = 2, \quad b_\Delta = -1, \tag{65}$$

so that $s_B(x) = x$, $s_C(x) = x$, and $s_\Delta(x) = 2x - 1$.

The input sequence is $x = (1.0, 0.0, 1.0)$. This represents two informative tokens separated by a noise token (zero). Walk through the three time steps explicitly.

Time step $t = 0$, with $x_0 = 1.0$. Compute the input-dependent parameters:

$$\begin{aligned}
\overline{B}_0 &= s_B(1.0) = 1.0, \\
C_0 &= s_C(1.0) = 1.0, \\
\Delta_0 &= \text{softplus}(2 \cdot 1.0 - 1) = \text{softplus}(1.0) \approx 1.31.
\end{aligned}$$

Discretise:

$$\overline{A}_0 = \exp(\Delta_0 \cdot \mathbf{A}) = \exp(-1.31) \approx 0.27.$$

Apply the recurrence with $h_{-1} = 0$:

$$\begin{aligned}
h_0 &= \overline{A}_0 \cdot 0 + \overline{B}_0 \cdot x_0 = 1.0, \\
y_0 &= C_0 \cdot h_0 = 1.0.
\end{aligned}$$

The model has stored the informative input.

Time step $t = 1$, with $x_1 = 0.0$. The noise token produces

$$\begin{aligned}
\overline{B}_1 &= s_B(0.0) = 0.0, \\
C_1 &= s_C(0.0) = 0.0, \\
\Delta_1 &= \text{softplus}(2 \cdot 0.0 - 1) = \text{softplus}(-1.0) \approx 0.31, \\
\overline{A}_1 &= \exp(-0.31) \approx 0.73.
\end{aligned}$$

Note that Δ_1 is small, which makes $\overline{A}_1$ close to 1 and the contribution of the new input $\overline{B}_1 \cdot x_1 = 0$. The recurrence gives

$$\begin{aligned}
h_1 &= 0.73 \cdot 1.0 + 0 = 0.73, \\
y_1 &= 0 \cdot 0.73 = 0.
\end{aligned}$$

The state is preserved (with mild decay), the noise input contributes nothing, and the output is zero. The model has correctly identified the noise token and kept its memory of the earlier informative input.

Time step $t = 2$, with $x_2 = 1.0$. The model returns to the informative regime:

$$\begin{aligned}
&\overline{B}_2 = 1.0, \quad C_2 = 1.0, \quad \Delta_2 \approx 1.31, \quad \overline{A}_2 \approx 0.27, \\
&h_2 = 0.27 \cdot 0.73 + 1.0 \cdot 1.0 \approx 1.20, \\
&y_2 = 1.0 \cdot 1.20 = 1.20.
\end{aligned}$$

The state has been updated with the new informative input, and the output reflects both the present input and the retained memory of the first one.

C.10 Summary of the Mechanism

The selection mechanism amounts to four small operations applied at every time step:

(a) three linear projections of the input x_t produce the candidate $\overline{\mathbf{B}}$, $\mathbf{C}$, and Δ for that time step;

(b) the broadcast operation copies the scalar Δ across the D channels of the model;

(c) softplus ensures Δ is positive;

(d) the discretisation rule converts $(\Delta, \mathbf{A}, \mathbf{B})$ into the time-varying parameters $(\overline{\mathbf{A}}_t, \overline{\mathbf{B}}_t)$ used in the recurrence.

The recurrence itself is unchanged from the LTI case. Only its parameters now depend on the input. The numerical example above shows that the result is a model that absorbs informative inputs and ignores uninformative ones, which is precisely the property an LTI SSM lacks.

D Tensor Shapes and GPU Computation

D.1 Following one clip through the detector

A tensor is an array with named dimensions. The input $(B, T, 3, H, W)$ separates examples, frames, channels, and spatial coordinates. The number of frames T is different from the scan length L. A scan can contain thousands of spatial positions even though the clip contains only five frames.

Table 11 traces the default configuration. Each scale contains target features with shape (B, C, H, W) and reference features with shape $(B, 4, C, H, W)$. The DAM returns four modified reference features, and the target is appended before pooling.

The six raw head channels represent four box values, objectness, and one class score. There are $64^2 + 32^2 + 16^2 = 5376$ candidate grid locations. Thresholding and NMS reduce these candidates to a variable number of detections. Raw scores are logits, so the decoder applies sigmoid before forming confidence values.

D.2 Why hidden-state storage can be large

At the finest feature scale, one scan has $L = 64 \times 64 = 4096$, $C = 128$, and state size $N = 32$. With batch size four, explicitly storing every state requires $4 \times 4096 \times 128 \times 32 = 67{,}108{,}864$ values. At four bytes per value, that is 256 MiB for this illustrative state history alone. A single running state has only $4 \times 128 \times 32$ values, or 64 KiB in FP32.

These storage estimates exclude gradients, other activations, scales, reference pairs, and temporary buffers. They are not measured peak GPU memory. They show why writing every state to device memory can be expensive.

Parallel prefix computation provides the basis for the scan (Blelloch 1990). A parallel scan exploits the fact that affine updates can be composed associatively. If one step maps h to $ah + b$ and the next maps it to $ch + d$, the pair maps h to $cah + cb + d$. Groups of updates can therefore be combined in a tree. A work-efficient scan has linear total work and logarithmic parallel depth under the usual model of parallel computation. Actual runtime also depends on hardware and memory access.

Kernel fusion keeps related operations together to reduce data transfers. Recomputation saves storage by recalculating intermediates during backpropagation. Both help explain the fused kernel, but neither replaces a measured detector benchmark.

The subsections below describe the same dimensions in general sequence-model terms. Following the Mamba paper, they write the channel count as D, whereas the rest of this report writes it as C. They also quote the common default state size $N = 16$; the recorded MOCID module uses $N = 32$.

D.3 What a Tensor Is

A *tensor* is a generalisation of a vector or matrix to any number of dimensions. A scalar is a 0-dimensional tensor (a single number), a vector is 1-dimensional (a list of numbers), and a matrix is 2-dimensional (a grid of numbers). A 3-dimensional tensor can be thought of as a stack of matrices, a 4-dimensional tensor as a stack of those stacks, and so on. The *shape* of a tensor is the tuple of its sizes along each dimension; for example, a tensor of shape $(3, 4, 5)$ contains $3 \times 4 \times 5 = 60$ numbers, organised so that there are three indices to specify which number is meant.

D.4 The Three Indices of a Sequence Input

The input to the SSM has three indices, written as a tensor of shape (B, L, D):

- **Batch (B).** During training, multiple independent examples are processed simultaneously to make better use of the GPU. The set of examples processed together is called a *mini-batch*, and B is its size. Each example is independent of the others. They share parameters but not hidden states.
- **Length (L).** Each example is a sequence of positions, and L is the number of positions. For a piece of text, this is the number of tokens. For an image processed sequentially (as in this project), it is the number of spatial positions in a feature map.
- **Channel (D).** At each position, the data is not a single number but a vector of D feature values. These features are the building blocks the network has learned to compute. For a colour image, the input layer would have $D = 3$ (red, green, blue). Deeper in the network, D might be 64 or 256, with each channel representing some learned visual feature.

D.5 The Project's Setting

In the satellite-imagery context, the three indices have the following meaning. The image is first passed through a convolutional backbone, producing a *feature map* of shape (H, W, D), where H and W are the spatial height and width of the feature map and D is the number of learned feature channels. To feed this feature map into an SSM, which expects a one-dimensional sequence, the spatial positions are *flattened* into a single dimension of length $L = H \times W$. The result is a tensor of shape (L, D) for one image, or (B, L, D) when B images are processed together as a mini-batch.

D.6 Where the Hidden State Lives

The SSM defined in Section 3.2 maintains a latent state vector $h \in \mathbb{R}^N$, where N is the state dimension (typically $N = 16$). The key point is that the SSM is applied *independently to each feature channel*. The same recurrence runs D times in parallel, each producing its own length-N hidden state. The full hidden state at a single position therefore has shape (D, N), which contains DN numbers. Across the full mini-batch and sequence, the hidden state tensor has shape (B, L, D, N), containing $BLDN$ numbers in total.

D.7 Why the Size Matters

The input tensor has shape (B, L, D) and contains BLD numbers. The hidden state tensor has shape (B, L, D, N) and contains $BLDN$ numbers. With a typical state size of $N = 16$, the hidden state is sixteen times larger than the input. This factor of N is what makes the naive implementation memory-intensive. Every element of the hidden state must be stored somewhere in GPU memory, and reading it back and forth between fast and slow memory regions becomes the bottleneck. The hardware-aware algorithm described in Appendix E avoids this cost by never materialising the full (B, L, D, N) tensor outside the GPU's fast on-chip memory.

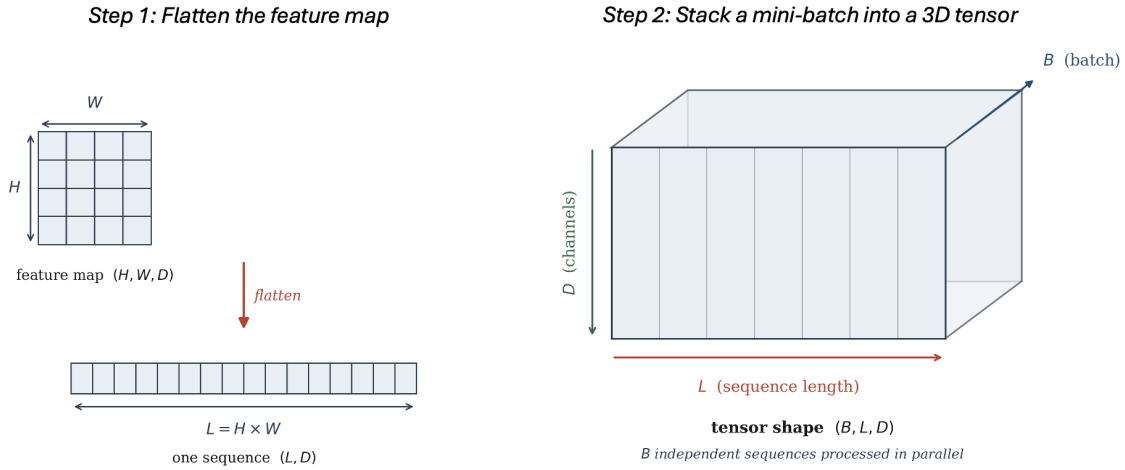


Figure 27: Tensor shapes encountered when feeding satellite imagery into an SSM. Step 1: a feature map of shape (H, W, D) produced by the convolutional backbone is flattened by reading its spatial positions in a fixed order, producing a one-dimensional sequence of length $L = H \times W$ in which each position carries D feature channels. Step 2: a mini-batch is formed by stacking B such sequences along an additional batch dimension, giving the input tensor of shape (B, L, D) that the SSM operates on. The same SSM is applied independently to each of the D channels within each sequence, producing a hidden state tensor of the larger shape (B, L, D, N).

E Why the Hardware-Aware Algorithm is Necessary

This appendix explains in plain terms why the selective scan needs the three techniques described in Section 3.5: parallel scan, kernel fusion, and recomputation.

E.1 Two Kinds of GPU Memory

A GPU has two kinds of memory, which differ enormously in speed and size. The first is the main memory, called *high-bandwidth memory* or HBM. It is large (tens of gigabytes) and stores the model's parameters, the input data, and any intermediate results that need to persist. The second is the on-chip memory, called *SRAM*. It is much faster than HBM, but each computational unit on the GPU has only a tiny amount of it (kilobytes to a few megabytes). The arithmetic units of the GPU can only act on data that is currently in SRAM, so any computation involves three steps. Load the relevant data from HBM into SRAM, do the arithmetic, and write the result back to HBM.

The crucial fact is that for most operations the loading and writing back is much slower than the arithmetic itself. The GPU's arithmetic units spend most of their time idle, waiting for data to arrive. Reducing the amount of data that has to travel between HBM and SRAM is therefore often the most effective way to speed up a computation (Dao et al. 2022).

E.2 Why a Naive Implementation is Slow

A naive implementation of the selective scan would treat the discretisation, the scan itself, and the multiplication by $\mathbf{C}$ as three separate steps, each loading its inputs from HBM and writing its outputs back to HBM. The intermediate tensors involved are large. The discretised parameters $\overline{\mathbf{A}}$ and $\overline{\mathbf{B}}$ alone have shape (B, L, D, N), where N is the SSM's state dimension (typically 16). For a typical batch size, sequence length, and channel count, this is several gigabytes of data being shuttled back and forth.

E.3 Trick One: Parallel Scan

The recurrence $h_t = \overline{\mathbf{A}}_t\, h_{t-1} + \overline{\mathbf{B}}_t\, x_t$ looks strictly sequential: h_t cannot be computed until h_{t-1} is known. A naive evaluation would therefore process the sequence one position at a time, leaving most of the GPU's parallel arithmetic units idle. The *parallel scan* algorithm, introduced by Blelloch (1990), exploits a structural property of the recurrence: the operation that takes one hidden state to the next can be combined associatively, in the same way that addition can be performed in any order. This means that the partial results can be combined as a tree rather than a chain. For a sequence of length L, the scan can be done in roughly $\log_2 L$ parallel steps instead of L sequential ones, recovering the GPU parallelism that was lost when the convolutional form was given up by the selection mechanism.

E.4 Trick Two: Kernel Fusion

A *kernel* in GPU terminology is a single function call that runs on the GPU. A typical operation may launch several kernels back to back, each loading inputs from HBM, performing some arithmetic, and writing outputs to HBM. Each launch incurs the cost of moving its inputs and outputs through the slow HBM–SRAM pathway.

Kernel fusion combines several operations into a single kernel so that the data only has to be loaded once and the intermediate results never leave SRAM. In the selective scan, this means that instead of three separate kernels (discretisation, scan, multiplication by $\mathbf{C}$) writing and re-reading the large (B, L, D, N) tensor each time, a single fused kernel loads the parameters $(\Delta, \mathbf{A}, \mathbf{B}, \mathbf{C})$ once from HBM, performs all three operations entirely in SRAM, and writes only

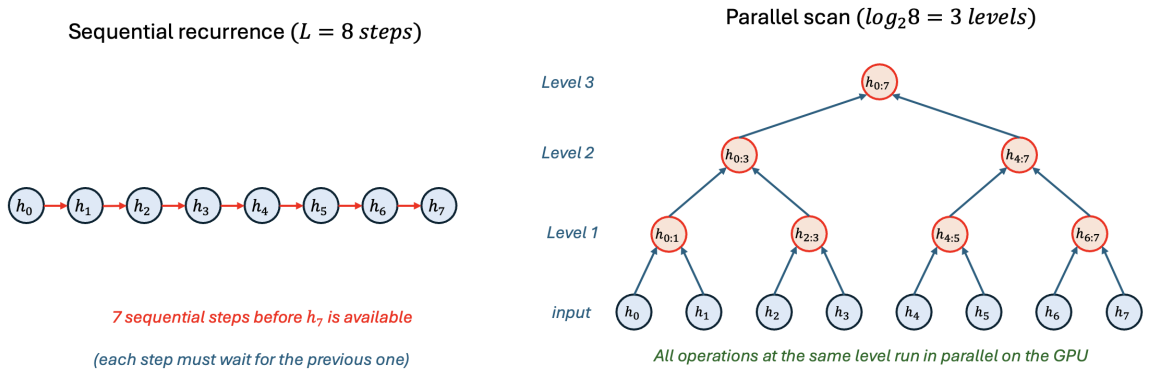


Figure 28: Sequential recurrence versus parallel scan for a sequence of length $L = 8$. Left: the naive evaluation produces the hidden states $h_0, h_1, \ldots, h_7$ in a single chain, with seven sequential steps required before h_7 becomes available. Each step must wait for the previous one to finish, so the GPU's parallel arithmetic units sit idle most of the time. Right: the parallel scan exploits the associativity of the recurrence to combine partial results in a binary tree. Each level of the tree can be evaluated entirely in parallel, and the full prefix $h_{0:7}$ is produced in $\log_2 8 = 3$ parallel steps. The right-hand structure recovers GPU parallelism along the sequence length without changing the final result.

the final output of shape (B, L, D) back to HBM. The intermediate (B, L, D, N) tensor is never written to HBM at all. This single change reduces the memory traffic by a factor of N, which translates to a 20–40× speedup in practice (Gu and Dao 2024).

E.5 Trick Three: Recomputation

Training a neural network requires *backpropagation*, which is a procedure for computing how each parameter should be updated to reduce the model's error. Backpropagation works by running the network forward, then running a second pass backward through the same operations to compute gradients with respect to each parameter. The crucial requirement of the backward pass is that the intermediate values from the forward pass must be available, because the gradients are computed in terms of them.

The naive way to satisfy this requirement is to store every intermediate value during the forward pass. For the selective scan, this would mean storing the full (B, L, D, N) hidden state tensor in HBM, which is exactly the cost that kernel fusion was meant to avoid. Mamba uses an alternative known as *recomputation* (sometimes called *gradient checkpointing*): the intermediate values are discarded at the end of the forward pass, and when the backward pass needs them they are recomputed from the inputs, which were saved. This trades a modest amount of additional arithmetic for a large saving in memory traffic, and is profitable because, as noted above, GPU arithmetic is cheap relative to data movement.

E.6 Summary

The three techniques work together. The parallel scan recovers the GPU parallelism that the selection mechanism took away. Kernel fusion keeps the large intermediate tensors out of slow HBM. Recomputation extends the same memory savings to the backward pass during training. The combined algorithm is what makes the selective SSM practical to train at scale, and what allows Mamba to match or exceed the throughput of highly optimised attention implementations such as FlashAttention (Dao et al. 2022).

F Reproduction Protocol and Evidence Package

F.1 Dataset audit

Table 13 records the fresh local count of each validation sequence. The count is based on unique frame paths in the supplied annotation lists and the five-frame continuity rule. It does not verify that the server used byte-identical manifests, so the exact run-specific lists should be archived.

The local training list contains 8,982 frames and 8,942 valid clips across ten different sequences. Neither split contains a duplicate frame path in this audit. The loader also accumulates labels if a frame is repeated in an annotation file, so manifest checks should be repeated whenever lists are regenerated. Record empty-label handling and filter invalid or zero-area boxes explicitly.

F.2 Reproducing an existing checkpoint

First identify the run's source commit, architecture class, input normalisation, and DAM options. Restore that configuration in an isolated checkout. A named profile does not identify the architecture by itself, because the same profile can exist on more than one branch.

Next verify the data manifest and checkpoint hash. Load with strict shape checks except for documented omissions when adding a fresh DAM to a stage-1 checkpoint. Confirm the number of scored clips before accepting any score. Then evaluate with the recorded BatchNorm policy, confidence floor, NMS, and AP convention. The `main.py` evaluator computes VOC AP50, so it does not recreate a server-side COCO result merely because the same checkpoint is used.

The following commands show the current interface. Paths in angle brackets must be replaced with the archived dataset and checkpoint paths. They are the commands a future team should run.

```
cd MOCID
python main.py -profile r0 params
python tools/check_dam.py -profile r0 -ckpt <stage1-checkpoint> -ap
python main.py -profile r0 eval -ckpt <checkpoint> -dam -perseq
```

For an older checkpoint requiring recalibration, add `-recal-bn 300` to the evaluation command and retain both the original and recalibrated scores. The check script requires a CUDA environment and the scan dependency.

F.3 Comparing new variants

Fix the dataset split, scored target frames, preprocessing, loss, optimiser groups, seed list, training budget, checkpoint rule, and evaluation procedure before training. Keep a matched control for each architectural question. If the baseline and candidate use different parameter counts, report the difference and consider a capacity control when the interpretation depends on it.

For every run, save the source commit and any uncommitted patch, resolved configuration, dataset-manifest hashes, seed, environment versions, hardware, and kernel source version. Save best and final checkpoints with their hashes, per-epoch losses, per-evaluation metrics, per-video results, raw detections, timings, and skipped/non-finite update counts. Checkpoint selection must be explicit and use the same rule across models.

For uncertainty, repeat complete runs with the same prespecified seed set across variants. Do not compute a confidence interval by treating nearby video frames or epochs as independent experiments. Per-video results and matched-seed differences are more informative about the limited data and training variability. If a held-out test set is available, freeze checkpoint and threshold choices before evaluating it.

F.4 Recommended inference timing protocol

A final timing comparison should use real images, one clip at a time, on an otherwise idle GPU, and it should report the median, the 95th percentile and peak GPU memory for every model. The loop below follows the method recommended by the project supervisor. It excludes data loading and preprocessing, warms the model up, and synchronises before and after each call, because GPU work runs asynchronously and a timer stopped without synchronising would measure only the launch of the work.

```
self.times = []  # inference time per image, excluding loading and preprocessing
with torch.no_grad():
    im, _ = self._load_sample(0)
    for _ in range(self.N_WARMUP):
        self.tester.model(im)
    self._sync()
    for idx in range(len(self.tester.test_data)):
        im, mask = self._load_sample(idx)
        self._sync()
        start = time.perf_counter()
        pred = self.tester.model(im)
        self._sync()
        end = time.perf_counter()
        self.times.append(end - start)
        self.evaluator.update(pred, mask)

def _sync(self):
    # wait until all work queued on the model's device has finished
    if self.device.type == 'cuda':
        torch.cuda.synchronize(self.device)
    elif self.device.type == 'mps':
        torch.mps.synchronize()
```

The times in Table 17 used a broader scope, including transfer, decoding and NMS, so they are not directly comparable with times from this loop.

G Project Record and Provenance

G.1 Inherited work and undergraduate development

Previous PhD students produced the original MOCID prototype, which provided the starting point for this work. The repository was imported from an archive on 20 August 2026, so file dates before that import do not indicate undergraduate authorship. The three variants are held on separate branches of the project repository, namely `dam-mambair` for MambaIR (commit `2027615`), `cfmw-test` for CFMW (commit `5cd8495`), and `mamba-tree` for MambaTree (commit `6f622ee`).

Table 19 summarises the dated technical record. Commit dates indicate when changes were recorded, which may differ from the date an experiment ran. Individual contributions are given in the Authorship and Contributions Statement.

Table 19: Milestones supported by the local history and experiment records.

Period in 2026	Recorded development
Before import	Previous PhD students' MOCID prototype.
20 August	Repository imported from an archive.
23 August	MOCID wrapper reorganised for detector registration.
Early September	Reproduction scaffold, Colab driver, and initial diagnostic work.
15 to 17 September	R0 stage-1 and plain Base results recorded.
20 September	Corrected Base experiments, weight-decay diagnosis, and revised learning-rate explanation.
21 September	DAM and training code review with measured handover failure.
29 to 30 September	Separate Base classes, common training entry point, profiles, handover repair, and BatchNorm recalibration.
6 October	MambaIR-inspired DAM and its completed training result.
7 to 8 October	Video demonstration tool and saved detection examples.
8 to 13 October	MambaTree implementation and comparison records on a separate branch.
15 to 19 October	CFMW implementation and comparison records on a separate branch.
Mid October	Composable decay and freezing profiles and additional MambaIR options.
21 October	GPU server becomes unavailable after a system update.

G.2 Levels of evidence

The strongest numerical evidence consists of saved evaluation logs whose summaries were recalculated for this report. These cover R0 stage 1, the Base runs, and all stage-2 DAM runs.

A second category consists of measurements retained in project notes, including the short decay ablations, the 89.52% and 87.07% ladder results, the BatchNorm recalibration checks, and the original stage-2 comparison. These are useful evidence, but not all of their original scripts or raw logs are in the repository.

The older `Eval/eval.ipynb` notebook computes box-based overlap and false-alarm-area proxies, which should not be equated with pixel-mask IoU or an operational false-alarm rate.

G.3 Diagnostics retained in project notes

Table 20 records the main measurements retained in notes. No missing measurement has been filled by estimation.

Table 20: Diagnostics retained in project notes, with their evidence limits. AP50 values are percentages.

Diagnostic	Recorded result	Source and limitation
Short Base decay tests	Epoch 14 AP50 of 28.94%, 73.35%, 85.80%	Corrected Base README. Raw ablation logs and scripts absent locally.
Full-frame ladder	Base 88.83%, FISTA 89.52%, original full model 87.07%	Code-review notes. COCO AP50 on 4,795 frames. Architecture changes are confounded.
Ladder handover	89.52% off, 67.44% on, 89.52% repaired	Code-review diagnostic. Original evaluation script not in the repository.
R0 original stage 2	Best 89.86%, final 88.53%	Experiment ledger and repaired-DAM README. VOC AP50 on 4,767 clips.
BN recalibration	Collapsed checkpoint 28.87% to 75.18%	Server guide. Exact checkpoint identity needs archiving.
R0 final recalibration	88.65% to 89.55%	Server guide. Full-stage recalibration differs from stage-2 downstream-only treatment.

G.4 Evidence records

Table 21 lists the repository records behind each part of the report, so that a future team can trace any result to its source. Paths are relative to the project repository, and the CFMW and MambaTree records are on their branches.

Table 21: Repository evidence records for each part of the report.

Report section	Records
Section 4, the detector	`MOCID/model.py`; `MOCID/base.py`; `MOCID/base_fista.py`; `MOCID/components/components.py`; `MOCID/components/dam.py`; `MOCID/FINDINGS.md`
Section 5.3, first complete run	`MOCID/results/R0/eval_log.csv`; `MOCID/EXPERIMENTS.md`; `MOCID/CODE-REVIEW.md`
Section 5.3, training and evaluation faults	`MOCID/results/Base-fixedwd-imagenet/README.md`; `MOCID/SERVER-GUIDE.md`; `MOCID/utils/utils.py`; `MOCID/train.py`
Section 5.6, handover	`MOCID/CODE-REVIEW.md`; `MOCID/results/R0-DAM-fixed/README.md`; `MOCID/tools/check_dam.py`
Sections 5.7 and 5.8, MambaIR	`MOCID/components/dam.py`; `MOCID/results/mambair-dam-1/eval_log.csv`; `MOCID/results/mambair-dam-1/README.md`
Sections 5.9 and 5.10, CFMW	`MOCID/components/cfm.py`; `MOCID/CFMW-DAM.md`; `MOCID/results/r0-cfm/eval_log.csv`; `MOCID/results/r0-cfm/README.md`
Sections 5.11 and 5.12, MambaTree	`MOCID/components/dam_tree.py`; `MOCID/third_party/TreeScan/README.md`; `MOCID/notebooks/mamba_tree.ipynb`; `MOCID/results/tree-d1-leah-1/eval_log.csv`; `MOCID/results/bidir-d1-leah-1/eval_log.csv`; `MOCID/results/tree-d8-leah-1/eval_log.csv`; `MOCID/results/tree-d1-leah-1/train_log.txt`
Section 7.8, timing and demonstrations	`MOCID/tools/animate.py`; `MOCID/results/demo-videos/README.md`

H Result Tables from the Literature

The tables below support the review in Section 2. They motivate a design direction but do not validate a model on DAUB or on the intended space-based application.

H.1 Detection Performance of Leading Object Detectors

TABLE I
PRECISION COMPARISON OF SEVERAL LEADING GENERIC OBJECT DETECTION ALGORITHMS ON THE MS COCO DATASET [17]

Model	Backbone	Dataset	Avg.Precision,IoU:			Avg.Precision,Area:			DOR
			0.5:0.95	0.5	0.75	S	M	L	
YOLOv2 [33]	DarkNet-19	COCO 2015 test-dev	21.6	44	19.2	5	22.4	35.5	**30.5**
RetinaNet [34]	ResNet-101	COCO 2015 test-dev	39.1	59.1	42.3	21.8	42.7	50.2	**28.4**
SSD513 [35]	ResNet-101	COCO 2015 test-dev	31.2	50.4	33.3	10.2	34.5	49.8	**39.6**
DSSD513 [36]	Residual-101	COCO 2015 test-dev	33.2	53.3	35.2	13	35.4	51.1	**38.1**
Faster R-CNN [30]	VGG	COCO 2015 test-dev	26.9	44.3	27.8	8.3	28.2	41.1	**32.8**
FPN [37]	ResNet-50	COCO 2014 minival	33.9	56.9	-	17.8	37.7	45.8	**28.0**
Mask R-CNN [14]	ResNet-101	COCO 2015 test-dev	38.2	60.3	41.7	20.1	41.1	50.2	**30.1**
Double-Head R-CNN [38]	ResNet-101	COCO 2014 minival	41.9	62.4	45.9	23.9	45.2	55.8	**31.9**
Cascade R-CNN [15]	ResNet-101	COCO 2015 test-dev	42.8	62.1	46.3	23.7	45.5	55.2	**31.5**
Hybrid Task Cascade [16]	ResNet-101-FPN	COCO 2015 test-dev	47.1	63.9	44.7	22.8	43.9	54.6	**31.8**

Figure 29: Detection performance of leading generic object detectors measured by Average Precision (AP) for small, medium and large objects, with corresponding Degree of Reduction (DOR) scores, showing the performance gap between small and large object detection. Reproduced from (G. Chen et al. 2020), Table 1.

H.2 Performance Comparison of 4 Pillars

TABLE IX
PERFORMANCE COMPARISON OF FOUR PILLOWS BASED ON MS COCO

Group	Model	Backbone	Dataset	Avg.Precision, IoU			Avg.Precision		
				0.5:0.95	0.5	0.75	S	M	L
Multi-scale representaion	MDSSD512 [63]	VGG16	COCO 2015 test-dev	30.1	50.5	31.4	13.9	-	-
	DR-CNN [64]	VGG16	COCO 2015 test-dev	36.5	59.7	38.2	18.6	39.3	47.7
	MRFSWSnet512 [66]	VGG16	COCO 2015 test-dev	33.1	53	34.7	17.5	37.1	47.9
	CADNet512 [72]	VGG16	COCO 2015 test-dev	30.5	50.8	32.1	11.4	35	44.8
Contextual information	ION [73]	VGG16	COCO 2015 test-dev	33.1	55.7	34.6	14.5	35.2	47.2
	DiCSSD300* [77]	VGG16	COCO 2015 test-dev	26.9	46.3	27.7	8.2	27.5	43.4
	SMN [78]	VGG16	COCO 2014 minval	31.6	52.2	33.2	14.4	35.7	45.8
Super-resolution	SOD-MTGAN [89]	ResNet101	COCO 2014 minval	41.5	62.5	45.4	25.1	44.6	54.1
Region-proposal	SSD-MCN [94]	VGG16	COCO 2015 test-dev	45.6	66.7	51.7	29.4	48.4	56.6
	PBLS-RPN [92]	VGG16	COCO 2015 test-dev	31.5	53.8	31.6	13.1	33.8	45.7

Figure 30: Average precision comparison of object detection models across four architectural groups (multi-scale representation, contextual information, super-resolution, and region-proposal) on MS COCO. Reproduced from (G. Chen et al. 2020), Table 1.

H.3 Performance Comparison of IRMamba Against Existing IRSTD Approaches

Method	Params ↓	Interface ↓	Type	NUDT-SIRST			IRSTD-1k			NUAA-SIRST		
				IoU ↑	P_d ↑	F_a ↓	IoU ↑	P_d ↑	F_a ↓	IoU ↑	P_d ↑	F_a ↓
IPI (Gao et al. 2013)	-	-	Traditional	30.93	81.98	17.99	27.92	81.37	16.18	1.09	87.05	30467
Top-Hat (Bai and Zhou 2010)	-	-	Traditional	22.40	89.90	174.1	10.06	75.11	1432	1.508	79.74	16456
MSLSTIPT (Sun, Yang, and An 2021)	-	-	Traditional	8.34	47.40	881	11.43	79.03	1524	1.080	0.052	8183
ACM (Dai et al. 2021a)	**0.52**	**0.010**	CNN-based	68.90	97.05	11.29	62.41	91.44	35.58	70.77	93.08	3.7
ALCNet (Dai et al. 2021b)	0.54	0.011	CNN-based	81.40	96.51	9.26	62.05	90.58	21.78	74.31	73.12	20.21
FC3-Net (Zhang et al. 2022b)	6.90	0.031	CNN-based	78.56	93.86	23.92	65.07	91.54	15.55	72.44	98.14	10.85
ISNet (Zhang et al. 2022c)	1.08	0.048	CNN-based	81.77	96.3	44.47	68.77	95.56	15.39	80.02	99.02	4.61
DNA-Net (Li et al. 2022)	4.7	0.023	CNN-based	88.19	98.62	9.00	69.38	93.3	11.66	79.26	98.48	2.3
UIU-Net (Wu, Hong, and Chanussot 2022)	50.54	0.032	CNN-based	92.19	97.77	15.44	69.96	93.98	22.07	78.25	97.45	2.296
Dim2Clear(Zhang et al. 2023)	5.71	0.024	CNN-based	81.37	96.23	9.17	66.34	93.75	20.93	77.29	99.10	6.72
GCI-Net (Zhang et al. 2024b)	0.71	0.021	CNN-based	92.43	98.25	8.96	67.75	93.89	12.84	78.81	99.34	2.108
RKformer (Zhang et al. 2022a)	29.00	0.065	Hybrid	92.25	96.86	6.58	64.12	93.27	18.65	77.24	99.11	1.58
IAANet (Wang et al. 2022)	14.05	0.092	Hybrid	90.22	97.26	8.32	66.25	93.15	14.20	74.22	93.53	22.70
SCTransNet (Yuan et al. 2024)	11.19	0.048	Hybrid	94.09	98.62	4.29	68.03	93.27	10.74	77.50	96.95	13.92
IRMamba (Ours)	10.51	0.044	Mamba	**95.18**	**99.26**	**1.309**	**70.04**	**95.81**	**5.92**	**80.2**	**99.36**	**0.887**

Table 1: Comparison with existing IRSTD approaches on the NUDT-SIRST, IRSTD-1k, and NUAA-SIRST datasets. The evaluation metrics are Params(M), Interface(s), IoU (10^{-2}), P_d (10^{-2}), and F_a (10^{-6}).

Figure 31: Performance comparison of traditional, CNN-based, hybrid, and Mamba-based IRSTD methods across three benchmark datasets. Reproduced from (M. Zhang et al. 2025a), Table 1.

H.4 ImageNet VID Detection Performances

Table 1
Quantitative experimental results of different approaches on ImageNet VID.

Model	AP_{50}	AP_{75}	$AP_{50:95}$	AP_S	AP_M	AP_L	FPS
SELSA [6]	82.3	58.2	52.3	11.4	28.8	57.9	6.4
RDN [24]	81.8	60.1	53.4	8.5	27.4	59.6	7.1
MEGA [7]	82.9	59.2	53.2	9.1	29.4	59.1	5.8
TDViT-B$^+$ [25]	84.7	64.2	56.1	10.0	29.9	61.8	12.2
EOVOD [26]	79.8	59.5	54.1	10.5	28.3	60.1	20.4
TransVOD Lite [8]	83.7	-	66.2	**14.7**	35.1	67.3	29.6
RMFA [27]	83.6	58.9	53.2	10.1	28.6	59.3	7.7
STPN [28]	86.1	71.6	63.6	12.1	33.8	70.3	–
BoxMask [29] + TROI [30]	80.7	57.8	51.8	–	–	–	7.2
FAQ [31]	81.7	66.9	60.1	13.2	33.1	66.9	12.3
SSGAN [32]	77.2	64.2	57.0	13.0	30.1	63.6	14.1
JSTFE-Net [33]	80.6	57.7	51.2	–	–	–	17.9
YOLOV++ (SwinT) [9]	83.9	69.5	63.2	12.9	33.2	69.8	**45.8**
YOLOV++ (SwinB) [9]	87.4	72.7	66.2	12.4	38.0	72.8	34.8
IntSTR [34]	85.3	70.7	63.1	**14.7**	38.2	72.5	–
TSCD [35]	79.8	65.3	58.5	13.4	31.2	65.0	3.4
Ours (SwinT)	85.6	70.8	64.4	14.2	34.4	70.9	11.2
Ours (SwinB)	**88.5**	**73.5**	**66.9**	12.7	**38.2**	**73.5**	7.0

Figure 32: Comparison of video object detection methods on ImageNet VID across AP metrics and inference speed. Reproduced from (Yao et al. 2026), Table 1.

Table 2
The detection performance under different motion speeds.

Model	AP_{50}	AP_{slow}	AP_{medium}	AP_{fast}	FPS
FGFA [12]	76.3	83.5	75.8	57.6	1.2
MEGA [7]	82.9	89.4	81.6	62.7	5.8
CFA [36]	85.0	89.4	83.9	70.4	1.1
TM-VoD [37]	83.1	87.9	81.9	66.7	–
GMLCN [38]	78.6	87.1	76.0	54.4	25.2
IFFNet [39]	79.7	87.5	78.7	60.6	–
TDViT-T$^+$ [25] + SELSA [6]	84.5	88.9	84.3	69.4	15.7
PAIAM [40]	84.6	89.9	83.1	69.3	–
CDANet [41]	85.4	90.2	84.5	70.7	12.4
ClipVID [42]	85.8	90.8	84.5	72.3	25.1
STFA + RR [14]	74.2	80.7	67.5	51.4	17.8
YOLOV + + (SwinT + REPP) [9]	86.0	91.1	84.7	72.0	**39.7**
YOLOV + + (SwinB + REPP) [9]	88.9	91.4	89.7	78.9	32.4
IMC-Det [11]	85.5	90.6	84.5	70.7	11.6
DGC-Net [43]	86.3	91.2	85.5	71.6	11.1
TSCD [35]	83.2	87.4	82.1	67.5	–
Ours (SwinT + REPP)	87.2	91.7	86.7	75.7	10.6
Ours (SwinB + REPP)	**89.7**	**92.6**	**90.5**	**79.1**	6.6

Figure 33: Detection performance comparison across slow, medium, and fast motion speeds on ImageNet VID. Reproduced from (Yao et al. 2026), Table 2.

The IRMamba comparison concerns infrared target segmentation metrics and must remain distinct from the bounding-box AP50 used in the MOCID experiments. The ImageNet VID results report AP for size and speed groups; AP at a fixed IoU should be distinguished from AP averaged across IoU thresholds when these are quoted.

The evidence from these sources has limits. A 3×3 target in a 128×128 image is an illustration of the application and not a description of the DAUB split. Mamba's linear scan complexity does not guarantee small-target accuracy, and full-attention scaling does not describe every Transformer variant. These distinctions are applied throughout the analysis in this report.

I Future Work Plan and Outstanding Measurements

I.1 Priority order

Table 22 expands the recommendations of Section 8 in the order that best reduces uncertainty, with the evidence each task should produce.

Table 22: Recommended future work and the evidence each task should produce.

Order	Task	Deliverable	Question resolved
1	Common checkpoint evaluation	One frame list, metric and recalibration setting, with saved predictions	How do existing models compare under one scoring procedure?
2	Remaining inference measurements	Timing, hit rates and per-video results for all variants (Table 23)	What does each variant cost, and where does it succeed or fail?
3	Redesigned displacement fusion	New pooling design with a DAM-off control in the same run	Can a DAM add information rather than replace it?
4	Repeated runs	Prespecified seeds, individual scores, mean and spread for all variants	Are the observed differences consistent?
5	Corrected-recipe comparison	All variants from one stage-1 checkpoint with `wdfix` and `freeze`	Do the findings persist under corrected training?
6	Variant-specific options	One-factor tests of LayerNorm, token grid and depth, and state size	Which design choices matter within each variant?
7	Capacity and architecture controls	FPN-controlled Base and width-controlled DAM comparisons	How much is attributable to the named building block?
8	Benchmark and application evaluation	Adapter equivalence check and completed scenario results	Does the result transfer to the intended framework and domain?

I.2 Outstanding measurements

Table 23 records which measurements were completed before the GPU server became unavailable on 21 October 2026. Before filling the gaps, a future team should fix the frames, AP calculation, threshold and checkpoint rule, seeds, GPU and timing scope, following Appendix F.

Table 23: Status of measurements for each model. Done marks a recorded measurement, and Outstanding marks one left to complete using Appendix F.

Model	AP50 and F1	Inference time	Hit rates	Parameters
Corrected original DAM	Done	Done	Done	Done
MambaIR-inspired DAM	Done	Done	Done	Done
CFMW-inspired DAM	Done	Outstanding	Outstanding	Done
MambaTree-inspired DAM	Done	Outstanding	Outstanding	Done

J Project Technical Objectives

This appendix gives the full specification of the technical objectives defined in the Project Plan. Section 1.4 and Section 9 assess their completion.

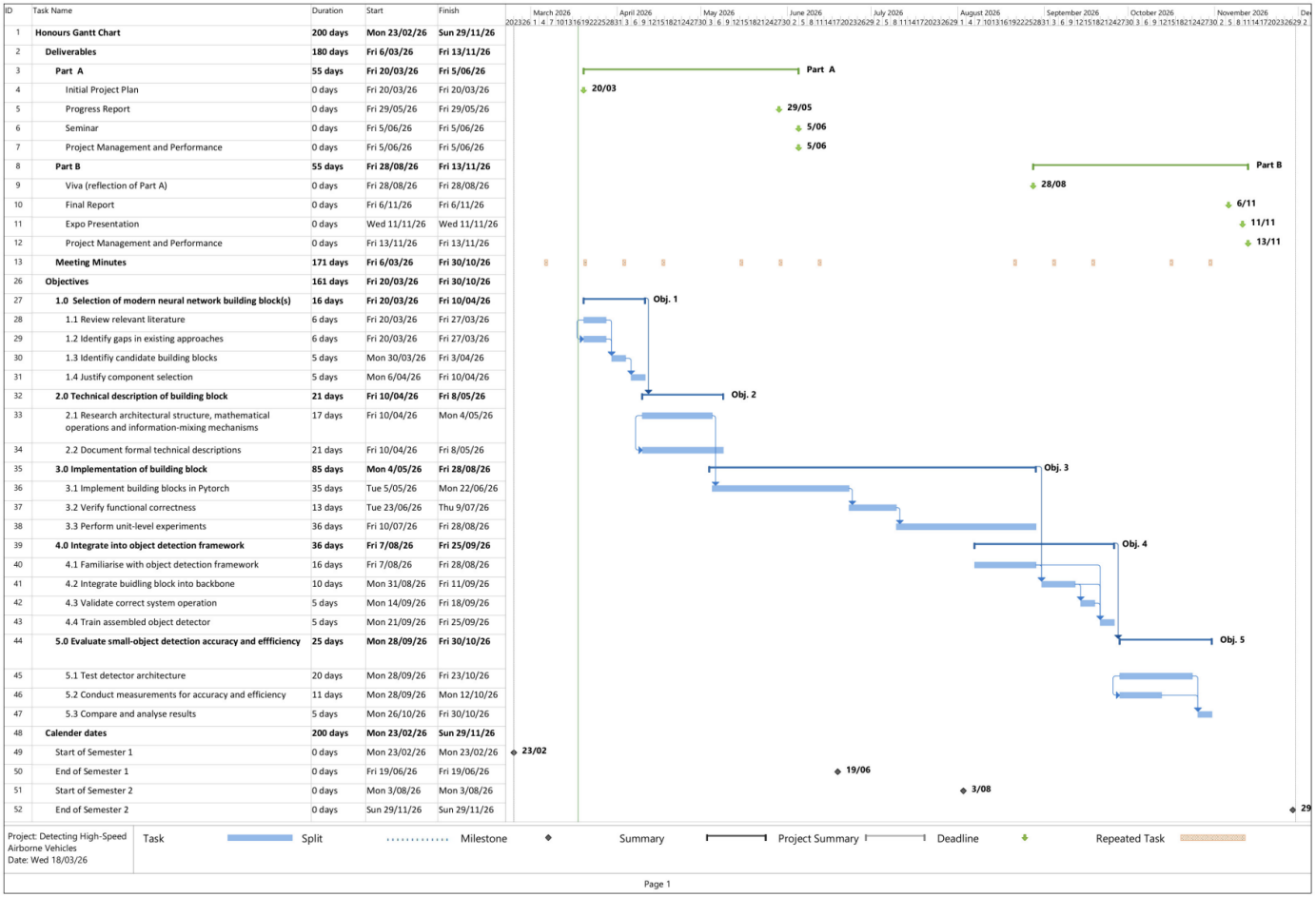
Table 24: Technical objectives for the investigation of a Mamba-based building block, specifying evaluation criteria and deliverables for each objective.

ID	Objective Description	Specification	Deliverable / Outputs
1	To identify and justify, through a systematic literature review, a modern neural network building block suitable for integration into object detection backbones, based on reported detection performance, computational efficiency, and availability of open-source implementations.	• Systematic literature review covering recent (past 5 years) neural network building blocks relevant to small-object detection. • Evaluation metrics include detection performance (precision, recall, mAP), computational efficiency (FLOPs, inference time, and memory footprint measured by the size of network parameters/weights), and the availability of open-source implementations.	• Ranked list of candidate building blocks with performance and efficiency comparison tables. • Justification report explaining the selected building block. • References to open-source repositories and implementations.
2	To develop a formal technical description of the selected building block, including its architectural structure, mathematical operations, and information-mixing mechanisms relevant to small-object feature representation.	• Develop detailed architectural diagrams showing input/output dimensions, convolution/attention layers, and feature-mixing mechanisms. Mathematical formulation of operations (e.g., convolution, attention, normalisation and information flow). • Analysis of suitability for small-object feature extraction and integration into object detection backbones (ResNet, DNA-Net and U-Net).	• Technical report with architecture diagrams and equations. • Annotated mathematical description of the selected building block. • Summary of its information-mixing mechanism and expected impact on small object detection.
3	To implement the selected building block in PyTorch and verify its functional correctness and computational behaviour through controlled unit-level experiments.	• Implement the building block as a reusable PyTorch module. • Unit-level tests to verify input/output consistency and computational behaviour. Benchmark computational cost (inference time, memory usage) on representative input tensors.	• PyTorch module implementing the building block. • A set of unit tests and test results demonstrating correctness. • Performance benchmark report for memory and computation.

ID	Objective Description	Specification	Deliverable / Outputs
4	To integrate the building block into a modular backbone within an object detection framework while keeping all other architectural components fixed, enabling controlled evaluation of its impact on detection performance.	• Integrate the building block into a fixed backbone, keeping other layers unchanged to isolate its impact. • Enable configuration for swapping in/out the building block for comparative experiments. • Ensure compatibility with the detection pipeline (data loaders, training loops, and evaluation scripts).	• Updated modular backbone code with the building block integrated. • Configurable experimental setup for ablation studies. • Documentation of integration and usage instructions.
5	To quantitatively evaluate the resulting object detection system in terms of detection accuracy, computational efficiency, and memory footprint under representative hypersonic aircraft and missile-like target scenarios.	• Benchmark detection accuracy using relevant datasets (these will be synthetic datasets). • Measure computational efficiency (FLOPs, inference time) and memory usage (size of network weights). • Compare performance against baseline without the new building block.	• Performance evaluation reports (tables, graphs, metrics). • Comparative analysis highlighting improvements or limitations. • Dataset and scripts used for reproducibility.

K Project Gantt Chart

This appendix presents the project Gantt chart. Section 8 and Appendix I set out the recommended work beyond this timeline.



ID	Task Name	Duration	Start	Finish
1	**Honours Gantt Chart**	**200 days**	**Mon 23/02/26**	**Sun 29/11/26**
2	**Deliverables**	**180 days**	**Fri 6/03/26**	**Fri 13/11/26**
3	**Part A**	**55 days**	**Fri 20/03/26**	**Fri 5/06/26**
4	Initial Project Plan	0 days	Fri 20/03/26	Fri 20/03/26
5	Progress Report	0 days	Fri 29/05/26	Fri 29/05/26
6	Seminar	0 days	Fri 5/06/26	Fri 5/06/26
7	Project Management and Performance	0 days	Fri 5/06/26	Fri 5/06/26
8	**Part B**	**55 days**	**Fri 28/08/26**	**Fri 13/11/26**
9	Viva (reflection of Part A)	0 days	Fri 28/08/26	Fri 28/08/26
10	Final Report	0 days	Fri 6/11/26	Fri 6/11/26
11	Expo Presentation	0 days	Wed 11/11/26	Wed 11/11/26
12	Project Management and Performance	0 days	Fri 13/11/26	Fri 13/11/26
13	**Meeting Minutes**	**171 days**	**Fri 6/03/26**	**Fri 30/10/26**
26	**Objectives**	**161 days**	**Fri 20/03/26**	**Fri 30/10/26**
27	**1.0 Selection of modern neural network building block(s)**	**16 days**	**Fri 20/03/26**	**Fri 10/04/26**
28	1.1 Review relevant literature	6 days	Fri 20/03/26	Fri 27/03/26
29	1.2 Identify gaps in existing approaches	6 days	Fri 20/03/26	Fri 27/03/26
30	1.3 Identify candidate building blocks	5 days	Mon 30/03/26	Fri 3/04/26
31	1.4 Justify component selection	5 days	Mon 6/04/26	Fri 10/04/26
32	**2.0 Technical description of building block**	**21 days**	**Fri 10/04/26**	**Fri 8/05/26**
33	2.1 Research architectural structure, mathematical operations and information-mixing mechanisms	17 days	Fri 10/04/26	Mon 4/05/26
34	2.2 Document formal technical descriptions	21 days	Fri 10/04/26	Fri 8/05/26
35	**3.0 Implementation of building block**	**85 days**	**Mon 4/05/26**	**Fri 28/08/26**
36	3.1 Implement building blocks in Pytorch	35 days	Tue 5/05/26	Mon 22/06/26
37	3.2 Verify functional correctness	13 days	Tue 23/06/26	Thu 9/07/26
38	3.3 Perform unit-level experiments	36 days	Fri 10/07/26	Fri 28/08/26
39	**4.0 Integrate into object detection framework**	**36 days**	**Fri 7/08/26**	**Fri 25/09/26**
40	4.1 Familiarise with object detection framework	16 days	Fri 7/08/26	Fri 28/08/26
41	4.2 Integrate buidling block into backbone	10 days	Mon 31/08/26	Fri 11/09/26
42	4.3 Validate correct system operation	5 days	Mon 14/09/26	Fri 18/09/26
43	4.4 Train assembled object detector	5 days	Mon 21/09/26	Fri 25/09/26
44	**5.0 Evaluate small-object detection accuracy and effficiency**	**25 days**	**Mon 28/09/26**	**Fri 30/10/26**
45	5.1 Test detector architecture	20 days	Mon 28/09/26	Fri 23/10/26
46	5.2 Conduct measurements for accuracy and efficiency	11 days	Mon 28/09/26	Mon 12/10/26
47	5.3 Compare and analyse results	5 days	Mon 26/10/26	Fri 30/10/26
48	**Calender dates**	**200 days**	**Mon 23/02/26**	**Sun 29/11/26**
49	Start of Semester 1	0 days	Mon 23/02/26	Mon 23/02/26
50	End of Semester 1	0 days	Fri 19/06/26	Fri 19/06/26
51	Start of Semester 2	0 days	Mon 3/08/26	Mon 3/08/26
52	End of Semester 2	0 days	Sun 29/11/26	Sun 29/11/26

Figure 34: Project Gantt Chart Outlining the timeline

L Work Breakdown Structure

This appendix breaks each objective of Section 1.4 into its tasks.

Selection of modern neural network building block(s)

1. To identify and justify a modern neural network building block suitable for object detection backbones

- **1.1 Review relevant literature**
  - 1.1.1 Identify key papers on object detection backbones
  - 1.1.2 Review recent neural network building blocks (CNN, Transformer, hybrid)
  - 1.1.3 Analyse detection performance metrics (mAP, precision, recall)
  - 1.1.4 Analyse computational efficiency (FLOPs, inference time and memory footprint measured by the size of network parameters/weights)
  - 1.1.5 Review availability of open-source implementations
- **1.2 Identify gaps in existing approaches**
  - 1.2.1 Compare methods for small-object detection
  - 1.2.2 Identify limitations in feature representation
  - 1.2.3 Identify trade-offs between accuracy and efficiency
- **1.3 Identify candidate building blocks**
  - 1.3.1 Shortlist promising architectures
  - 1.3.2 Evaluate compatibility with detection backbones
  - 1.3.3 Compare implementation complexity
- **1.4 Justify component selection**
  - 1.4.1 Select final building block
  - 1.4.2 Document justification based on literature and metrics
  - 1.4.3 Verify availability of implementation resources

Technical description of building block

2. To develop a formal technical description of the selected building block

- **2.1 Research architectural structure / operations and develop mathematical formulation**
  - 2.1.1 Analyse layer structure and data flow
  - 2.1.2 Identify key operations (convolution, attention, etc.)
  - 2.1.3 Study information-mixing mechanisms

  - 2.1.4 Derive equations for core operations
  - 2.1.5 Define input/output tensor relationships
  - 2.1.6 Analyse computational complexity
- **2.2 Document technical description**
  - 2.2.1 Create architecture diagrams
  - 2.2.2 Write formal technical documentation
  - 2.2.3 Review and refine description

Implementation of building block

3. To implement the selected building block and verify its behaviour

- **3.1 Implement building block in PyTorch**
  - 3.1.1 Set up development environment
  - 3.1.2 Implement module structure
  - 3.1.3 Implement forward pass
  - 3.1.4 Integrate with test inputs
- **3.2 Verify functional correctness**
  - 3.2.1 Validate input/output dimensions
  - 3.2.2 Verify gradient flow (backpropagation)
  - 3.2.3 Debug implementation errors
- **3.3 Perform unit-level experiments**
  - 3.3.1 Benchmark inference time
  - 3.3.2 Measure FLOPs
  - 3.3.3 Measure memory footprint
  - 3.3.4 Document performance results

Integration into object detection framework

4. To integrate the building block into an object detection framework

- **4.1 Familiarise with object detection framework**
  - 4.1.1 Select baseline framework (e.g., YOLO)
  - 4.1.2 Review architecture and pipeline
  - 4.1.3 Set up training environment
- **4.2 Integrate building block into backbone**

  - 4.2.1 Modify backbone architecture
  - 4.2.2 Replace baseline component with new block
  - 4.2.3 Ensure compatibility with detection head
- **4.3 Validate system operation**
  - 4.3.1 Run forward-pass validation
  - 4.3.2 Debug integration issues
  - 4.3.3 Confirm stable training behaviour
- **4.4 Train assembled object detector**
  - 4.4.1 Prepare dataset
  - 4.4.2 Configure training parameters
  - 4.4.3 Train model and monitor performance

Evaluation of detection accuracy and efficiency

5. To evaluate the object detection system performance

- **5.1 Test detector architecture**
  - 5.1.1 Run inference on test dataset
  - 5.1.2 Generate detection outputs
  - 5.1.3 Visualise predictions
- **5.2 Conduct measurements for accuracy and efficiency**
  - 5.2.1 Compute mAP, precision, and recall
  - 5.2.2 Measure inference time
  - 5.2.3 Measure memory usage
- **5.3 Compare and analyse results**
  - 5.3.1 Compare against baseline model
  - 5.3.2 Analyse improvements and trade-offs
  - 5.3.3 Generate plots and tables
  - 5.3.4 Document findings